

C R E D E N C E

CREDIT RISK INTELLIGENCE PLATFORM

Full Project Walkthrough

The pipeline explained, every file of real source code, and an interview Q&A set ordered easiest to hardest.

- | | |
|--------|--|
| PART 1 | The project explained — data, architecture, methodology |
| PART 2 | 21 pipeline files in full, real source, data cleaning through explainability |
| PART 3 | 14 CREDENCE core files in full + 11 more summarised |
| PART 4 | 35 interview questions with full answers, easiest to hardest |

Code is the real, current source — read fresh from the repository, not retyped.

The Project, Explained

What this project is

A credit-risk pipeline built on a real, public dataset (Lending Club's "accepted loans," wordsforthewise Kaggle mirror), plus a dashboard (CREDENCE) that presents its output. The pipeline estimates the probability a loan defaults (PD), combines that with exposure (EAD) and loss severity (LGD) into an expected-loss figure, backtests an approval policy against what actually happened to those loans, checks the policy for disparate impact across income/region/home-ownership, and reports how stable the model's own population looks between an older and a newer cohort. Every real figure in the dashboard traces back to one of these computations — nothing is invented at the presentation layer.

The dataset

640,919 real 36-month personal loans, issued January 2012 through February 2016, each one already resolved (paid off or charged off) as of the 2018Q4 snapshot. The window isn't arbitrary: a preflight check (`sql/preflight_maturity.sql`) measures, month by month, what share of that month's loans have reached a terminal status, and the window is drawn where at least 98% have resolved — so the model is trained and tested on loans whose fate is actually known, not loans still open and therefore censored. Default is defined precisely in `sql/03_features_outcome.sql`: `loan_status` IN ('Charged Off','Default') is a default, 'Fully Paid' is not, everything else (still open) is excluded from modelling entirely.

The five-stage architecture

`raw` -> `stage` -> `features/outcome` -> `train/score` -> `marts`, each stage a DuckDB table or view, each one readable independently. Stage 1 loads the raw CSV schema-agnostically. Stage 2 (`sql/02_staging.sql`) types everything, deduplicates, normalises categories, and applies the maturity/quality filters. Stage 3 (`sql/03_features_outcome.sql`) is the most important architectural decision in the project: it splits the cleaned data into three DISJOINT marts — `mart_loan_features` (only origination-time fields the model is allowed to see), `mart_loan_benchmark` (grade/sub_grade/int_rate — Lending Club's own risk output, used only to benchmark against, never fed to the model), and `mart_loan_outcomes` (the label and cash-flow fields, used only for training targets and backtesting). A loan-status field or a post-origination payment field physically cannot reach the model by accident, because it was never placed in the table the model reads from. `src/features.py` adds a second, independent check — a runtime assertion that a FORBIDDEN column list never appears in the modelling frame — as defence in depth on top of the schema separation, not instead of it.

The model, and why this one

`HistGradientBoostingClassifier` (scikit-learn), isotonic-calibrated, trained on roughly 33 origination-time features — bureau attributes (FICO, DTI, revolving utilisation, inquiry recency, trade-line history), application fields (loan amount, purpose, home ownership), and two engineered ratios (log annual income, loan-to-income). Gradient boosting over logistic regression because it captures non-linear interactions among bureau variables without manual interaction terms, and because the project runs both and reports the honest comparison: HGB 0.691 AUC vs logistic 0.680 vs Lending Club's own grade alone at 0.669 — a real, if modest, edge, tested for significance with a DeLong test ($z=19.6$, $p=3e-85$) rather than just asserted from a bigger number. Isotonic calibration matters because the whole downstream system — Expected Loss, the backtest, the fairness check — depends on PD meaning an actual probability, not just a ranking score.

Out-of-time validation, not k-fold

Loans issued before 2015-01-01 are the training set (306,462 loans); loans issued from 2015-01-01 onward are the test set (333,721 loans) — a genuinely temporal split, not a random shuffle. This is the single most important validation choice in the project: a model that only looks good on loans from the same time period it was trained on tells you nothing about how it performs on the next loan that walks through the door. Every headline number in this project — the 0.691 AUC, the calibration deciles, the backtest curve — is the out-of-time TEST result, not the easier training-set number (train AUC is a noticeably higher 0.714, which is expected and reported, not hidden).

LGD and Expected Loss

LGD (loss given default) is estimated from real recovery data, not assumed: $1 - (\text{principal repaid} + \text{post-charge-off recoveries}) / \text{funded amount}$, over the 40,596 loans that actually charged off in the training period — giving 49.95% against a textbook-assumed 45%. That number is then segmented by Lending Club grade and Buhlmann credibility-weighted (src/lgd.py): a grade with abundant charged-off history (grade C, 13,090 loans) mostly trusts its own recovery rate; a grade with almost none (grade G, 73 loans) shrinks most of the way back to the flat portfolio figure, because a raw estimate from 73 loans is not trustworthy on its own. This segmented LGD is what actually feeds Expected Loss (sql/06_marts.sql: $EL = PD \times EAD \times LGD$, computed once and reused everywhere downstream), not a supplementary side-statistic. EAD is a real, stated limitation: it's the funded amount at origination, not a time-varying outstanding balance, so it overstates exposure for a loan that's already made two years of payments — documented, not hidden, in reports/memo.md.

The policy backtest — real validation, with a real limitation

src/backtest.py answers a specific question: if we'd approved loans using a given PD cut-off, what would ACTUALLY have happened, using the test loans' real payment history — not the model's own predictions about itself. It sweeps 60 cut-offs from 3% to 40% and computes, at each one, the realised profit (actual cash in minus funding cost minus servicing cost) versus the model's own projected profit. The profit-maximising cut-off from the model (18.7% PD) and the one that would actually have maximised realised profit (17.4%) are close — following the model's choice instead of the perfect one costs about \$180K on an \$8B book, which is a strong result. The real limitation, stated plainly: every loan in this dataset was actually approved by Lending Club historically, so there's no outcome data for anyone who would have been rejected under a stricter hypothetical policy. Tightening the cut-off is validated by real data; loosening it walks toward a population this backtest has never observed. This is why the word used throughout is 'backtest' or 'threshold sweep,' never 'optimization' — it's a brute-force evaluation of one lever across 60 points, not a solver finding a true optimum over multiple constraints.

Fairness — a real, bounded check

Lending Club's data has no protected-class fields (race, sex, age, national origin), so a true ECOA fair-lending test is not possible on this dataset — stated directly rather than faked. What IS real: a 4/5ths-rule adverse-impact check on the proxy dimensions the data does have (income band, census region, home ownership). Income band and home ownership both fail (AIR 0.42 and 0.68, both well under the 0.80 threshold); region passes (0.96). Both failures are backed by a nonparametric bootstrap confidence interval, Bonferroni-corrected for testing three dimensions at once (src/fairness.py) — the upper bound of each CI stays below 0.80, so this isn't noise from a small sample, it's a real, statistically defensible finding. Notably, the approved-book default rate stays close across every group in every check — the gap is in who gets approved, not in how those approved loans perform.

Monitoring, explainability — built honestly, not faked

Two capabilities a real risk team would want, built to the extent the data honestly supports and labelled precisely where it doesn't: src/monitor.py computes a real Population Stability Index between the pipeline's own train and test

cohorts — every feature reads stable, which is itself informative (the AUC softening from train to test is genuine model seasoning, not a population shift in disguise) — but it's retrospective, not a live monitor, because there's no scored production traffic to watch continuously. `src/explain.py` computes a REAL per-loan sensitivity on the actual trained model for a sample of real test loans: each feature is reset to the population's typical value one at a time and the resulting PD change is measured — a genuine, correct, single-order attribution, explicitly not an exact Shapley-value (SHAP) decomposition and not a production per-decision reason-code system. Both are exactly as far as this project's data and scope honestly go.

The dashboard: CREDENCE

A static React/TypeScript single-page app, zero backend, zero runtime API calls — every real number is transcribed at build time from the pipeline's JSON artifacts into `src/data/real.ts`, architecturally separated from `src/data/demo.ts` (seeded, clearly-labelled synthetic data used only where the pipeline has no row-level data — individual borrower applications, monthly time series). The separation isn't just a convention: the two files export different TypeScript types, so a REAL number and a DEMO number can't be silently swapped by mistake. Every synthetic panel carries a visible 'DEMO DATA' badge; every real one that needs it carries a 'REAL' tag; the Data/Model page has a full ledger stating exactly which panel is which and a 'Known limitations' section stating plainly what this build deliberately does not fake.

The Pipeline — Every File

In real pipeline order: raw ingestion, cleaning, the leakage-safe feature/outcome split, modelling, expected loss, then every analysis stage (backtest, fairness, monitoring, insight, explainability), orchestration, and tests.

config.yaml

Every assumption and threshold in the pipeline in one place: the maturity window, the out-of-time split date, model hyperparameters, LGD mode, and the simulator's economic assumptions (cost of funds, servicing cost, PD/FICO policy). Downstream code reads these values rather than hardcoding them — though the column SCHEMA itself (which fields exist, what they're named) is not config-driven, only these thresholds are; that's a real, stated scope limit, not an oversight.

```
# All assumptions and paths live here. Nothing hard-coded in the pipeline.

data:
# Lending Club "accepted" loans. Kaggle: wordsforthewise/lending-club
raw_csv: data/accepted_2007_to_2018Q4.csv.gz
kaggle_mirror: "wordsforthewise/lending-club"
snapshot_label: "2018Q4" # data goes through 2018-Q4; used only for documentation

maturity:
term_months: 36 # 36-month loans only in v1 (see spec §2.1)
terminal_fraction_threshold: 0.98 # an issue month is "mature" if >= this share reached a terminal status
# Pre-flight (2260701 rows, wordsforthewise mirror): 36-month loans are >=98% resolved
# through 2016-02 (2016-03 drops to 0.87). Data max issue = 2018-12, so 34 months back
# lands exactly on 2016-02. Window: 2012-01 .. 2016-02, ~640k terminal loans, DR ~14%.
min_months_since_issue: 34
issue_year_min: 2012 # ignore the thin, pre-standardised early years
issue_year_max: 2016 # the month-diff rule trims the tail of 2016

split:
oot_cutoff: "2015-01-01" # train: issue_d < cutoff ; test: issue_d >= cutoff (within mature window)

model:
# Audit finding (fixed): this used to list "Late (120+ days)" as a positive-label status,
# but that status is NOT terminal (sql/02_staging.sql: is_terminal = loan_status IN
# ('Fully Paid','Charged Off','Default')) — a loan still Late is still open, not yet
# resolved, so it's correctly excluded from modeling (censored), not labeled a default.
# These two keys are informational only and are NOT read by any SQL/Python at runtime —
# the authoritative label definition lives in sql/03_features_outcome.sql (default_flag).
# Keeping the values here consistent with that SQL rather than aspirational/unused.
target_positive_statuses: ["Charged Off", "Default"]
target_negative_statuses: ["Fully Paid"]
# Primary PD model. "hgb" = HistGradientBoosting (sklearn, no libomp needed).
# Finding (with the expanded ~33-field bureau allowlist): logistic AUC ~0.680,
# HGB ~0.691, grade-alone 0.669 — BOTH models now beat grade, HGB by +0.022.
# (With the minimal 17-field allowlist logistic lost to grade; adding bona-fide
# origination-time bureau attributes closed and reversed the gap.)
# All three AUCs are always written to metrics.json (spec §2.4).
type: "hgb" # "hgb" | "logistic"
logistic_C: 1.0
hgb_max_iter: 400
hgb_learning_rate: 0.05
hgb_max_leaf_nodes: 31
calibration_method: "isotonic" # "isotonic" | "sigmoid" | "none"
random_state: 42
auc_repro_tolerance: 0.005

expected_loss:
# LGD: "data" estimates it from charged-off TRAIN-period recoveries
# LGD = 1 - (sum(principal repaid + post-charge-off recoveries) / sum(funded_amnt))
# "fixed" uses lgd_fixed (kept for the sensitivity comparison in the memo).
lgd_mode: "data" # "data" | "fixed"
lgd_fixed: 0.45
lgd: 0.45 # effective value; overwritten by the `lgd` stage when lgd_mode=data
ead_mode: "funded" # "funded" = loan_amnt (decision-time) | "outstanding" = out_prncp
```

```
(portfolio only)

simulator:
cost_of_funds: 0.04 # r_f, annual
horizon_years: 3.0 # T - all money terms put on this horizon
amort_factor: 0.52 # avg outstanding balance / original principal (36-mo level-pay)
# Annual servicing cost as a fraction of original principal. 0.4%/yr is a common figure
# for unsecured consumer instalment loans. (An earlier 1.2%/yr made even prime lending
# unprofitable in the realized-outcome backtest - see reports/memo.md, sensitivity.)
servicing_cost: 0.004
default_pd_cutoff: 0.15
default_min_fico: 660

paths:
duckdb: data/credit.duckdb
model: artifacts/model.joblib
metrics: artifacts/metrics.json
exports_dir: exports
figures_dir: reports/figures
```

src/config.py

Loads config.yaml and resolves relative paths to absolute ones. `sql\_params()` flattens the config into the flat string map the SQL templating engine (src/db.py) substitutes into `\${...}` placeholders in the .sql files.

```
"""Load config.yaml and expose it as a plain dict + a flat placeholder map for SQL."""
from __future__ import annotations

import pathlib
import yaml

ROOT = pathlib.Path(__file__).resolve().parents[1]
CONFIG_PATH = ROOT / "config.yaml"

def load() -> dict:
    with open(CONFIG_PATH) as fh:
        cfg = yaml.safe_load(fh)
        # resolve paths relative to repo root
        for k, v in cfg["paths"].items():
            cfg["paths"][k] = str((ROOT / v).resolve())
        cfg["data"]["raw_csv"] = str((ROOT / cfg["data"]["raw_csv"]).resolve())
    return cfg

def sql_params(cfg: dict) -> dict[str, str]:
    """Flat string map for `${...}` substitution in the .sql files."""
    return {
        "raw_csv": cfg["data"]["raw_csv"],
        "term_months": str(cfg["maturity"]["term_months"]),
        "issue_year_min": str(cfg["maturity"]["issue_year_min"]),
        "issue_year_max": str(cfg["maturity"]["issue_year_max"]),
        "min_months_since_issue": str(cfg["maturity"]["min_months_since_issue"]),
        "oot_cutoff": cfg["split"]["oot_cutoff"],
        # placeholder default; the `lgd` / `marts` stages overwrite it with the effective
        # value (data-estimate when expected_loss.lgd_mode == "data").
        "lgd": str(cfg["expected_loss"].get("lgd_fixed", cfg["expected_loss"].get("lgd", 0.45))),
        "ead_mode": cfg["expected_loss"]["ead_mode"],
    }
```

src/db.py

Thin DuckDB wrapper: opens a connection, and `run\_sql\_file()` reads a .sql file, strips comments, substitutes `\${...}` placeholders from the params map, splits on `;`, and executes each statement in order. This is string-substitution templating, not parameterized queries — safe here because every substituted value comes from the trusted local config.yaml, not from any external input.

```

"""Thin DuckDB helpers: open the project DB, run a .sql file with ${...} substitution."""
from __future__ import annotations

import pathlib
import re
import textwrap

import duckdb

ROOT = pathlib.Path(__file__).resolve().parents[1]
SQL_DIR = ROOT / "sql"

def connect(db_path: str, read_only: bool = False) -> duckdb.DuckDBPyConnection:
    pathlib.Path(db_path).parent.mkdir(parents=True, exist_ok=True)
    return duckdb.connect(db_path, read_only=read_only)

def _render(sql_text: str, params: dict[str, str]) -> str:
    def repl(m: re.Match) -> str:
        key = m.group(1)
        if key not in params:
            raise KeyError(f"SQL placeholder ${{{key}}} has no value in sql_params()")
        return str(params[key])

    return re.sub(r"\${(w+)\}", repl, sql_text)

def run_sql_file(con: duckdb.DuckDBPyConnection, name: str, params: dict[str, str]) -> list:
    """Execute every statement in sql/<name>. Returns the result of the LAST statement
    (as a list of tuples) so the runner can print a small summary.

    Line comments (-- ...) are stripped BEFORE splitting on `;` so a semicolon
    inside a comment cannot break the split. String literals in these files never
    contain `---` or `;`; keep it that way.
    """
    path = SQL_DIR / name
    rendered = _render(path.read_text(), params)
    no_comments = "\n".join(re.sub(r"--.*$", "", line) for line in rendered.splitlines())
    statements = [s.strip() for s in no_comments.split(";") if s.strip()]
    last = []
    for stmt in statements:
        last = con.execute(stmt).fetchall()
    return last

def show(rows: list, headers: list[str] | None = None) -> None:
    if not rows:
        print(" (no rows)")
        return
    for r in rows[:25]:
        print(" " + " | ".join(str(x) for x in r))
    if len(rows) > 25:
        print(f" ... ({len(rows) - 25} more)")

```

sql/01_raw.sql

Stage 1 (bronze): loads the raw Lending Club CSV into DuckDB with every column read as text (`all_varchar=true`) — schema-agnostic on purpose, so nothing here can fail on a type-parsing surprise. Typing happens in the next stage, deliberately.

```

-- Stage 1 (bronze): load the Lending Club accepted-loans CSV verbatim.
-- Everything stays VARCHAR here; typing happens in 02_staging.sql.

DROP TABLE IF EXISTS raw_loans;

CREATE TABLE raw_loans AS
SELECT *
FROM read_csv(
  '${raw_csv}',
  all_varchar = true,
  header = true,
  sample_size = -1,
  ignore_errors = true,
  null_padding = true
);

-- Quick shape check (printed by the runner).
SELECT count(*) AS raw_row_count FROM raw_loans;

```

sql/02_staging.sql

Stage 2 (silver): the actual data-cleaning stage. Types every field with TRY_CAST (silently NULLs on a parse failure rather than crashing the pipeline), strips percent signs and normalises casing on categoricals, maps the free-text `emp\_length` field to a number, deduplicates by loan_id keeping the earliest record, filters out loans that failed Lending Club's own credit policy, and applies the maturity window (term = 36 months, issue year in range, and — the real guardrail — only issue-months where at least 98% of loans have already reached a terminal status). Writes a data-quality summary (row counts, null rates, terminal fraction) alongside the cleaned table.

```

-- Stage 2 (silver): typed, de-duplicated, quality-filtered, maturity-windowed.
-- One clean row per loan. Applies the foundation rules from spec $2.1-$2.3.
--
-- Placeholders substituted by run_pipeline.py:
-- ${term_months} ${issue_year_min} ${issue_year_max} ${min_months_since_issue}

DROP TABLE IF EXISTS stg_loans;

CREATE TABLE stg_loans AS
WITH typed AS (
SELECT
TRY_CAST(id AS BIGINT) AS loan_id,
try_strptime(trim(issue_d), '%b-%Y')::DATE AS issue_d,
TRY_CAST(regexp_extract(trim(term), '[0-9]+') AS INTEGER) AS term_months,
trim(loan_status) AS loan_status,

-- ---- origination-time features (allowlist; see src/features.py) ----
TRY_CAST(loan_amnt AS DOUBLE) AS loan_amnt,
TRY_CAST(annual_inc AS DOUBLE) AS annual_inc,
TRY_CAST(dti AS DOUBLE) AS dti,
CASE
WHEN emp_length IS NULL OR trim(emp_length) IN ('', 'n/a') THEN NULL
WHEN trim(emp_length) = '< 1 year' THEN 0
ELSE TRY_CAST(regexp_extract(trim(emp_length), '[0-9]+') AS INTEGER)
END AS emp_length_years,
upper(trim(home_ownership)) AS home_ownership,
lower(trim(purpose)) AS purpose,
upper(trim(addr_state)) AS addr_state,
trim(verification_status) AS verification_status,
TRY_CAST(fico_range_low AS DOUBLE) AS fico_range_low,
TRY_CAST(fico_range_high AS DOUBLE) AS fico_range_high,
try_strptime(trim(earliest_cr_line), '%b-%Y')::DATE AS earliest_cr_line,
TRY_CAST(open_acc AS DOUBLE) AS open_acc,
TRY_CAST(total_acc AS DOUBLE) AS total_acc,
TRY_CAST(revol_bal AS DOUBLE) AS revol_bal,
TRY_CAST(replace(trim(revol_util), '%', '')) AS DOUBLE) AS revol_util,
TRY_CAST(delinq_2yrs AS DOUBLE) AS delinq_2yrs,
TRY_CAST(inq_last_6mths AS DOUBLE) AS inq_last_6mths,
TRY_CAST(pub_rec AS DOUBLE) AS pub_rec,
TRY_CAST(mort_acc AS DOUBLE) AS mort_acc,

-- ---- expanded bureau attributes (credit-bureau pull at application; <4% null
-- in the 2012-01..2016-02 window; all origination-time, no leakage) ----
TRY_CAST(acc_open_past_24mths AS DOUBLE) AS acc_open_past_24mths,
TRY_CAST(replace(trim(bc_util), '%', '')) AS DOUBLE) AS bc_util,
TRY_CAST(bc_open_to_buy AS DOUBLE) AS bc_open_to_buy,
TRY_CAST(mo_sin_old_rev_tl_op AS DOUBLE) AS mo_sin_old_rev_tl_op,
TRY_CAST(mths_since_recent_inq AS DOUBLE) AS mths_since_recent_inq,
TRY_CAST(mths_since_recent_bc AS DOUBLE) AS mths_since_recent_bc,
TRY_CAST(num_actv_bc_tl AS DOUBLE) AS num_actv_bc_tl,
TRY_CAST(num_tl_op_past_12m AS DOUBLE) AS num_tl_op_past_12m,
TRY_CAST(num_accts_ever_120_pd AS DOUBLE) AS num_accts_ever_120_pd,
TRY_CAST(num_tl_90g_dpd_24m AS DOUBLE) AS num_tl_90g_dpd_24m,
TRY_CAST(replace(trim(pct_tl_nvr_dlq), '%', '')) AS DOUBLE) AS pct_tl_nvr_dlq,
TRY_CAST(replace(trim(percent_bc_gt_75), '%', '')) AS DOUBLE) AS percent_bc_gt_75,

```

```

TRY_CAST(tot_hi_cred_lim AS DOUBLE) AS tot_hi_cred_lim,
TRY_CAST(total_bal_ex_mort AS DOUBLE) AS total_bal_ex_mort,
TRY_CAST(total_bc_limit AS DOUBLE) AS total_bc_limit,
TRY_CAST(tot_cur_bal AS DOUBLE) AS tot_cur_bal,
TRY_CAST(avg_cur_bal AS DOUBLE) AS avg_cur_bal,
TRY_CAST(tot_coll_amt AS DOUBLE) AS tot_coll_amt,
TRY_CAST(pub_rec_bankruptcies AS DOUBLE) AS pub_rec_bankruptcies,
nullif(trim(application_type), '') AS application_type,

-- ---- benchmark-only (NOT features): Lending Club's own risk model ----
trim(grade) AS lc_grade,
trim(sub_grade) AS lc_sub_grade,
TRY_CAST(replace(trim(int_rate), '%', '')) AS DOUBLE) AS int_rate,

-- ---- post-origination fields (outcomes / LGD / EAD only; never features) ----
TRY_CAST(funded_amnt AS DOUBLE) AS funded_amnt,
TRY_CAST(out_prncp AS DOUBLE) AS out_prncp,
TRY_CAST(total_pymnt AS DOUBLE) AS total_pymnt,
TRY_CAST(total_rec_prncp AS DOUBLE) AS total_rec_prncp,
TRY_CAST(recoveries AS DOUBLE) AS recoveries,
try_strptime(trim(last_pymnt_d), '%b-%Y')::DATE AS last_pymnt_d
FROM raw_loans
WHERE loan_status IS NOT NULL
AND trim(loan_status) NOT LIKE 'Does not meet the credit policy%'
),
flagged AS (
SELECT
*,
extract('year' FROM issue_d) AS issue_year,
(CASE WHEN loan_status IN ('Fully Paid', 'Charged Off', 'Default') THEN TRUE ELSE FALSE END)
AS is_terminal
FROM typed
WHERE loan_id IS NOT NULL
AND issue_d IS NOT NULL
AND term_months = ${term_months}
),
maturity AS (
-- an issue month is usable only if the whole cohort has had time to resolve
SELECT date_diff('month', min_issue, max_issue) AS ignore,
max_issue
FROM (SELECT min(issue_d) AS min_issue, max(issue_d) AS max_issue FROM flagged)
),
windowed AS (
SELECT f.*
FROM flagged f, maturity m
WHERE f.issue_year BETWEEN ${issue_year_min} AND ${issue_year_max}
AND date_diff('month', f.issue_d, m.max_issue) >= ${min_months_since_issue}
-- impossible-value guards
AND f.loan_amnt > 0
AND (f.annual_inc IS NULL OR f.annual_inc >= 0)
AND (f.dti IS NULL OR f.dti BETWEEN -1 AND 1000)
AND (f.revol_util IS NULL OR f.revol_util BETWEEN 0 AND 1000)
)
SELECT * EXCLUDE (issue_year)
FROM windowed

```

```

QUALIFY row_number() OVER (PARTITION BY loan_id ORDER BY issue_d) = 1;

-- data-quality summary (printed by the runner + logged to reports/dq_log.md by src)
SELECT
count(*) AS n_loans,
min(issue_d) AS first_issue,
max(issue_d) AS last_issue,
round(avg(CASE WHEN is_terminal THEN 1.0 ELSE 0.0 END), 4) AS frac_terminal,
round(avg(CASE WHEN is_terminal AND loan_status <> 'Fully Paid' THEN 1.0
WHEN is_terminal THEN 0.0 END), 4) AS observed_default_rate,
sum(CASE WHEN annual_inc IS NULL THEN 1 ELSE 0 END) AS null_annual_inc,
sum(CASE WHEN dti IS NULL THEN 1 ELSE 0 END) AS null_dti,
sum(CASE WHEN revol_util IS NULL THEN 1 ELSE 0 END) AS null_revol_util
FROM stg_loans;

```

sql/03_features_outcome.sql

Stage 3 (gold, part 1) — the leakage firewall. Splits the cleaned data into THREE physically separate tables: `mart_loan_features` (only origination-time fields, the only thing the model is ever allowed to see), `mart_loan_benchmark` (grade/sub_grade/int_rate — Lending Club's own risk output, benchmark-only, never a model input), and `mart_loan_outcomes` (default_flag and every cash-flow field, used only for labels and backtesting). `default_flag` is defined precisely here: Charged Off or Default = 1, Fully Paid = 0, everything else excluded (censored, not guessed at). `v_model_frame` is the join of features + outcome the model actually trains on.

```
-- Stage 3 (gold, part 1): split the clean table into three disjoint marts so a
-- post-origination or benchmark column can NEVER reach the model by accident.
--
-- mart_loan_features origination-time ALLOWLIST only (spec §2.2, §4)
-- mart_loan_benchmark Lending Club grade / int_rate (benchmark + profit calc only)
-- mart_loan_outcomes default flag + fields for EAD / LGD
--
-- Placeholder: ${oot_cutoff} (YYYY-MM-DD)

DROP TABLE IF EXISTS mart_loan_features;
CREATE TABLE mart_loan_features AS
SELECT
  loan_id,
  issue_d,
  CASE WHEN issue_d < DATE '${oot_cutoff}' THEN 'train' ELSE 'test' END AS split_set,

  loan_amnt,
  annual_inc,
  dti,
  emp_length_years,
  home_ownership,
  purpose,
  addr_state,
  verification_status,
  (fico_range_low + fico_range_high) / 2.0 AS fico_mid,
  date_diff('month', earliest_cr_line, issue_d) AS credit_history_months,
  open_acc,
  total_acc,
  revol_bal,
  revol_util,
  delinq_2yrs,
  inq_last_6mths,
  pub_rec,
  mort_acc,

  -- expanded bureau attributes (spec §4; see 02_staging.sql)
  acc_open_past_24mths,
  bc_util,
  bc_open_to_buy,
  mo_sin_old_rev_tl_op,
  mths_since_recent_inq,
  mths_since_recent_bc,
  num_actv_bc_tl,
  num_tl_op_past_12m,
  num_accts_ever_120_pd,
  num_tl_90g_dpd_24m,
  pct_tl_nvr_dlq,
  percent_bc_gt_75,
  tot_hi_cred_lim,
  total_bal_ex_mort,
  total_bc_limit,
  tot_cur_bal,
  avg_cur_bal,
  tot_coll_amt,
  pub_rec_bankruptcies,
```

```

application_type
FROM stg_loans;

DROP TABLE IF EXISTS mart_loan_benchmark;
CREATE TABLE mart_loan_benchmark AS
SELECT loan_id, issue_d, lc_grade, lc_sub_grade, int_rate
FROM stg_loans;

DROP TABLE IF EXISTS mart_loan_outcomes;
CREATE TABLE mart_loan_outcomes AS
SELECT
loan_id,
issue_d,
loan_status,
is_terminal,
CASE
WHEN loan_status IN ('Charged Off', 'Default') THEN 1
WHEN loan_status = 'Fully Paid' THEN 0
ELSE NULL -- non-terminal: excluded from modelling
END AS default_flag,
funded_amnt,
loan_amnt,
out_prncp,
total_pymnt, -- total cash received (principal + interest + fees) – realized-P&L backtest
total_rec_prncp, -- principal repaid – LGD estimation + realized loss
recoveries -- post-charge-off recoveries – LGD estimation + realized loss
FROM stg_loans;

-- modelling frame = features + label, terminal loans only
DROP VIEW IF EXISTS v_model_frame;
CREATE VIEW v_model_frame AS
SELECT f.*, o.default_flag
FROM mart_loan_features f
JOIN mart_loan_outcomes o USING (loan_id)
WHERE o.default_flag IS NOT NULL;

SELECT split_set, count(*) AS n, round(avg(default_flag), 4) AS default_rate
FROM v_model_frame GROUP BY 1 ORDER BY 1;

```

src/features.py

The feature CONTRACT. ALLOWLIST is the single source of truth for what the model is allowed to see — about 33 origination-time fields, explicitly chosen because grade/sub_grade/int_rate would be circular (Lending Club's own risk model's output feeding this one). FORBIDDEN is a second, independent list asserted against at train time — defence in depth on top of the SQL-level mart separation, not a replacement for it. `engineer()` adds two derived ratios (log income, loan-to-income) and scrubs infinities before scaling.

```

"""The feature contract (spec §2.2, §4).

ALLOWLIST is the single source of truth for what enters the PD model. Anything not
listed here is excluded by construction. `grade`, `sub_grade`, `int_rate` are NOT here
on purpose – they are Lending Club's own risk-model output (circularity) and live in
mart_loan_benchmark, used only for the benchmark and the profit calculation.
"""

from __future__ import annotations

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

NUMERIC: list[str] = [
    # --- core application fields ---
    "loan_amnt",
    "annual_inc",
    "dti",
    "emp_length_years",
    "fico_mid",
    "credit_history_months",
    "open_acc",
    "total_acc",
    "revol_bal",
    "revol_util",
    "delinq_2yrs",
    "inq_last_6mths",
    "pub_rec",
    "mort_acc",
    # --- expanded bureau attributes (credit-bureau pull at application; spec §4).
    # All knowable at underwriting, all <4% null in the modelling window. ---
    "acc_open_past_24mths", # recent credit-seeking
    "bc_util", # bankcard utilisation (distinct from revol_util)
    "bc_open_to_buy", # unused bankcard credit
    "mo_sin_old_rev_tl_op", # age of oldest revolving trade (file thickness)
    "mths_since_recent_inq", # inquiry recency
    "mths_since_recent_bc", # months since most recent bankcard opened
    "num_actv_bc_tl", # active bankcard trades
    "num_tl_op_past_12m", # trades opened in last 12m
    "num_accts_ever_120_pd", # accounts ever 120+ DPD (history of serious delinquency)
    "num_tl_90g_dpd_24m", # trades 90+ DPD in last 24m
    "pct_tl_nvr_dlq", # % of trades never delinquent
    "percent_bc_gt_75", # % of bankcards over 75% utilised (distress)
    "tot_hi_cred_lim", # total high credit limit (capacity)
    "total_bal_ex_mort", # total non-mortgage balance (leverage)
    "total_bc_limit", # total bankcard limit
    "tot_cur_bal", # total current balance across all accounts
    "avg_cur_bal", # average balance per account
    "tot_coll_amt", # total amount currently in collections
    "pub_rec_bankruptcies", # public-record bankruptcies
]

```

```

CATEGORICAL: list[str] = [
    "home_ownership",
    "purpose",
    "verification_status",
    "application_type", # Individual vs Joint App
    # addr_state is intentionally omitted by default (fairness, spec §6).
    # Add it here only if you also add the disparate-impact check.
]

ALLOWLIST: list[str] = NUMERIC + CATEGORICAL

# columns that must never appear in the modelling frame – asserted at train time
FORBIDDEN = {
    "grade", "sub_grade", "lc_grade", "lc_sub_grade", "int_rate",
    "last_pymnt_d", "last_pymnt_amnt", "total_pymnt", "total_rec_prncp", "total_rec_int",
    "recoveries", "collection_recovery_fee", "out_prncp", "out_prncp_inv",
    "last_fico_range_high", "last_fico_range_low", "next_pymnt_d",
    "loan_status", "default_flag", "funded_amnt",
}

def engineer(df: pd.DataFrame) -> pd.DataFrame:
    """Light, leakage-safe derived features. Everything here uses only allowlist inputs."""
    out = df.copy()
    if "annual_inc" in out.columns:
        out["annual_inc"] = out["annual_inc"].clip(lower=0)
        out["log_annual_inc"] = np.log1p(out["annual_inc"].fillna(0.0))
    if {"loan_amnt", "annual_inc"}.issubset(out.columns):
        inc = out["annual_inc"].where(out["annual_inc"] > 0)
        out["loan_to_income"] = (out["loan_amnt"] / inc).clip(upper=5.0)

    # scrub non-finite values from every numeric column so downstream scaling / the
    # linear solver never see inf (median imputation then fills the resulting NaNs).
    num = NUMERIC + DERIVED_NUMERIC
    present = [c for c in num if c in out.columns]
    out[present] = out[present].replace([np.inf, -np.inf], np.nan)
    return out

DERIVED_NUMERIC = ["log_annual_inc", "loan_to_income"]

def build_preprocessor() -> ColumnTransformer:
    num_cols = NUMERIC + DERIVED_NUMERIC
    numeric = Pipeline(
        [
            ("impute", SimpleImputer(strategy="median")),
            ("scale", StandardScaler())
        ]
    )
    categorical = Pipeline(
        [
            ("impute", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", min_frequency=0.01, sparse_output=False)),
        ]
    )
    return ColumnTransformer(
        [
            ("num", numeric, num_cols),
            ("cat", categorical, CATEGORICAL)],
        remainder="drop",
        verbose_feature_names_out=False,
    )

def assert_no_leakage(df: pd.DataFrame) -> None:
    bad = FORBIDDEN.intersection(df.columns)
    if bad:
        raise AssertionError(f"Leakage guard: forbidden columns in modelling frame: {sorted(bad)}")

```

src/train.py

Fits the primary model (HistGradientBoosting, per config) inside a preprocessing pipeline (median-impute + scale numerics, most-frequent-impute + one-hot categoricals), wraps it in CalibratedClassifierCV(method='isotonic', cv=3) so calibration is fit on held-out folds rather than the same data used to train the base learner. Also fits a logistic-regression alternative and a naive 'grade-alone' baseline for comparison, runs a DeLong significance test on

the primary model vs that baseline, and writes every metric — train and test AUC/Gini/KS/Brier, the three-way benchmark, the calibration decile table — to artifacts/metrics.json.

```
"""Fit the PD model, calibrate it, benchmark it against Lending Club grade, write artefacts.

Reads : v_model_frame (features + label, terminal loans, split_set column)
mart_loan_benchmark (grade - benchmark only)
Writes : artifacts/model.joblib, artifacts/metrics.json, reports/figures/*.png
Returns: test AUC (float) - used by the pipeline's reproducibility assertion.
"""
from __future__ import annotations

import contextlib
import pathlib
import warnings

import joblib
import numpy as np
import pandas as pd
from sklearn.calibration import CalibratedClassifierCV
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.pipeline import Pipeline

from src import evaluate, features
from src.config import load
from src.db import connect

ROOT = pathlib.Path(__file__).resolve().parents[1]

@contextlib.contextmanager
def _quiet_blas():
    """Some OpenBLAS builds emit spurious divide/overflow RuntimeWarnings from `matmul`
    inside the LBFGS solver. They do not affect the fitted coefficients - silence them."""
    with np.errstate(divide="ignore", over="ignore", invalid="ignore"), warnings.catch_warnings():
        warnings.filterwarnings("ignore", message=".*matmul.*", category=RuntimeWarning)
    yield

def _load_frame(con) -> pd.DataFrame:
    df = con.execute("SELECT * FROM v_model_frame").fetchdf()
    df = features.engineer(df)
    features.assert_no_leakage(df.drop(columns=["default_flag"]))
    return df

def _estimator(kind: str, cfg: dict) -> Pipeline:
    """Unfitted preprocessing + classifier pipeline for the requested model type.

    No class_weight anywhere: this is a probability-of-default model and calibration is
    the point (spec $5.2). A ~13-15% positive rate is not rare enough to need balancing,
    and balancing forces the calibrator to undo its own distortion.
    """
    if kind == "logistic":
        clf = LogisticRegression(
            C=cfg["model"]["logistic_C"], max_iter=2000,
```

```

random_state=cfg["model"]["random_state"],
)
elif kind == "hgb":
    clf = HistGradientBoostingClassifier(
        max_iter=cfg["model"]["hgb_max_iter"],
        learning_rate=cfg["model"]["hgb_learning_rate"],
        max_leaf_nodes=cfg["model"]["hgb_max_leaf_nodes"],
        early_stopping=True, validation_fraction=0.1,
        random_state=cfg["model"]["random_state"],
    )
else:
    raise SystemExit(f"config.model.type must be 'hgb' or 'logistic', got {kind!r}")
return Pipeline([("prep", features.build_preprocessor()), ("clf", clf)])

def _fit_calibrated(kind: str, X: pd.DataFrame, y: np.ndarray, cfg: dict):
    base = _estimator(kind, cfg)
    method = cfg["model"]["calibration_method"]
    with _quiet_blas():
        if method == "none":
            return base.fit(X, y)
        model = CalibratedClassifierCV(base, method=method, cv=3)
        model.fit(X, y)
    return model

def _auc_of(kind: str, Xtr, ytr, Xte, yte, cfg) -> tuple[float, object]:
    m = _fit_calibrated(kind, Xtr, ytr, cfg)
    with _quiet_blas():
        p = m.predict_proba(Xte)[: , 1]
    return float(roc_auc_score(yte, p)), m

def _grade_benchmark(bench_train: pd.DataFrame, bench_test: pd.DataFrame,
    y_train: np.ndarray, y_test: np.ndarray) -> dict:
    """Baseline PD = historical default rate of the loan's grade, learned on train."""
    rate_by_grade = (
        pd.DataFrame({"g": bench_train["lc_grade"].values, "y": y_train})
        .groupby("g")["y"].mean()
    )
    overall = float(np.mean(y_train))
    pd_test = bench_test["lc_grade"].map(rate_by_grade).fillna(overall).to_numpy(dtype=float)
    return {
        "auc": float(roc_auc_score(y_test, pd_test)),
        "rate_by_grade": {k: float(v) for k, v in rate_by_grade.sort_index().items()},
        "pd_test": pd_test,
    }

def _lightgbm_delta(X_train, y_train, X_test, y_test, cfg) -> dict:
    try:
        from lightgbm import LGBMClassifier
    except Exception as exc: # noqa: BLE001
        return {"available": False, "note": f"lightgbm not usable: {exc}"}
    pipe = Pipeline(

```

```

[("prep", features.build_preprocessor()),
 ("clf", LGBMClassifier(n_estimators=400, learning_rate=0.03, num_leaves=31,
 subsample=0.8, colsample_bytree=0.8,
 random_state=cfg["model"]["random_state"], verbose=-1))]
)
pipe.fit(X_train, y_train)
p = pipe.predict_proba(X_test)[: , 1]
return {"available": True, "auc": float(roc_auc_score(y_test, p))}

def main() -> float:
    cfg = load()
    con = connect(cfg["paths"]["duckdb"])

    frame = _load_frame(con)
    bench = con.execute("SELECT loan_id, lc_grade FROM mart_loan_benchmark").fetchdf()
    frame = frame.merge(bench, on="loan_id", how="left")

    train = frame[frame["split_set"] == "train"].reset_index(drop=True)
    test = frame[frame["split_set"] == "test"].reset_index(drop=True)
    if len(train) == 0 or len(test) == 0:
        raise SystemExit("Empty train or test split – check config.split.oout_cutoff vs the mature window.")

    feat_cols = features.ALLOWLIST + features.DERIVED_NUMERIC
    Xtr, ytr = train[feat_cols], train["default_flag"].to_numpy()
    Xte, yte = test[feat_cols], test["default_flag"].to_numpy()

    primary_kind = cfg["model"]["type"]
    secondary_kind = "logistic" if primary_kind == "hgb" else "hgb"

    model = _fit_calibrated(primary_kind, Xtr, ytr, cfg)
    with _quiet_blas():
        p_tr = model.predict_proba(Xtr)[: , 1]
        p_te = model.predict_proba(Xte)[: , 1]

    # three-way comparison (spec §2.4): primary vs the other model type vs grade-alone
    secondary_auc, _ = _auc_of(secondary_kind, Xtr, ytr, Xte, yte, cfg)
    grade_bm = _grade_benchmark(train[["lc_grade"]], test[["lc_grade"]], ytr, yte)
    lgbm = _lightgbm_delta(Xtr, ytr, Xte, yte, cfg)

    fig_dir = cfg["paths"]["figures_dir"]
    evaluate.plot_calibration(yte, p_te, f"{fig_dir}/calibration_test.png",
        "Calibration – out-of-time test")
    dec = evaluate.decile_table(yte, p_te)
    evaluate.plot_decile_lift(dec, f"{fig_dir}/decile_lift_test.png",
        "PD deciles – out-of-time test")

    primary_auc = float(roc_auc_score(yte, p_te))
    delong = evaluate.delong_roc_test(yte, p_te, grade_bm["pd_test"])
    grade_bm = {k: v for k, v in grade_bm.items() if k != "pd_test"} # don't serialise the vector
    metrics = {
        "dataset": {
            "snapshot": cfg["data"]["snapshot_label"],
            "oot_cutoff": cfg["split"]["oot_cutoff"],
            "n_train": int(len(train)), "n_test": int(len(test)),

```

```

"train_default_rate": float(ytr.mean()), "test_default_rate": float(yte.mean()),
},
"primary_model": primary_kind,
"model_train": evaluate.summary(ytr, p_tr),
"model_test": evaluate.summary(yte, p_te),
"comparison_test_auc": {
    primary_kind: primary_auc,
    secondary_kind: secondary_auc,
    "grade_only": grade_bm["auc"],
},
"benchmark_grade_test": grade_bm,
"delta_auc_vs_grade": primary_auc - grade_bm["auc"],
"delong_primary_vs_grade": delong,
"lightgbm_test": lgbm,
"decile_table_test": dec.to_dict(orient="records"),
"config": {"type": primary_kind,
"calibration_method": cfg["model"]["calibration_method"],
"logistic_C": cfg["model"]["logistic_C"]},
}

joblib.dump({"model": model, "feature_columns": feat_cols}, cfg["paths"]["model"])
evaluate.write_metrics(metrics, cfg["paths"]["metrics"])

print(f" primary ({primary_kind}) AUC : {primary_auc:.4f}")
print(f" {secondary_kind}<14} AUC : {secondary_auc:.4f}")
print(f" grade-only AUC : {grade_bm['auc']:.4f}")
print(f" delta vs grade : {metrics['delta_auc_vs_grade']:+.4f} "
f"(DeLong z={delong['z']:.1f}, p={delong['p_value']:.2e})")
if lgbm.get("available"):
    print(f" lightgbm AUC : {lgbm['auc']:.4f}")
print(f" test KS / Brier : {metrics['model_test']['ks']:.4f} /
{metrics['model_test']['brier']:.4f}")
con.close()
return primary_auc

if __name__ == "__main__":
    main()

```

src/evaluate.py

The statistical toolkit shared by train.py and backtest.py: ``ks_statistic()`` (max separation between cumulative good/bad distributions), ``decile_table()`` (sorts by predicted PD into 10 equal groups and compares predicted vs observed rate — checks calibration group by group, not just on average), and ``delong_roc_test()`` — a real implementation of the DeLong test for comparing two correlated ROC AUCs on the same sample, so 'model beats grade' is a tested claim with a p-value, not just a bigger number.

```

"""Discrimination + calibration metrics and figures (spec §5.2, §5.7)."""
from __future__ import annotations

import json
import pathlib

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss, roc_auc_score

def ks_statistic(y_true: np.ndarray, y_score: np.ndarray) -> float:
    order = np.argsort(y_score)
    y = np.asarray(y_true)[order]
    pos = np.cumsum(y) / max(y.sum(), 1)
    neg = np.cumsum(1 - y) / max((1 - y).sum(), 1)
    return float(np.max(np.abs(pos - neg)))

def decile_table(y_true: np.ndarray, y_score: np.ndarray, k: int = 10) -> pd.DataFrame:
    df = pd.DataFrame({"y": np.asarray(y_true), "pd": np.asarray(y_score)})
    df["bucket"] = pd.qcut(df["pd"].rank(method="first"), k, labels=False)
    g = df.groupby("bucket").agg(n=("y", "size"), mean_pd=("pd", "mean"), obs_rate=("y", "mean"))
    g["lift"] = g["obs_rate"] / df["y"].mean()
    return g.reset_index()

def _midrank(x: np.ndarray) -> np.ndarray:
    order = np.argsort(x)
    x_sorted = x[order]
    n = len(x)
    tr = np.zeros(n)
    i = 0
    while i < n:
        j = i
        while j < n and x_sorted[j] == x_sorted[i]:
            j += 1
        tr[i:j] = 0.5 * (i + j - 1) + 1
        i = j
    out = np.empty(n)
    out[order] = tr
    return out

def delong_roc_test(y_true: np.ndarray, score_a: np.ndarray, score_b: np.ndarray) -> dict:
    """Fast DeLong test for two correlated ROC AUCs on the same sample (Sun & Xu 2014).

    Returns both AUCs, the AUC difference, the DeLong z-statistic and a two-sided p-value
    for H0: AUC_a == AUC_b. Use it so "model beats grade" is a tested claim, not a point
    estimate.

```

```

"""
from scipy import stats

y = np.asarray(y_true).astype(int)
pos = np.c_[score_a, score_b][y == 1].T # 2 x m
neg = np.c_[score_a, score_b][y == 0].T # 2 x n
m, n = pos.shape[1], neg.shape[1]
k = 2

tx = np.array([_midrank(pos[r]) for r in range(k)])
ty = np.array([_midrank(neg[r]) for r in range(k)])
txy = np.array([_midrank(np.r_[pos[r], neg[r]]) for r in range(k)])

aucs = txy[:, :m].sum(axis=1) / (m * n) - (m + 1) / (2.0 * n)
v01 = (txy[:, :m] - tx) / n
v10 = 1.0 - (txy[:, m:] - ty) / m
s01 = np.cov(v01)
s10 = np.cov(v10)
cov = s01 / m + s10 / n
var_diff = cov[0, 0] + cov[1, 1] - 2 * cov[0, 1]
diff = aucs[0] - aucs[1]
z = float(diff / np.sqrt(var_diff)) if var_diff > 0 else 0.0
p = float(2 * stats.norm.sf(abs(z)))
return {
    "auc_a": float(aucs[0]),
    "auc_b": float(aucs[1]),
    "auc_diff": float(diff),
    "z": z,
    "p_value": p,
    "significant_at_0.05": p < 0.05,
}

def summary(y_true: np.ndarray, y_score: np.ndarray) -> dict:
    return {
        "auc": float(roc_auc_score(y_true, y_score)),
        "gini": float(2 * roc_auc_score(y_true, y_score) - 1),
        "ks": ks_statistic(y_true, y_score),
        "brier": float(brier_score_loss(y_true, y_score)),
        "base_rate": float(np.mean(y_true)),
        "n": int(len(y_true)),
    }

def plot_calibration(y_true: np.ndarray, y_score: np.ndarray, path: str, title: str) -> None:
    frac_pos, mean_pred = calibration_curve(y_true, y_score, n_bins=10, strategy="quantile")
    fig, ax = plt.subplots(figsize=(4.5, 4.5))
    ax.plot([0, 1], [0, 1], "--", color="#888", lw=1, label="perfect")
    ax.plot(mean_pred, frac_pos, "o-", color="#1f4e79", label="model")
    ax.set_xlabel("mean predicted PD")
    ax.set_ylabel("observed default rate")
    ax.set_title(title)
    ax.legend()
    fig.tight_layout()
    _save(fig, path)

```

```

def plot_decile_lift(dec: pd.DataFrame, path: str, title: str) -> None:
    fig, ax = plt.subplots(figsize=(5.5, 3.5))
    ax.bar(dec["bucket"].astype(str), dec["obs_rate"], color="#1f4e79")
    ax.plot(dec["bucket"].astype(str), dec["mean_pd"], "o-", color="#c17f00", label="predicted PD")
    ax.set_xlabel("PD decile (0 = safest)")
    ax.set_ylabel("default rate")
    ax.set_title(title)
    ax.legend()
    fig.tight_layout()
    _save(fig, path)

def _save(fig, path: str) -> None:
    pathlib.Path(path).parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, dpi=130)
    plt.close(fig)

def write_metrics(metrics: dict, path: str) -> None:
    pathlib.Path(path).parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w") as fh:
        json.dump(metrics, fh, indent=2, default=float)

```

src/score.py

Batch-scores every loan in the mature window (not just test-split loans — the whole book, so the portfolio views and simulator cover everything) using the trained model's `predict_proba`, writes `loan_id + pd_hat` to `mart_scores`. Simple by design: this is the only place the model actually gets called for inference.

```

"""Batch-score every loan in the mature window and write mart_scores.

Scores ALL loans in mart_loan_features (terminal or not) so the portfolio view and the
simulator cover the whole book. Expected Loss is then built in sql/06_marts.sql.
"""
from __future__ import annotations

import pathlib

import joblib

from src import features
from src.config import load
from src.db import connect

ROOT = pathlib.Path(__file__).resolve().parents[1]

def main() -> int:
    cfg = load()
    bundle = joblib.load(cfg["paths"]["model"])
    model, feat_cols = bundle["model"], bundle["feature_columns"]

    con = connect(cfg["paths"]["duckdb"])
    df = con.execute("SELECT * FROM mart_loan_features").fetchdf()
    df = features.engineer(df)

    df["pd_hat"] = model.predict_proba(df[feat_cols])[:, 1]

    con.execute("DROP TABLE IF EXISTS mart_scores")
    con.execute("CREATE TABLE mart_scores (loan_id BIGINT, pd_hat DOUBLE)")
    con.register("_scores_df", df[["loan_id", "pd_hat"]])
    con.execute("INSERT INTO mart_scores SELECT loan_id, pd_hat FROM _scores_df")
    n = con.execute("SELECT count(*), round(avg(pd_hat), 4) FROM mart_scores").fetchone()
    print(f"scored {n[0]} loans, mean PD {n[1]}")
    con.close()
    return int(n[0])

if __name__ == "__main__":
    main()

```

src/lgd.py

Loss-given-default, estimated from data rather than assumed: 1 minus the recovery rate (principal repaid + recoveries, over funded amount) on charged-off TRAIN loans only — kept out of the test set so the number that feeds Expected Loss isn't estimated on the same loans used to validate the policy. ``estimate_by_grade()`` repeats this per Lending Club grade, then applies Buhlmann credibility weighting toward the flat figure — ``credibility = n / (n + K)`` — so a grade with a handful of charged-offs (G: 73) doesn't get a wild, untrustworthy point estimate; ``write_grade_table()`` materializes this as a small DuckDB table the marts stage joins against.

```
"""Estimate LGD from the data instead of assuming it (council rec #2).

For unsecured consumer loans the textbook ~45% LGD is optimistic. Lending Club reports,
for each charged-off loan, the principal repaid before charge-off (`total_rec_prncp`) and
any post-charge-off recovery (`recoveries`). So on the CHARGED-OFF loans in the training
period:

recovery_rate = sum(total_rec_prncp + recoveries) / sum(funded_amnt)
LGD = 1 - recovery_rate

Train period only, so the number that feeds Expected Loss is not estimated on the
out-of-time test set.

Expected Loss is segmented by grade (audit finding: a single flat LGD hides real
dispersion — 44% on grade A vs 60% on grade G). Grade-level LGD uses the same formula
above, restricted to that grade, then Buhlmann credibility-weighted toward the flat
portfolio LGD so thin tail grades (e.g. G, ~70 charged-off loans) don't get a noisy raw
point estimate:

credibility_g = n_g / (n_g + K) K = full_credibility_n (config)
lgd_credible_g = credibility_g * lgd_raw_g + (1 - credibility_g) * lgd_flat

This is standard actuarial credibility theory, not an arbitrary cutoff — it shrinks
sparse segments toward the population estimate in proportion to how much data actually
supports them, rather than a hard include/exclude threshold.
"""
from __future__ import annotations

import json
import pathlib

from src.config import load
from src.db import connect

ROOT = pathlib.Path(__file__).resolve().parents[1]
LGD_JSON = ROOT / "artifacts" / "lgd.json"
FULL_CREDIBILITY_N = 2000 # charged-off loans at which a grade's own LGD gets ~full weight

def estimate(con, oot_cutoff: str) -> dict:
    row = con.execute(
        f"""
        SELECT count(*) AS n_charged_off,
        sum(funded_amnt) AS ead_sum,
        sum(coalesce(total_rec_prncp, 0)
        + coalesce(recoveries, 0)) AS recovered_sum,
        avg(1.0 - (coalesce(total_rec_prncp, 0) + coalesce(recoveries, 0))
        / nullif(funded_amnt, 0)) AS lgd_loan_mean
        FROM mart_loan_outcomes
        WHERE default_flag = 1
        AND issue_d < DATE '{oot_cutoff}'
        """
    ).fetchone()
    n, ead_sum, rec_sum, lgd_loan_mean = row
    recovery_rate = float(rec_sum / ead_sum)
```

```

return {
    "lgd_data": round(1.0 - recovery_rate, 4), # exposure-weighted (feeds EL)
    "lgd_loan_mean": round(float(lgd_loan_mean), 4), # equal-weighted, for reference
    "recovery_rate": round(recovery_rate, 4),
    "n_charged_off_train": int(n),
    "method": "1 - sum(total_rec_prncp + recoveries) / sum(funded_amnt), charged-off train loans",
}

def estimate_by_grade(con, oot_cutoff: str, lgd_flat: float) -> list[dict]:
    """Per-grade LGD (same formula as `estimate`), credibility-weighted toward `lgd_flat`."""
    rows = con.execute(
        f"""
        SELECT b.lc_grade AS grade,
        count(*) AS n_charged_off,
        sum(o.funded_amnt) AS ead_sum,
        sum(coalesce(o.total_rec_prncp, 0) + coalesce(o.recoveries, 0)) AS recovered_sum
        FROM mart_loan_outcomes o
        JOIN mart_loan_benchmark b USING (loan_id)
        WHERE o.default_flag = 1 AND o.issue_d < DATE '{oot_cutoff}'
        GROUP BY b.lc_grade
        ORDER BY b.lc_grade
        """
    ).fetchdf()
    out = []
    for _, r in rows.iterrows():
        n = int(r["n_charged_off"])
        lgd_raw = 1.0 - float(r["recovered_sum"] / r["ead_sum"]) if r["ead_sum"] else lgd_flat
        credibility = n / (n + FULL_CREDIBILITY_N)
        lgd_credible = credibility * lgd_raw + (1 - credibility) * lgd_flat
        out.append({
            "grade": r["grade"],
            "n_charged_off_train": n,
            "lgd_raw": round(lgd_raw, 4),
            "credibility": round(credibility, 3),
            "lgd": round(lgd_credible, 4), # credibility-weighted – this is what feeds EL
        })
    return out

def write_grade_table(con, by_grade: list[dict]) -> None:
    """Materialize a small `lgd_by_grade(grade, lgd)` table for 06_marts.sql to join against."""
    con.execute("DROP TABLE IF EXISTS lgd_by_grade")
    import pandas as pd # local import: only needed here, keeps module import light
    df = pd.DataFrame(by_grade)[["grade", "lgd", "n_charged_off_train"]]
    con.register("_lgd_by_grade_df", df)
    con.execute("CREATE TABLE lgd_by_grade AS SELECT grade, lgd, n_charged_off_train FROM _lgd_by_grade_df")
    con.unregister("_lgd_by_grade_df")

def resolve(cfg: dict, con=None) -> float:
    """Effective LGD for this run: data-estimate (cached in artifacts/lgd.json) or the fixed value."""
    el = cfg["expected_loss"]
    if el["lgd_mode"] != "data":
        return float(el["lgd_fixed"])

```

```

if LGD_JSON.exists():
    return float(json.loads(LGD_JSON.read_text())["lgd_data"])
if con is None:
    con = connect(cfg["paths"]["duckdb"])
    return float(estimate(con, cfg["split"]["oot_cutoff"])["lgd_data"])

def main() -> float:
    cfg = load()
    con = connect(cfg["paths"]["duckdb"])
    if cfg["expected_loss"]["lgd_mode"] != "data":
        lgd = float(cfg["expected_loss"]["lgd_fixed"])
        print(f" lgd_mode=fixed -> LGD = {lgd}")
    con.close()
    return lgd
    info = estimate(con, cfg["split"]["oot_cutoff"])
    by_grade = estimate_by_grade(con, cfg["split"]["oot_cutoff"], info["lgd_data"])
    write_grade_table(con, by_grade)
    info["by_grade"] = by_grade
    info["by_grade_method"] = (
        f"Buhlmann credibility toward the flat LGD, full_credibility_n={FULL_CREDIBILITY_N} "
        "charged-off loans. This segmented table feeds Expected Loss (06_marts.sql); "
        "`lgd_data` above remains the flat fallback for any grade not in the table."
    )
    LGD_JSON.parent.mkdir(parents=True, exist_ok=True)
    LGD_JSON.write_text(json.dumps(info, indent=2))
    print(f" charged-off train loans : {info['n_charged_off_train']:,}")
    print(f" recovery rate : {info['recovery_rate']:.3f}")
    print(f" LGD (exposure-weighted, flat fallback) : {info['lgd_data']:.3f} [fixed assumption was "
          f"{cfg['expected_loss']['lgd_fixed']}]")
    print(" LGD by grade (credibility-weighted, feeds EL):")
    for g in by_grade:
        print(f" {g['grade']}: {g['lgd']:.3f} (raw {g['lgd_raw']:.3f}, credibility "
              f"{g['credibility']:.2f}, n={g['n_charged_off_train']:,})")
    con.close()
    return info["lgd_data"]

if __name__ == "__main__":
    main()

```

sql/06_marts.sql

Stage 3 (gold, part 2) — the serving layer, built after scoring. `mart_loan_el` joins each loan to its grade-specific LGD (falling back to the flat portfolio LGD if a grade has none) and computes Expected Loss = PD x EAD x LGD exactly once, reused by every downstream consumer so there's no risk of two different EL numbers drifting apart. `mart_simulator_base` and `mart_portfolio_summary` are the pre-aggregated views the Streamlit simulator, CREDENCE, and Power BI all read from.

```

-- Stage 3 (gold, part 2): serving marts, built AFTER scoring writes mart_scores.
-- mart_loan_el per-loan expected loss
-- mart_simulator_base loan-level input for the Streamlit simulator
-- mart_portfolio_summary aggregated view for Power BI page 1
--
-- Placeholders: ${lgd} ${ead_mode} (ead_mode: 'funded' | 'outstanding')
-- ${lgd} is the flat, exposure-weighted portfolio LGD (fallback only, e.g. a grade with
-- no charged-off history). Real LGD is segmented by grade via `lgd_by_grade` (written by
-- src/lgd.py: estimate_by_grade + write_grade_table, credibility-weighted toward the flat
-- value for thin grades) – this is what actually feeds Expected Loss below. Flat LGD
-- alone hid a real 44%-vs-60% spread by grade; see artifacts/lgd.json's "by_grade" key.

DROP TABLE IF EXISTS mart_loan_el;
CREATE TABLE mart_loan_el AS
SELECT
  s.loan_id,
  s.pd_hat,
  CASE '${ead_mode}'
  WHEN 'outstanding' THEN COALESCE(o.out_prncp, o.funded_amnt, o.loan_amnt)
  ELSE COALESCE(o.funded_amnt, o.loan_amnt)
  END AS ead,
  COALESCE(g.lgd, ${lgd}) AS lgd,
  s.pd_hat *
  (CASE '${ead_mode}'
  WHEN 'outstanding' THEN COALESCE(o.out_prncp, o.funded_amnt, o.loan_amnt)
  ELSE COALESCE(o.funded_amnt, o.loan_amnt)
  END) * COALESCE(g.lgd, ${lgd}) AS expected_loss
FROM mart_scores s
JOIN mart_loan_outcomes o USING (loan_id)
LEFT JOIN mart_loan_benchmark b USING (loan_id)
LEFT JOIN lgd_by_grade g ON g.grade = b.lc_grade;

DROP TABLE IF EXISTS mart_simulator_base;
CREATE TABLE mart_simulator_base AS
SELECT
  f.loan_id,
  f.issue_d,
  date_trunc('year', f.issue_d) AS vintage_year,
  f.split_set,
  f.loan_amnt,
  f.fico_mid,
  f.purpose,
  b.lc_grade,
  b.int_rate / 100.0 AS int_rate, -- stored as %, expose as
  fraction
  s.pd_hat,
  el.ead,
  el.expected_loss,
  o.default_flag,
  o.is_terminal
FROM mart_loan_features f
JOIN mart_loan_benchmark b USING (loan_id)
JOIN mart_scores s USING (loan_id)
JOIN mart_loan_el el USING (loan_id)
JOIN mart_loan_outcomes o USING (loan_id);

```

```

DROP TABLE IF EXISTS mart_portfolio_summary;
CREATE TABLE mart_portfolio_summary AS
WITH banded AS (
SELECT
*,
CASE
WHEN pd_hat < 0.05 THEN '1 Low (<5%)'
WHEN pd_hat < 0.10 THEN '2 Moderate (5-10%)'
WHEN pd_hat < 0.20 THEN '3 Elevated (10-20%)'
ELSE '4 High (20%+)'
END AS risk_band
FROM mart_simulator_base
)
SELECT
'grade' AS dimension, lc_grade AS bucket,
count(*) AS n_loans, sum(loan_amnt) AS funded,
round(avg(pd_hat), 4) AS avg_pd, sum(expected_loss) AS total_el,
round(avg(CASE WHEN is_terminal THEN default_flag END), 4) AS observed_default_rate,
sum(CASE WHEN pd_hat >= 0.20 THEN loan_amnt ELSE 0 END) AS high_risk_exposure
FROM banded GROUP BY 2
UNION ALL SELECT 'purpose', purpose, count(*), sum(loan_amnt), round(avg(pd_hat),4),
sum(expected_loss), round(avg(CASE WHEN is_terminal THEN default_flag END),4),
sum(CASE WHEN pd_hat >= 0.20 THEN loan_amnt ELSE 0 END)
FROM banded GROUP BY 2
UNION ALL SELECT 'vintage', CAST(extract('year' FROM vintage_year) AS VARCHAR), count(*), sum(loan_amnt),
round(avg(pd_hat),4), sum(expected_loss), round(avg(CASE WHEN is_terminal THEN default_flag END),4),
sum(CASE WHEN pd_hat >= 0.20 THEN loan_amnt ELSE 0 END)
FROM banded GROUP BY 2
UNION ALL SELECT 'risk_band', risk_band, count(*), sum(loan_amnt), round(avg(pd_hat),4),
sum(expected_loss), round(avg(CASE WHEN is_terminal THEN default_flag END),4),
sum(CASE WHEN pd_hat >= 0.20 THEN loan_amnt ELSE 0 END)
FROM banded GROUP BY 2
ORDER BY dimension, bucket;

SELECT dimension, count(*) AS buckets, sum(n_loans) AS loans FROM mart_portfolio_summary GROUP BY 1;

```

src/backtest.py

The realized-outcome policy backtest. Reloads the test-split, terminal loans with their REAL cash-flow fields (not the model's predictions about them), sweeps 60 PD cut-offs, and at each one computes both the model's projected profit and the ACTUAL realised profit (interest/fees collected minus funding cost minus servicing cost minus real credit losses). Comparing where each curve peaks is the real test of whether the model's chosen threshold would have been good policy — not just internally consistent with itself.

```

"""Realized-outcome backtest of the lending-policy curve (council rec #1).

The simulator's profit-vs-approval curve is built from the model's OWN predicted PD, so a
sceptic can call the optimum circular. This re-runs the same policy sweep on the
out-of-time TEST loans using their ACTUAL outcomes – realized cash flows and realized
charge-offs – and checks whether the profit peak lands in the same place.

Per loan (test, terminal only):
realized_net_cash = total_pymnt + recoveries - funded_amnt # actual interest/fees earned, net of
actual credit loss
funding = funded_amnt * r_f * T * b
servicing = funded_amnt * s * T
realized_profit = realized_net_cash - funding - servicing
realized_credit_loss = max(funded_amnt - total_rec_prncp - recoveries, 0) if charged off else 0

Model side at each cut-off uses the same P&L skeleton but with predicted PD and the
data-estimated LGD already baked into mart_simulator_base.expected_loss.
"""
from __future__ import annotations

import json
import pathlib

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from src.config import load
from src.db import connect

ROOT = pathlib.Path(__file__).resolve().parents[1]

def _load_test(con) -> pd.DataFrame:
    return con.execute(
        """
        SELECT s.loan_id, s.pd_hat, s.fico_mid, s.loan_amnt, s.int_rate, s.expected_loss,
        o.default_flag, o.funded_amnt, o.total_pymnt, o.total_rec_prncp, o.recoveries
        FROM mart_simulator_base s
        JOIN mart_loan_outcomes o USING (loan_id)
        WHERE s.split_set = 'test' AND o.is_terminal AND o.default_flag IS NOT NULL
        """
    ).fetchdf()

def _curves(df: pd.DataFrame, fico_min: float, rf: float, s: float, T: float, b: float,
            n_points: int = 60) -> pd.DataFrame:
    fa = df["funded_amnt"].to_numpy()
    df = df.assign(
        realized_net_cash=df["total_pymnt"] + df["recoveries"] - fa,
        funding=fa * rf * T * b,
        servicing=fa * s * T,

```

```

realized_credit_loss=np.where(
df["default_flag"] == 1,
np.clip(fa - df["total_rec_prncp"] - df["recoveries"], 0, None),
0.0,
),
)
df["realized_profit"] = df["realized_net_cash"] - df["funding"] - df["servicing"]
df["model_interest"] = df["loan_amnt"] * df["int_rate"] * T * b * (1.0 - df["pd_hat"])
df["model_profit"] = df["model_interest"] - df["funding"] - df["servicing"] - df["expected_loss"]

n_all = len(df)
rows = []
for cut in np.linspace(0.03, 0.40, n_points):
a = df[(df["pd_hat"] < cut) & (df["fico_mid"] >= fico_min)]
if len(a) == 0:
continue
rows.append(
dict(
pd_cut=float(cut),
approval_rate=len(a) / n_all,
pred_default_rate=float(a["pd_hat"].mean()),
actual_default_rate=float(a["default_flag"].mean()),
model_expected_loss=float(a["expected_loss"].sum()),
realized_credit_loss=float(a["realized_credit_loss"].sum()),
model_profit=float(a["model_profit"].sum()),
realized_profit=float(a["realized_profit"].sum()),
n=int(len(a)),
)
)
return pd.DataFrame(rows)

def _plot(curve: pd.DataFrame, path: str) -> None:
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
ax1.plot(curve["approval_rate"], curve["model_profit"] / 1e6, "o-", color="#c17f00", label="model-PD profit")
ax1.plot(curve["approval_rate"], curve["realized_profit"] / 1e6, "o-", color="#1f4e79", label="realized profit")
for c, col in [("model_profit", "#c17f00"), ("realized_profit", "#1f4e79")]:
r = curve.loc[curve[c].idxmax()]
ax1.axvline(r["approval_rate"], color=col, ls="--", lw=1)
ax1.set_xlabel("approval rate")
ax1.set_ylabel("portfolio profit ($M, T=3)")
ax1.set_title("Profit vs approval – model PD vs realized outcomes")
ax1.legend()

ax2.plot(curve["approval_rate"], curve["pred_default_rate"], "o-", color="#c17f00", label="predicted default rate")
ax2.plot(curve["approval_rate"], curve["actual_default_rate"], "o-", color="#1f4e79", label="actual default rate")
ax2.set_xlabel("approval rate")
ax2.set_ylabel("default rate of the approved book")
ax2.set_title("Decision calibration: predicted vs actual")
ax2.legend()
fig.tight_layout()

```

```

pathlib.Path(path).parent.mkdir(parents=True, exist_ok=True)
fig.savefig(path, dpi=130)
plt.close(fig)

def main() -> dict:
    cfg = load()
    sim = cfg["simulator"]
    con = connect(cfg["paths"]["duckdb"])
    df = _load_test(con)
    curve = _curves(
        df, sim["default_min_fico"], sim["cost_of_funds"], sim["servicing_cost"],
        sim["horizon_years"], sim["amort_factor"],
    )
    con.close()

    model_opt = curve.loc[curve["model_profit"].idxmax()]
    realized_opt = curve.loc[curve["realized_profit"].idxmax()]
    # profit left on the table by following the model optimum instead of the realized one
    at_model = curve.iloc[(curve["pd_cut"] - model_opt["pd_cut"]).abs().idxmin()]
    regret = float(realized_opt["realized_profit"] - at_model["realized_profit"])

    out = {
        "fico_min": sim["default_min_fico"],
        "model_optimum": {k: float(model_opt[k]) for k in
            ["pd_cut", "approval_rate", "pred_default_rate", "actual_default_rate",
            "model_profit", "realized_profit"]},
        "realized_optimum": {k: float(realized_opt[k]) for k in
            ["pd_cut", "approval_rate", "actual_default_rate",
            "realized_credit_loss", "realized_profit"]},
        "regret_following_model_optimum": regret,
        "loss_ratio_realized_over_model": float(
            curve["realized_credit_loss"].sum() / max(curve["model_expected_loss"].sum(), 1)
        ),
        "curve": curve.to_dict(orient="records"),
    }
    (ROOT / "artifacts").mkdir(exist_ok=True)
    (ROOT / "artifacts" / "backtest.json").write_text(json.dumps(out, indent=2))
    _plot(curve, f"{cfg['paths']['figures_dir']}/backtest_profit.png")

    print(f" model optimum : PD<{model_opt['pd_cut']:.3f} approval {model_opt['approval_rate']:.1%}")
    print(f" realized optimum : PD<{realized_opt['pd_cut']:.3f} approval
    {realized_opt['approval_rate']:.1%}")
    print(f" regret (following model opt) : ${regret/1e6:.1f}M")
    print(f" realized loss / model EL : {out['loss_ratio_realized_over_model']:.2f}x")
    return out

if __name__ == "__main__":
    main()

```

src/fairness.py

Disparate-impact / 4-5ths-rule check on the approval policy across income band, census region, and home ownership (the only demographic-adjacent fields this dataset has — no race/sex/age/national-origin data exists, stated directly in the output). `\_bootstrap\_air\_ci()` adds real statistical rigor: a 2000-sample stratified bootstrap gives a confidence interval on the adverse-impact ratio, Bonferroni-corrected for testing three dimensions at once, so a flag means the gap is real, not sampling noise.

```

"""Fairness & adverse-action check (council rec #3).

Lending Club data contains no protected-class attributes (race, sex, age, national
origin), so a true fair-lending test is impossible here – that is itself a finding, and
it is stated in the memo. What we CAN do is check the recommended policy for disparate
impact across the segments we do observe (income band, region, home ownership): the
"4/5ths rule" adverse-impact ratio on approval rates, plus outcome parity (realized
default rate of the approved book by group).

A production model would also owe ECOA / Reg B adverse-action reason codes – per-decision
SHAP attributions. That is noted, not built (see src/explain.py for a real GLOBAL
attribution and a small real per-loan sample – still not a full production reason-code
system).

Audit finding (fixed): the AIR was previously a bare point estimate with no uncertainty
and no correction for testing three dimensions at once. Both are now handled:
- a nonparametric bootstrap (resample approved/declined indicators within each group,
B=2000) gives a 95% CI on the AIR;
- a Bonferroni correction (alpha=0.05 / 3 dimensions tested) tightens the pass/fail
bar so "FLAG" isn't just noise from testing three things and expecting one to wobble.
"""
from __future__ import annotations

import json
import pathlib

import numpy as np
import pandas as pd

from src.config import load
from src.db import connect

ROOT = pathlib.Path(__file__).resolve().parents[1]
N_BOOTSTRAP = 2000
N_DIMENSIONS_TESTED = 3 # income_band, region, home_ownership
ALPHA = 0.05
ALPHA_BONFERRONI = ALPHA / N_DIMENSIONS_TESTED
RNG_SEED = 42

_REGION = {
    "Northeast": ["CT", "ME", "MA", "NH", "RI", "VT", "NJ", "NY", "PA"],
    "Midwest": ["IL", "IN", "MI", "OH", "WI", "IA", "KS", "MN", "MO", "NE", "ND", "SD"],
    "South": ["DE", "FL", "GA", "MD", "NC", "SC", "VA", "DC", "WV", "AL", "KY", "MS",
    "TN", "AR", "LA", "OK", "TX"],
    "West": ["AZ", "CO", "ID", "MT", "NV", "NM", "UT", "WY", "AK", "CA", "HI", "OR", "WA"],
}
_STATE2REGION = {s: r for r, ss in _REGION.items() for s in ss}

def _load(con) -> pd.DataFrame:
    df = con.execute(
        """
        SELECT s.loan_id, s.pd_hat, s.fico_mid,
        f.annual_inc, f.home_ownership, f.addr_state,
        o.default_flag, o.is_terminal

```

```

FROM mart_simulator_base s
JOIN mart_loan_features f USING (loan_id)
LEFT JOIN mart_loan_outcomes o USING (loan_id)
WHERE s.split_set = 'test'
"""
).fetchdf()
df["income_band"] = pd.cut(
df["annual_inc"], [-1, 40_000, 70_000, 120_000, 1e12],
labels=["<40k", "40-70k", "70-120k", "120k+"],
)
df["region"] = df["addr_state"].map(_STATE2REGION).fillna("Other")
return df

def _bootstrap_air_ci(
df: pd.DataFrame, col: str, approved: pd.Series, keep_groups, alpha: float,
n_boot: int = N_BOOTSTRAP, seed: int = RNG_SEED,
) -> tuple[float, float] | None:
"""Nonparametric bootstrap CI on the adverse-impact ratio: resample the approved/declined
indicator WITHIN each group (stratified – preserves group sizes), recompute AIR, repeat
n_boot times, take the percentile interval at the given alpha. Answers: given sampling
noise, how confident are we the AIR is really below 0.80, not just below it by chance?"""
sub = df[[col]].copy()
sub["approved"] = approved.to_numpy()
sub = sub[sub[col].isin(keep_groups)]
if sub[col].nunique() < 2:
return None
rng = np.random.default_rng(seed)
group_arrays = [g["approved"].to_numpy() for _, g in sub.groupby(col, observed=True)]
boot = np.empty(n_boot)
for i in range(n_boot):
rates = np.array([
arr[rng.integers(0, len(arr), len(arr))].mean() for arr in group_arrays
])
boot[i] = rates.min() / rates.max() if rates.max() > 0 else np.nan
boot = boot[~np.isnan(boot)]
if len(boot) == 0:
return None
lo = float(np.percentile(boot, 100 * alpha / 2))
hi = float(np.percentile(boot, 100 * (1 - alpha / 2)))
return lo, hi

def _by(df: pd.DataFrame, col: str, approved: pd.Series, min_group: int = 500) -> dict:
grp = df.assign(approved=approved).groupby(col, observed=True)
sizes = grp.size()
keep = sizes[sizes >= min_group].index # ignore tiny groups when scoring the rule
appr_rate = grp["approved"].mean()
appr_rate_scored = appr_rate.loc[appr_rate.index.isin(keep)]
term = df["is_terminal"] & (df["default_flag"].notna())
dd = df[approved & term]
obs_dr = dd.groupby(col, observed=True)["default_flag"].mean() if len(dd) else pd.Series(dtype=float)
air = (float(appr_rate_scored.min() / appr_rate_scored.max()))
if len(appr_rate_scored) and appr_rate_scored.max() > 0 else None)

```

```

ci = _bootstrap_air_ci(df, col, approved, keep, ALPHA_BONFERRONI) if air is not None else None
# "flagged" = we are CONFIDENT (at the Bonferroni-corrected level, accounting for testing
# 3 dimensions at once) the true AIR sits below 0.80 – not just that the point estimate
# does. A dimension can fail the raw 4/5ths point estimate but not be statistically
# distinguishable from 0.80 at this sample size; that distinction is the point of the CI.
flagged_significant = ci is not None and ci[1] < 0.8

return {
    "group_n": {str(k): int(v) for k, v in sizes.items()},
    "approval_rate": {str(k): round(float(v), 4) for k, v in appr_rate.items()},
    "adverse_impact_ratio": round(air, 3) if air is not None else None,
    "air_ci": [round(ci[0], 3), round(ci[1], 3)] if ci is not None else None,
    "air_ci_method": (
        f"nonparametric stratified bootstrap, n={N_BOOTSTRAP}, "
        f"{100 * (1 - ALPHA_BONFERRONI):.2f}% CI (Bonferroni-corrected for "
        f"{N_DIMENSIONS_TESTED} dimensions tested)"
    ),
    "passes_4_5ths_rule": (air is not None and air >= 0.8),
    "flagged_significant": flagged_significant,
    "approved_book_default_rate": {str(k): round(float(v), 4) for k, v in obs_dr.items()},
}

def main() -> dict:
    cfg = load()
    sim = cfg["simulator"]
    con = connect(cfg["paths"]["duckdb"])
    df = _load(con)
    con.close()

    approved = (df["pd_hat"] < sim["default_pd_cutoff"]) & (df["fico_mid"] >= sim["default_min_fico"])
    out = {
        "policy": {"pd_cutoff": sim["default_pd_cutoff"], "min_fico": sim["default_min_fico"]},
        "note": ("No protected-class attributes in Lending Club data – this is a disparate-impact "
        "proxy check, not a fair-lending test. A production model owes ECOA/Reg B "
        "adverse-action reason codes (per-decision SHAP); not built here. "
        f"AIR confidence intervals are Bonferroni-corrected for testing "
        f"{N_DIMENSIONS_TESTED} dimensions at once (alpha={ALPHA} -> "
        f"{ALPHA_BONFERRONI:.4f} per dimension)."),
        "overall_approval_rate": round(float(approved.mean()), 4),
        "by_income_band": _by(df, "income_band", approved),
        "by_region": _by(df, "region", approved),
        "by_home_ownership": _by(df, "home_ownership", approved),
    }
    (ROOT / "artifacts").mkdir(exist_ok=True)
    (ROOT / "artifacts" / "fairness.json").write_text(json.dumps(out, indent=2))

    for dim in ("by_income_band", "by_region", "by_home_ownership"):
        d = out[dim]
        sig = "significant FLAG" if d["flagged_significant"] else ("pass" if d["passes_4_5ths_rule"] else
        "flag not significant at Bonferroni level")
        print(f" {dim:18s} AIR = {d['adverse_impact_ratio']} CI {d['air_ci']} ({sig})")
    return out

if __name__ == "__main__":
    main()

```

src/monitor.py

A real, retrospective Population Stability Index between the pipeline's own train (2012-14) and test (2015-16) cohorts — on the model's own score and its strongest features. Explicitly labelled as retrospective, not a live production monitor, because there's no scored production traffic to watch continuously; this is the honest version of 'population stability monitoring' given what the data can actually support.

```
"""Real population-stability (PSI) check – audit finding fixed.
```

The dashboard's "Monitoring" page previously showed PSI as a wholly synthetic monthly series (no production traffic exists to compute a live one against). What we DO have, for real, is two genuinely different historical populations already built into the pipeline: the train cohort (loans issued 2012-01 .. 2014-12) and the test cohort (issued 2015-01 .. 2016-02) – a real, if retrospective, population-shift comparison, not a live monitoring feed. This module computes it honestly and labels it as exactly that: a train-vs-test population stability check, not a production drift monitor.

PSI, standard credit-risk definition, deciles cut on the REFERENCE (train) population:
PSI = sum over deciles of (pct_test - pct_train) * ln(pct_test / pct_train)
Conventional read: <0.10 no material shift, 0.10-0.25 moderate, >0.25 material shift – these thresholds are cited as-is from standard risk-monitoring practice, not derived here.

Computes PSI for:

- the model's own predicted-PD score (the single number a real risk team watches first)
- a handful of the model's strongest inputs (src/insight.py's feature-importance ranking)

Reads : v_model_frame, mart_scores

Writes : artifacts/monitor.json

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import pathlib
```

```
import numpy as np
```

```
import pandas as pd
```

```
from src.config import load
```

```
from src.db import connect
```

```
ROOT = pathlib.Path(__file__).resolve().parents[1]
```

```
N_BINS = 10
```

```
# The features actually worth watching – top of src/insight.py's permutation-importance
```

```
# ranking, i.e. the inputs the model leans on most, not an arbitrary list.
```

```
WATCH_FEATURES = ["fico_mid", "acc_open_past_24mths", "loan_to_income", "dti", "annual_inc"]
```

```
def _psi(reference: pd.Series, comparison: pd.Series, n_bins: int = N_BINS) -> dict:
```

```
    ref = reference.dropna()
```

```
    cmp_ = comparison.dropna()
```

```
    edges = np.unique(np.quantile(ref, np.linspace(0, 1, n_bins + 1)))
```

```
    if len(edges) < 3: # degenerate (near-constant) feature – PSI is meaningless
```

```
        return {"psi": None, "n_bins_effective": len(edges) - 1, "note": "too few distinct values for decile binning"}
```

```
    edges[0], edges[-1] = -np.inf, np.inf
```

```
    ref_counts = pd.cut(ref, edges, include_lowest=True).value_counts(sort=False)
```

```
    cmp_counts = pd.cut(cmp_, edges, include_lowest=True).value_counts(sort=False)
```

```
    ref_pct = (ref_counts / ref_counts.sum()).clip(lower=1e-6)
```

```
    cmp_pct = (cmp_counts / cmp_counts.sum()).clip(lower=1e-6)
```

```
    psi = float(((cmp_pct - ref_pct) * np.log(cmp_pct / ref_pct)).sum())
```

```
    return {"psi": round(psi, 4), "n_bins_effective": len(edges) - 1}
```

```

def _band(psi: float | None) -> str | None:
    if psi is None:
        return None
    if psi < 0.10:
        return "stable"
    if psi < 0.25:
        return "moderate shift"
    return "material shift"

def main() -> dict:
    cfg = load()
    con = connect(cfg["paths"]["duckdb"])

    scores = con.execute(
        "SELECT f.loan_id, f.split_set, s.pd_hat, f.fico_mid, f.acc_open_past_24mths, "
        " f.dti, f.annual_inc, f.loan_amnt "
        "FROM v_model_frame f JOIN mart_scores s USING (loan_id)"
    ).fetchdf()
    scores["loan_to_income"] = scores["loan_amnt"] / scores["annual_inc"].where(scores["annual_inc"] > 0)
    con.close()

    train = scores[scores["split_set"] == "train"]
    test = scores[scores["split_set"] == "test"]

    score_psi = _psi(train["pd_hat"], test["pd_hat"])
    feature_psi = {}
    for feat in WATCH_FEATURES:
        r = _psi(train[feat], test[feat])
        r["band"] = _band(r["psi"])
        feature_psi[feat] = r

    out = {
        "scope": (
            "Retrospective train-vs-test population stability – NOT a live production drift "
            "monitor (there is no scored production traffic in this project). train = loans "
            "issued 2012-01..2014-12 (306,462); test = 2015-01..2016-02 (333,721).",
        ),
        "method": "Decile PSI, deciles cut on the train (reference) population, standard formula.",
        "thresholds": {"stable": "< 0.10", "moderate shift": "0.10-0.25", "material shift": ">= 0.25"},
        "score_psi": {"score_psi", "band": _band(score_psi["psi"])},
        "feature_psi": feature_psi,
        "n_train": int(len(train)),
        "n_test": int(len(test)),
    }
    (ROOT / "artifacts").mkdir(exist_ok=True)
    (ROOT / "artifacts" / "monitor.json").write_text(json.dumps(out, indent=2))

    print(f" score PSI (predicted PD, train vs test) : {out['score_psi']['psi']} "
          f"({out['score_psi']['band']})")
    for feat, r in feature_psi.items():
        print(f" {feat:<24s} PSI = {r['psi']} ({r['band']})")
    return out

if __name__ == "__main__":
    main()

```

src/insight.py

Two supplementary real analyses. Global permutation feature importance: how much the model's AUC drops when one input is shuffled, computed directly on the shipped estimator (not assumed from domain knowledge) — a real global attribution, though not a per-loan one. And a reporting wrapper around src/lgd.py's segmented LGD, so the same credibility-weighted numbers that feed Expected Loss are also surfaced on their own.

```

"""Two real supplementary analyses added for the CREDENCE dashboard audit:

1. Global permutation feature importance of the ACTUAL trained model (model.joblib)
on the out-of-time test split – a real answer to "what drives the model", not a
per-loan SHAP explainer (out of scope here), but a genuine, reproducible global
attribution computed against the shipped estimator.

2. LGD segmented by grade (same method as lgd.py – 1 minus recovery rate on charged-off
TRAIN loans – grouped by Lending Club grade) so Expected Loss is legible as a range
instead of one flat portfolio-wide number. This is additive: it does NOT change the
headline LGD/EL figures already written by lgd.py / marts, which still use the
single exposure-weighted LGD (documented limitation, now made visible instead of
silently flat).

Reads : artifacts/model.joblib, v_model_frame (test split), mart_loan_outcomes,
mart_loan_benchmark (grade)
Writes : artifacts/insight.json
"""
from __future__ import annotations

import json
import pathlib

import joblib
import numpy as np
import pandas as pd
from sklearn.inspection import permutation_importance
from sklearn.metrics import roc_auc_score

from src import features
from src.config import load
from src.db import connect

ROOT = pathlib.Path(__file__).resolve().parents[1]

# Human-readable labels for the allowlisted features (spec §2.2) – used only for display.
LABEL = {
    "fico_mid": "Credit score (FICO)",
    "dti": "Debt-to-income ratio",
    "revol_util": "Revolving utilisation",
    "loan_to_income": "Loan-to-income ratio",
    "log_annual_inc": "Annual income (log)",
    "inq_last_6mths": "Inquiries, last 6 months",
    "delinq_2yrs": "Delinquencies, last 2 years",
    "bc_util": "Bankcard utilisation",
    "num_tl_90g_dpd_24m": "Trades 90+ DPD, last 24m",
    "pct_tl_nvr_dlq": "% trades never delinquent",
    "mo_sin_old_rev_tl_op": "Age of oldest revolving trade",
    "acc_open_past_24mths": "Accounts opened, last 24m",
    "mths_since_recent_inq": "Months since last inquiry",
    "mths_since_recent_bc": "Months since newest bankcard",
    "revol_bal": "Revolving balance",
    "total_bc_limit": "Total bankcard limit",
    "num_actv_bc_tl": "Active bankcard trades",
    "bc_open_to_buy": "Unused bankcard credit",

```

```

"num_tl_op_past_12m": "Trades opened, last 12m",
"annual_inc": "Annual income",
"num_accts_ever_120_pd": "Accounts ever 120+ DPD",
"percent_bc_gt_75": "% bankcards over 75% utilised",
"total_bal_ex_mort": "Total balance ex-mortgage",
"credit_history_months": "Credit history length",
"emp_length_years": "Employment length",
"pub_rec": "Public records",
"pub_rec_bankruptcies": "Public-record bankruptcies",
"open_acc": "Open accounts",
"total_acc": "Total accounts",
"mort_acc": "Mortgage accounts",
"loan_amnt": "Loan amount",
"home_ownership": "Home ownership",
"purpose": "Loan purpose",
"verification_status": "Income verification",
"application_type": "Application type",
}

def _feature_importance(con, cfg, sample_n: int = 15_000, n_repeats: int = 5) -> dict:
    bundle = joblib.load(cfg["paths"]["model"])
    model, feat_cols = bundle["model"], bundle["feature_columns"]

    frame = con.execute("SELECT * FROM v_model_frame WHERE split_set = 'test'").fetchdf()
    frame = features.engineer(frame)
    features.assert_no_leakage(frame.drop(columns=["default_flag"]))

    rng = np.random.default_rng(42)
    idx = rng.choice(len(frame), size=min(sample_n, len(frame)), replace=False)
    sample = frame.iloc[idx]
    X, y = sample[feat_cols], sample["default_flag"].to_numpy()

    baseline_auc = float(roc_auc_score(y, model.predict_proba(X)[: , 1]))

    result = permutation_importance(
        model, X, y, scoring="roc_auc", n_repeats=n_repeats, random_state=42, n_jobs=-1,
    )
    rows = [
        {
            "feature": col,
            "label": LABEL.get(col, col),
            "importance_mean": float(result.importances_mean[i]),
            "importance_std": float(result.importances_std[i]),
        }
        for i, col in enumerate(feat_cols)
    ]
    rows.sort(key=lambda r: r["importance_mean"], reverse=True)
    return {
        "method": "sklearn.inspection.permutation_importance", scoring="roc_auc",
        "sample_n": int(len(sample)),
        "n_repeats": n_repeats,
        "baseline_auc_on_sample": baseline_auc,
        "top": rows[:12],
        "note": (

```

```

"Global attribution on the shipped model, not per-loan SHAP – mean AUC drop when a "
"feature's values are shuffled, averaged over 5 repeats on a 15,000-loan sample of the "
"out-of-time test set. Per-decision reason codes (ECOA/Reg B adverse-action) would need "
"a real per-loan explainer; not built here."
),
}

def _lgd_by_grade(con, cfg) -> dict:
    """Reads the canonical segmented-LGD estimate – same function src/lgd.py uses to build
    the `lgd_by_grade` table that 06_marts.sql joins for Expected Loss. Single source of
    truth: this module reports it, it does not independently recompute it."""
    from src import lgd as lgd_mod

    flat = lgd_mod.resolve(cfg, con)
    by_grade = lgd_mod.estimate_by_grade(con, cfg["split"]["oot_cutoff"], flat)
    return {
        "method": (
            "1 – (principal repaid + recoveries) / funded, per grade, train charged-off loans "
            f"only, Buhlmann credibility-weighted toward the flat LGD (full_credibility_n="
            f"{lgd_mod.FULL_CREDIBILITY_N})."
        ),
        "by_grade": by_grade,
        "note": (
            "This is the LGD actually used in Expected Loss (mart_loan_el joins this table by "
            "grade, falling back to the flat portfolio LGD for any grade not present).",
        ),
    }

def main() -> dict:
    cfg = load()
    con = connect(cfg["paths"]["duckdb"])
    out = {
        "feature_importance": _feature_importance(con, cfg),
        "lgd_by_grade": _lgd_by_grade(con, cfg),
    }
    con.close()
    (ROOT / "artifacts").mkdir(exist_ok=True)
    (ROOT / "artifacts" / "insight.json").write_text(json.dumps(out, indent=2))
    print("Top features by permutation importance (AUC drop):")
    for r in out["feature_importance"]["top"][:8]:
        print(f" {r['label']}:<32} {r['importance_mean']:+.4f}")
    print("LGD by grade:")
    for r in out["lgd_by_grade"]["by_grade"]:
        print(f" {r['grade']}: {r['lgd']}")
    return out

if __name__ == "__main__":
    main()

```

src/explain.py

Real per-loan attribution on the actual trained model for a small stratified sample of real test loans (3 per risk band). For each loan, every feature is reset one at a time to the test population's typical value (median for numeric, mode for categorical) and the resulting real PD change is measured — a genuine, correct single-feature sensitivity on the real model. Explicitly documented as NOT an exact Shapley-value (SHAP) decomposition (no combinatorial averaging over feature orderings) and not a production per-decision reason-code system.

"""Real per-loan attribution on the trained model (audit finding: the dashboard's "why is this borrower risky?" panel runs on a synthetic borrower set and a hand-written scoring function, not the real model – labelled DEMO, correctly, but that means no real per-loan explanation exists anywhere in the project).

This computes one for a small real sample, honestly labelled for what it actually is: single-feature marginal-contribution attribution, NOT an exact Shapley-value (SHAP) decomposition. shap.Explainer's default tabular masker assumes numeric data and breaks on this project's mixed categorical/numeric feature set; rather than force-fit that, this implements the simpler, fully-inspectable method directly against the real model:

```
baseline_pd = model.predict_proba(loan) [loan's actual PD]
typical_pd_f = model.predict_proba(loan with feature f reset to the population's
typical value – median for numeric, mode for
categorical, computed on the test population)
contribution_f = baseline_pd - typical_pd_f
```

Positive contribution_f: this loan's actual value of f is pushing its PD UP relative to a typical applicant. This is a real, correct, single-order sensitivity on the actual trained model – not a combinatorial Shapley average over feature orderings, and it says so.

Reads : artifacts/model.joblib, v_model_frame (test split)
Writes : artifacts/explanations.json – a stratified sample (3 per risk band) of REAL test-set loans with real per-loan attribution.

```
"""
from __future__ import annotations

import json
import pathlib

import joblib
import numpy as np
import pandas as pd

from src import features
from src.config import load
from src.db import connect

ROOT = pathlib.Path(__file__).resolve().parents[1]
N_PER_BAND = 3
BAND_EDGES = [0.0, 0.05, 0.10, 0.20, 1.0]
BAND_LABELS = ["Low", "Medium", "High", "Critical"]

LABEL = {
    "fico_mid": "Credit score (FICO)", "dti": "Debt-to-income ratio",
    "revol_util": "Revolving utilisation", "loan_to_income": "Loan-to-income ratio",
    "log_annual_inc": "Annual income (log)", "annual_inc": "Annual income",
    "inq_last_6mths": "Inquiries, last 6 months", "delinq_2yrs": "Delinquencies, last 2 years",
    "bc_util": "Bankcard utilisation", "num_tl_90g_dpd_24m": "Trades 90+ DPD, last 24m",
    "pct_tl_nvr_dlq": "% trades never delinquent", "mo_sin_old_rev_tl_op": "Age of oldest revolving trade",
    "acc_open_past_24mths": "Accounts opened, last 24m", "mths_since_recent_inq": "Months since last inquiry",
    "mths_since_recent_bc": "Months since newest bankcard", "num_accts_ever_120_pd": "Accounts ever 120+ DPD",
    "percent_bc_gt_75": "% bankcards over 75% utilised", "total_bal_ex_mort": "Total balance ex-mortgage",
```

```

"credit_history_months": "Credit history length", "emp_length_years": "Employment length",
"pub_rec": "Public records", "pub_rec_bankruptcies": "Public-record bankruptcies",
"open_acc": "Open accounts", "total_acc": "Total accounts", "mort_acc": "Mortgage accounts",
"loan_amnt": "Loan amount", "home_ownership": "Home ownership", "purpose": "Loan purpose",
"verification_status": "Income verification", "application_type": "Application type",
"num_tl_op_past_12m": "Trades opened, last 12m",
"avg_cur_bal": "Average balance per account", "bc_open_to_buy": "Unused bankcard credit",
"num_actv_bc_tl": "Active bankcard trades", "tot_hi_cred_lim": "Total high credit limit",
"total_bc_limit": "Total bankcard limit", "tot_cur_bal": "Total current balance",
"tot_coll_amt": "Total in collections", "revol_bal": "Revolving balance",
}

def _typical_values(pop: pd.DataFrame, feat_cols: list[str]) -> dict:
    typical = {}
    for col in feat_cols:
        s = pop[col]
        typical[col] = s.mode(dropna=True).iloc[0] if s.dtype == object else float(s.median())
    return typical

def _explain_loan(model, feat_cols: list[str], row_df: pd.DataFrame, typical: dict, top_n: int = 6) -> dict:
    # row_df: a 1-row slice of the ORIGINAL multi-row frame (df.loc[[idx], feat_cols]) – keeps
    # each column's real dtype intact. A Series -> to_frame().T round trip silently corrupts
    # NaN handling (pd.NA vs np.nan) for mixed numeric/categorical rows and breaks sklearn's
    # array validation; replicating the row instead avoids that entirely.
    baseline_pd = float(model.predict_proba(row_df)[0, 1])
    batch = pd.concat([row_df] * len(feat_cols), ignore_index=True)
    for i, col in enumerate(feat_cols):
        batch.at[i, col] = typical[col]
    typical_pd = model.predict_proba(batch)[0, 1]
    contributions = [
        {"feature": col, "label": LABEL.get(col, col), "contribution_pp": round(float((baseline_pd -
        typical_pd[i]) * 100), 2)}
    ]
    for i, col in enumerate(feat_cols):
        contributions[i] = {"feature": col, "label": LABEL.get(col, col), "contribution_pp": round(float((baseline_pd -
        typical_pd[i]) * 100), 2)}
    contributions.sort(key=lambda c: abs(c["contribution_pp"]), reverse=True)
    return {"pd": round(baseline_pd, 4), "top_drivers": contributions[:top_n]}

def main(n_per_band: int = N_PER_BAND, seed: int = 20240611) -> dict:
    cfg = load()
    bundle = joblib.load(cfg["paths"]["model"])
    model, feat_cols = bundle["model"], bundle["feature_columns"]

    con = connect(cfg["paths"]["duckdb"])
    df = con.execute(
        "SELECT * FROM v_model_frame WHERE split_set = 'test'"
    ).fetchdf()
    df = features.engineer(df)
    features.assert_no_leakage(df.drop(columns=["default_flag"]))
    con.close()

    df["pd_hat"] = model.predict_proba(df[feat_cols])[0, 1]
    df["risk_band"] = pd.cut(df["pd_hat"], BAND_EDGES, labels=BAND_LABELS)

```

```

typical = _typical_values(df, feat_cols)

rng = np.random.default_rng(seed)
sample_idx = []
for band in BAND_LABELS:
    band_idx = df.index[df["risk_band"] == band].to_numpy()
    if len(band_idx) == 0:
        continue
    take = min(n_per_band, len(band_idx))
    sample_idx.extend(rng.choice(band_idx, size=take, replace=False).tolist())

loans = []
for idx in sample_idx:
    row = df.loc[idx]
    row_df = df.loc[[idx], feat_cols].reset_index(drop=True)
    expl = _explain_loan(model, feat_cols, row_df, typical)
    loans.append({
        "loan_id": int(row["loan_id"]),
        "risk_band": str(row["risk_band"]),
        "fico_mid": float(row["fico_mid"]),
        "dti": float(row["dti"]),
        "purpose": str(row["purpose"]),
        "loan_amnt": float(row["loan_amnt"]),
        **expl,
    })

out = {
    "method": (
        "Single-feature marginal-contribution attribution on the ACTUAL trained model "
        "(model.joblib), not a synthetic scoring function. For each loan, every feature "
        "is reset one at a time to the test population's typical value (median for "
        "numeric, mode for categorical) and the PD change is recorded. This is a real, "
        "correct sensitivity on the real model – it is NOT an exact Shapley-value (SHAP) "
        "decomposition (no combinatorial averaging over feature orderings); labelled "
        "accordingly, not oversold as SHAP."
    ),
    "sample": f"{n_per_band} real test-set loans per risk band, seed={seed}",
    "loans": loans,
}
(ROOT / "artifacts").mkdir(exist_ok=True)
(ROOT / "artifacts" / "explanations.json").write_text(json.dumps(out, indent=2))
print(f" explained {len(loans)} real test-set loans across {len(BAND_LABELS)} risk bands")
for loan in loans[:3]:
    top = loan["top_drivers"][0]
    print(f" loan {loan['loan_id']} PD={loan['pd']:.3f} top driver: {top['label']} "
          f"({top['contribution_pp']:+.1f}pp)")
return out

if __name__ == "__main__":
    main()

```

run_pipeline.py

The orchestrator: a named, ordered list of stages (raw -> stage -> features -> lgd -> train -> score -> marts -> export -> backtest -> fairness -> monitor -> insight -> explain -> check), each one a small dispatch function. `check` re-trains the model and hard-fails if the test AUC drifts beyond a tolerance from the last run — a real reproducibility guard, run in CI on every push.

```

#!/usr/bin/env python3
"""Stage runner for the Credit Risk & Lending Policy pipeline.

Usage:
python run_pipeline.py preflight # step 0 – decide the maturity window
python run_pipeline.py all # raw -> stage -> features -> train -> score -> marts -> export ->
check
python run_pipeline.py train # any single stage
python run_pipeline.py stage features marts export # a subset, in order

Stages: raw, stage, features, train, score, marts, export, check
"""
from __future__ import annotations

import json
import pathlib
import sys
import warnings

import numpy as np

# Some OpenBLAS builds emit spurious FP RuntimeWarnings from `matmul` inside the LBFGS
# solver. They do not affect results; keep the CLI output clean.
np.seterr(all="ignore")
for _m in ("invalid value encountered", "divide by zero encountered", "overflow encountered"):
    warnings.filterwarnings("ignore", message=_m, category=RuntimeWarning)

from src.config import load, sql_params
from src.db import connect, run_sql_file, show

ROOT = pathlib.Path(__file__).resolve().parent
STAGES = ["raw", "stage", "features", "lgd", "train", "score", "marts",
"export", "backtest", "fairness", "monitor", "insight", "explain", "check"]

def _banner(name: str) -> None:
    print(f"\n=== {name} ===")

def stage_raw(cfg, params, con):
    csv = pathlib.Path(cfg["data"]["raw_csv"])
    if not csv.exists():
        raise SystemExit(
            f"Raw CSV not found: {csv}\n"
            "Download 'wordsforthewise/lending-club' from Kaggle, unzip "
            "accepted_2007_to_2018Q4.csv into data/, or edit config.yaml -> data.raw_csv."
        )
    show(run_sql_file(con, "01_raw.sql", params))

def stage_stage(cfg, params, con):
    show(run_sql_file(con, "02_staging.sql", params))

def stage_features(cfg, params, con):

```

```

show(run_sql_file(con, "03_features_outcome.sql", params))

def stage_lgd(cfg, params, con):
    """Estimate LGD from charged-off train recoveries (council rec #2); feed it to the marts."""
    from src.lgd import main as lgd_main

    params["lgd"] = str(lgd_main())

def stage_train(cfg, params, con):
    from src.train import main as train_main

    auc = train_main()
    (ROOT / "artifacts").mkdir(exist_ok=True)
    (ROOT / "artifacts" / "_last_train_auc.json").write_text(json.dumps({"test_auc": auc}))

def stage_score(cfg, params, con):
    from src.score import main as score_main

    score_main()

def stage_marts(cfg, params, con):
    from src.lgd import resolve as resolve_lgd

    params["lgd"] = str(resolve_lgd(cfg, con)) # data-estimate (cached) or fixed
    print(f" EL uses LGD = {params['lgd']}")
    show(run_sql_file(con, "06_marts.sql", params))
    # star-schema views (BI exhibit) – safe to rebuild every run
    show(run_sql_file(con, "../schema/star_schema.sql", params))

def stage_export(cfg, params, con):
    out = pathlib.Path(cfg["paths"]["exports_dir"])
    out.mkdir(parents=True, exist_ok=True)
    for tbl, fname in [
        ("mart_simulator_base", "simulator_base.parquet"),
        ("mart_portfolio_summary", "portfolio_summary.parquet"),
        ("mart_loan_el", "loan_el.parquet"),
    ]:
        con.execute(f"COPY {tbl} TO '{out / fname}' (FORMAT PARQUET)")
        print(f" wrote {out / fname}")
    # small committed sample for the hosted Streamlit app
    con.execute(
        f"COPY (SELECT * FROM mart_simulator_base USING SAMPLE 50000 ROWS (reservoir, 42)) "
        f"TO '{out / 'simulator_base_sample.parquet'}' (FORMAT PARQUET)"
    )
    print(f" wrote {out / 'simulator_base_sample.parquet'}")

def stage_backtest(cfg, params, con):
    from src.backtest import main as backtest_main

```

```

backtest_main()

def stage_fairness(cfg, params, con):
    from src.fairness import main as fairness_main

    fairness_main()

def stage_monitor(cfg, params, con):
    from src.monitor import main as monitor_main

    monitor_main()

def stage_insight(cfg, params, con):
    from src.insight import main as insight_main

    insight_main()

def stage_explain(cfg, params, con):
    from src.explain import main as explain_main

    explain_main()

def stage_check(cfg, params, con):
    """Retrain and assert the test AUC reproduces – guards against a stale committed model."""
    from src.train import main as train_main

    prev = json.loads((ROOT / "artifacts" / "_last_train_auc.json").read_text())["test_auc"]
    auc = train_main()
    tol = cfg["model"]["auc_repro_tolerance"]
    delta = abs(auc - prev)
    print(f" reproduced test AUC {auc:.4f} vs {prev:.4f} (delta {delta:.4f}, tol {tol})")
    if delta > tol:
        raise SystemExit(f"AUC reproducibility FAILED: delta {delta:.4f} > tol {tol}")
    print(" reproducibility OK")

DISPATCH = {
    "raw": stage_raw, "stage": stage_stage, "features": stage_features,
    "lgd": stage_lgd, "train": stage_train, "score": stage_score, "marts": stage_marts,
    "export": stage_export, "backtest": stage_backtest, "fairness": stage_fairness,
    "monitor": stage_monitor, "insight": stage_insight, "explain": stage_explain,
    "check": stage_check,
}

def run_preflight(cfg, params):
    con = connect(cfg["paths"]["duckdb"])
    stage_raw(cfg, params, con)
    _banner("preflight – maturity by issue month (36-month loans)")
    rows = run_sql_file(con, "preflight_maturity.sql", params)

```

```

print(" issue_month | n | frac_terminal | frac_charged_off")
show(rows)
thr = cfg["maturity"]["terminal_fraction_threshold"]
mature = [r for r in rows if r[2] is not None and r[2] >= thr]
if mature:
    print(f"\n newest month with frac_terminal >= {thr}: {mature[-1][0]}")
    print(" -> set config.split.oort_cutoff a few months before this, and")
    print(" config.maturity.issue_year_max to that year (or earlier).")
con.close()

def main(argv: list[str]) -> None:
    cfg = load()
    params = sql_params(cfg)
    args = argv or ["all"]

    if args == ["preflight"]:
        run_preflight(cfg, params)
        return

    stages = STAGES if args == ["all"] else args
    unknown = [s for s in stages if s not in DISPATCH]
    if unknown:
        raise SystemExit(f"Unknown stage(s): {unknown}\nValid: {STAGES}")

    con = connect(cfg["paths"]["duckdb"])
    for s in stages:
        _banner(s)
        DISPATCH[s](cfg, params, con)
    con.close()
    print("\nDone.")

if __name__ == "__main__":
    main(sys.argv[1:])

```

tests/test_evaluate.py

Real assertions on the statistical code: the DeLong test correctly finds no significant difference between identical scores, correctly finds a significant difference between a better and a worse score, and the KS statistic stays in [0,1] and is higher for a genuinely better score.

```

"""Sanity tests for the DeLong test and KS. Run: python tests/test_evaluate.py"""
from __future__ import annotations

import pathlib
import sys

import numpy as np

sys.path.append(str(pathlib.Path(__file__).resolve().parents[1]))
from src.evaluate import delong_roc_test, ks_statistic

rng = np.random.default_rng(0)
n = 4000
y = rng.integers(0, 2, n)
# a good score and a slightly noisier version of it
good = y + rng.normal(0, 0.8, n)
noisier = y + rng.normal(0, 1.4, n)
random_score = rng.normal(0, 1, n)

def test_identical_scores_not_significant():
    r = delong_roc_test(y, good, good.copy())
    assert abs(r["auc_diff"]) < 1e-9
    assert r["p_value"] > 0.99
    assert not r["significant_at_0.05"]

def test_better_vs_random_is_significant():
    r = delong_roc_test(y, good, random_score)
    assert r["auc_a"] > r["auc_b"]
    assert r["p_value"] < 1e-6
    assert r["significant_at_0.05"]

def test_small_edge_direction():
    r = delong_roc_test(y, good, noisier)
    assert r["auc_diff"] > 0 # the less noisy score ranks better
    assert 0.0 <= r["p_value"] <= 1.0

def test_ks_bounds():
    assert 0.0 <= ks_statistic(y, good) <= 1.0
    assert ks_statistic(y, random_score) < ks_statistic(y, good)

if __name__ == "__main__":
    for name, fn in sorted(globals().items()):
        if name.startswith("test_") and callable(fn):
            fn()
    print(f"ok {name}")
    print("all evaluate tests passed")

```

tests/test_sim_core.py

Real assertions on the simulator's P&L math: the profit identity holds component-by-component, the threshold filter behaves correctly at its edges, and the backtest curve has a genuine interior optimum (not monotonic, i.e. approving everything isn't actually optimal) — catching a regression in the core formula, not just a smoke test.

```

"""Sanity tests for the simulator math. Run: python -m pytest -q (or python tests/test_sim_core.py)"""
from __future__ import annotations

import pathlib
import sys

import pandas as pd

sys.path.append(str(pathlib.Path(__file__).resolve().parents[1]))
from app.sim_core import AMORT_FACTOR, SERVICING_COST, approved_metrics, best_policy, money, profit_curve

DF = pd.DataFrame(
{
    "loan_id": range(6),
    "loan_amnt": [10_000.0] * 6,
    "fico_mid": [640, 660, 680, 700, 720, 740],
    "int_rate": [0.15] * 6,
    "pd_hat": [0.30, 0.20, 0.12, 0.08, 0.04, 0.02],
    "ead": [10_000.0] * 6,
    "expected_loss": [1350, 900, 540, 360, 180, 90],
    "lc_grade": list("EDCBAA"),
    "default_flag": [1, 1, 0, 0, 0, 0],
}
)

def test_thresholds_filter():
m = approved_metrics(DF, pd_cut=0.10, fico_min=660, cost_of_funds=0.04, horizon=3.0)
assert m["n"] == 3 # pd 0.08, 0.04, 0.02 and fico >= 660
assert m["approval_rate"] == 0.5
assert m["volume"] == 30_000.0

def test_profit_identity_and_components():
m = approved_metrics(DF, 0.40, 600, cost_of_funds=0.04, horizon=3.0)
# all six approved
b, s, T = AMORT_FACTOR, SERVICING_COST, 3.0
surv = sum(1 - p for p in DF["pd_hat"]) # 5.24
exp_interest = 10_000 * 0.15 * T * b * surv
exp_funding = 6 * 10_000 * 0.04 * T * b
exp_serv = 6 * 10_000 * s * T
assert abs(m["interest_income"] - exp_interest) < 1e-6
assert abs(m["funding_cost"] - exp_funding) < 1e-6
assert abs(m["servicing_cost"] - exp_serv) < 1e-6
assert abs(m["exp_profit"] - (m["interest_income"] - m["funding_cost"] - m["servicing_cost"] - m["exp_loss"])) < 1e-6

def test_curve_has_interior_optimum():
# amortisation factor + (1 - PD) haircut + servicing must make the curve turn over:
# the profit-maximising cut-off is NOT the loosest one.
c = profit_curve(DF, fico_min=600, cost_of_funds=0.04, horizon=3.0, n_points=40)
assert c["exp_profit"].idxmax() < len(c) - 1
assert c["exp_profit"].iloc[-1] < c["exp_profit"].max()

```

```

def test_empty_approved_set():
    m = approved_metrics(DF, pd_cut=0.001, fico_min=600, cost_of_funds=0.04, horizon=3.0)
    assert m == dict(approval_rate=0.0, volume=0.0, exp_default_rate=0.0, exp_loss=0.0,
        interest_income=0.0, funding_cost=0.0, servicing_cost=0.0,
        exp_profit=0.0, n=0)

def test_curve_and_best():
    c = profit_curve(DF, fico_min=600, cost_of_funds=0.04, horizon=3.0, n_points=20)
    assert len(c) == 20 and {"approval_rate", "exp_profit", "pd_cut"} <= set(c.columns)
    assert c["approval_rate"].is_monotonic_increasing
    bp = best_policy(c)
    assert bp["exp_profit"] == c["exp_profit"].max()

def test_money():
    assert money(1_500_000) == "$1.5M"
    assert money(2_400) == "$2.4k"
    assert money(90) == "$90"

if __name__ == "__main__":
    for name, fn in sorted(globals().items()):
        if name.startswith("test_") and callable(fn):
            fn()
    print(f"ok {name}")
    print("all simulator-core tests passed")

```

tests/test_methodology.py

New tests covering the audit-driven fixes: LGD credibility weighting genuinely shrinks a thin segment toward the flat figure and trusts a thick one; PSI reads near-zero for identical distributions and correctly flags a real 3-sigma shift as material; the fairness bootstrap CI correctly flags a real, deliberate approval-rate gap as significant and does NOT flag a synthetic case with no real gap.

```

"""Tests for the audit-driven fixes: credibility-weighted LGD, PSI, and the fairness
bootstrap CI. No DB dependency – these exercise the pure functions directly on small
synthetic frames, matching the existing lightweight test style. Run: python tests/test_methodology.py"""
from __future__ import annotations

import pathlib
import sys

import numpy as np
import pandas as pd

sys.path.append(str(pathlib.Path(__file__).resolve().parents[1]))
from src.fairness import _bootstrap_air_ci, _by
from src.monitor import _psi

def test_lgd_credibility_shrinks_thin_segments_to_flat():
    # Reimplements the exact formula in src/ldg.py::estimate_by_grade – kept independent
    # of the DB-backed function so this test needs no fixture data.
    FULL_CREDIBILITY_N = 2000

    def credible(n, lgd_raw, lgd_flat):
        credibility = n / (n + FULL_CREDIBILITY_N)
        return credibility * lgd_raw + (1 - credibility) * lgd_flat

    lgd_flat = 0.50
    # a grade with almost no charged-off history should land close to the flat portfolio LGD,
    # even if its own raw estimate is wildly different
    thin = credible(n=5, lgd_raw=0.90, lgd_flat=lgd_flat)
    assert abs(thin - lgd_flat) < 0.02, f"thin segment should shrink to ~flat, got {thin}"
    # a grade with abundant history should land close to its OWN raw estimate
    thick = credible(n=50_000, lgd_raw=0.90, lgd_flat=lgd_flat)
    assert abs(thick - 0.90) < 0.02, f"thick segment should trust its own data, got {thick}"

def test_psi_identical_distributions_near_zero():
    rng = np.random.default_rng(0)
    x = pd.Series(rng.normal(0, 1, 5000))
    y = pd.Series(rng.normal(0, 1, 5000)) # same distribution, different draw
    result = _psi(x, y)
    assert result["psi"] is not None
    assert result["psi"] < 0.02, f"same-distribution PSI should be ~0, got {result['psi']}"

def test_psi_shifted_distribution_is_large():
    rng = np.random.default_rng(0)
    x = pd.Series(rng.normal(0, 1, 5000))
    y = pd.Series(rng.normal(3, 1, 5000)) # a large, genuine population shift
    result = _psi(x, y)
    assert result["psi"] > 0.25, f"a 3-sigma population shift should read as material, got {result['psi']}"

def test_psi_degenerate_feature_does_not_crash():
    x = pd.Series([5.0] * 100) # constant – no real distribution to bin
    y = pd.Series([5.0] * 100)

```

```

result = _psi(x, y)
assert result["psi"] is None # can't be legitimately computed; must say so, not fabricate a number

def _synthetic_fairness_frame(n_per_group=2000, gap=True):
    """Two groups with a real, deliberate approval-rate gap (or none, if gap=False)."""
    rng = np.random.default_rng(1)
    rows = []
    for group, base_rate in [("low", 0.30 if gap else 0.60), ("high", 0.60)]:
        approved = rng.random(n_per_group) < base_rate
        rows.append(pd.DataFrame({
            "seg": group,
            "default_flag": rng.integers(0, 2, n_per_group).astype(float),
            "is_terminal": True,
        }).assign(_approved=approved))
    df = pd.concat(rows, ignore_index=True)
    approved = df.pop("_approved")
    return df, approved

def test_fairness_ci_flags_a_real_gap():
    df, approved = _synthetic_fairness_frame(gap=True)
    result = _by(df, "seg", approved, min_group=100)
    assert result["adverse_impact_ratio"] < 0.8
    assert result["air_ci"] is not None
    lo, hi = result["air_ci"]
    assert hi < 0.8, "a real, large gap (0.30 vs 0.60 approval) should stay below 0.8 even at the CI's upper bound"
    assert result["flagged_significant"] is True

def test_fairness_ci_does_not_flag_when_there_is_no_gap():
    df, approved = _synthetic_fairness_frame(gap=False)
    result = _by(df, "seg", approved, min_group=100)
    assert result["adverse_impact_ratio"] > 0.85
    assert result["flagged_significant"] is False

if __name__ == "__main__":
    for name, fn in sorted(globals().items()):
        if name.startswith("test_") and callable(fn):
            fn()
    print(f"ok {name}")
    print("all methodology tests passed")

```

CREDENCE Dashboard — Core Files

A static React/TypeScript SPA, zero backend. Below is the full core — entry point, routing, the REAL/DEMO data layer, shared libs, the base design-system components, and one representative page (Overview) showing how every page is actually built from that layer. The remaining pages and components follow the identical pattern and are listed at the end rather than repeated in full.

credence/src/main.tsx

Entry point. Mounts the app inside HashRouter (so client-side routing works on any static host with zero server config — no rewrite rules needed) and a settings context provider, and pulls in the two global stylesheets.

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { HashRouter } from "react-router-dom";
import "../styles/base.css";
import "../styles/app.css";
import { App } from "../App";
import { SettingsProvider } from "../app/settings";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <HashRouter>
      <SettingsProvider>
        <App />
      </SettingsProvider>
    </HashRouter>
  </StrictMode>,
);
```

credence/src/App.tsx

The route table: one <Route> per page (Overview, Portfolio, Borrowers, Decisions, Monitoring, Data/Model, Settings), wrapped in the persistent AppShell (the left navigation rail).

```
import { Route, Routes } from "react-router-dom";
import { AppShell } from "../components/AppShell";
import { Overview } from "../pages/Overview";
import { Portfolio } from "../pages/Portfolio";
import { Borrowers } from "../pages/Borrowers";
import { Decisions } from "../pages/Decisions";
import { Monitoring } from "../pages/Monitoring";
import { DataModel } from "../pages/DataModel";
import { Settings } from "../pages/Settings";

export function App() {
  return (
    <AppShell>
      <Routes>
        <Route path="/" element={<Overview />} />
        <Route path="/portfolio" element={<Portfolio />} />
        <Route path="/borrowers" element={<Borrowers />} />
        <Route path="/decisions" element={<Decisions />} />
        <Route path="/monitoring" element={<Monitoring />} />
        <Route path="/data-model" element={<DataModel />} />
        <Route path="/settings" element={<Settings />} />
        <Route path="*" element={<Overview />} />
      </Routes>
    </AppShell>
  );
}
```

credence/src/data/types.ts

The type contract that architecturally enforces the REAL/DEMO split: REAL interfaces (PortfolioTotals, BandRow, SegmentRow, FairnessDim, PsiResult, ExplainedLoan, ...) and DEMO interfaces (Borrower, MonthPoint, Alert, ...) are distinct types, so a synthetic value can't silently be passed where a real one is expected without TypeScript complaining.

```
import type { RiskBand } from "../lib/risk";

/* ----- REAL: pipeline aggregates ----- */

export interface PortfolioTotals {
  loans: number;
  exposure: number;
  expected_loss: number;
  observed_default_rate: number;
  lgd: number;
}

export interface BandRow {
  band: RiskBand;
  range: string;
  loans: number;
  exposure: number;
  expected_loss: number;
  observed_default_rate: number;
}

export interface SegmentRow {
  key: string;
  label: string;
  loans: number;
  exposure: number;
  avg_pd: number;
  expected_loss: number;
  observed_default_rate: number;
}

export interface ModelSplit {
  auc: number;
  gini: number;
  ks: number;
  brier: number;
  n: number;
  base_rate: number;
}

export interface DecileRow {
  bucket: number;
  n: number;
  mean_pd: number;
  obs_rate: number;
  lift: number;
}

export interface BacktestPoint {
  pd_cut: number;
  approval_rate: number;
  pred_default_rate: number;
  actual_default_rate: number;
  model_expected_loss: number;
  realized_credit_loss: number;
}
```

```

model_profit: number;
realized_profit: number;
n: number;
}

export interface FairnessGroup {
group: string;
approval_rate: number;
approved_book_default_rate: number;
}

export interface FairnessDim {
groups: FairnessGroup[];
air: number;
airCi: [number, number];
passes: boolean;
flaggedSignificant: boolean; // true if the CI's upper bound stays below 0.80 – a real
// finding, not noise, even after correcting for testing 3 dimensions at once (Bonferroni)
}

export interface FeatureImportanceRow {
feature: string;
label: string;
importance: number; // mean AUC drop when the feature is shuffled (permutation importance)
}

export interface LgdByGradeRow {
grade: string;
lgd: number;
n_charged_off_train: number;
}

export type PsiBand = "stable" | "moderate shift" | "material shift";

export interface PsiFeatureResult {
feature: string;
label: string;
psi: number | null;
band: PsiBand | null;
}

export interface PsiResult {
scope: string;
scorePsi: number;
scoreBand: PsiBand;
features: PsiFeatureResult[];
nTrain: number;
nTest: number;
}

export interface ExplainedLoanDriver {
feature: string;
label: string;
contributionPp: number; // percentage points of PD, real model output
}

```

```

export interface ExplainedLoan {
  loanId: number;
  riskBand: RiskBand;
  pd: number;
  ficoMid: number;
  dti: number;
  purpose: string;
  loanAmt: number;
  topDrivers: ExplainedLoanDriver[];
}

/* ----- DEMO: synthetic ----- */

export type Decision = "Approved" | "Declined" | "Manual review";
export type LoanStatus = "Current" | "Delinquent" | "Default" | "Paid";
export type EmploymentType =
  | "Full-time"
  | "Part-time"
  | "Self-employed"
  | "Contract"
  | "Retired";

export interface RiskDriver {
  factor: string;
  contribution: number; // ± points
  note: string;
}

export interface Borrower {
  loan_id: string;
  borrower_id: string;
  application_date: string;
  loan_amount: number;
  annual_income: number;
  employment_length: number;
  employment_type: EmploymentType;
  credit_score: number;
  debt_to_income: number;
  credit_history_length: number;
  home_ownership: "Rent" | "Mortgage" | "Own";
  loan_purpose: string;
  interest_rate: number;
  term: 36 | 60;
  delinquencies_2y: number;
  risk_score: number; // 0-100
  probability_of_default: number;
  risk_category: RiskBand;
  expected_loss: number;
  decision: Decision;
  loan_status: LoanStatus;
  drivers: RiskDriver[];
}

export interface MonthPoint {

  month: string; // YYYY-MM
  default_rate: number;
  delinquency_rate: number;
  expected_loss: number;
  avg_credit_score: number;
  avg_dti: number;
  psi: number;
}

export type Severity = "High" | "Medium" | "Low";

export interface Alert {
  id: string;
  severity: Severity;
  title: string;
  detail: string;
  magnitude: string;
  window: string;
  segment: string;
  date: string;
}

```

credence/src/data/real.ts

Every REAL number in the dashboard, transcribed from the pipeline's JSON artifacts. No runtime fetch — these are build-time constants, re-generated by hand from artifacts/*.json whenever the pipeline reruns. This is also where derived-but-still-real values live (e.g. `operatingPoint`'s precision/recall/F1, computed from the real risk-band aggregates at the PD $\geq$ 20% flag).

```
/**
 * REAL data – transcribed verbatim from the credit-risk-lending-policy pipeline outputs
 * (../artifacts/metrics.json, lgd.json, backtest.json, fairness.json, monitor.json,
 * insight.json, explanations.json, ../exports/portfolio_summary.parquet). Dataset:
 * Lending Club accepted loans (wordsworthwise mirror), 36-month term, issued
 * 2012-01..2016-02, 640,919 terminal loans, snapshot 2018Q4. Model: isotonic-calibrated
 * HistGradientBoosting on ~33 origination-time features. EAD = funded amount.
 *
 * Post-audit pass: LGD is now segmented by grade (Buhlmann credibility-weighted) and
 * feeds Expected Loss directly – not a single flat number. Fairness AIR carries a real
 * bootstrap confidence interval, Bonferroni-corrected for testing 3 dimensions. A real
 * (retrospective) population-stability check and a real per-loan attribution sample on
 * the actual trained model are new exports below (psi, explainedLoans).
 */
import type {
  BacktestPoint,
  BandRow,
  DecileRow,
  ExplainedLoan,
  FairnessDim,
  FeatureImportanceRow,
  LgdByGradeRow,
  ModelSplit,
  PortfolioTotals,
  PsiResult,
  SegmentRow,
} from "../types";

export const PROVENANCE = {
  dataset: "Lending Club accepted loans (wordsworthwise mirror)",
  window: "issued 2012-01 - 2016-02",
  loans: 640919,
  term: "36-month",
  snapshot: "2018Q4",
  model: "HistGradientBoosting, isotonic-calibrated, ~33 origination-time features",
  lgdBasis: "1 – (principal repaid + recoveries) / funded, over 40,596 charged-off training loans, segmented by grade (Buhlmann credibility)",
  eadBasis: "funded amount at origination",
  oot: "train issued < 2015-01 · test issued ≥ 2015-01",
} as const;

export const totals: PortfolioTotals = {
  loans: 640919,
  exposure: 8_157_055_225,
  expected_loss: 519_948_331,
  observed_default_rate: 0.141,
  lgd: 0.4995,
};

export const lgd = {
  value: 0.4995,
  recovery_rate: 0.5005,
  n_charged_off_train: 40596,
};
```

```

/* portfolio_summary.parquet – risk_band rows (EL now uses grade-segmented LGD) */
export const byRiskBand: BandRow[] = [
  { band: "Low", range: "PD < 5%", loans: 79767, exposure: 1_231_269_725, expected_loss: 19_982_689,
    observed_default_rate: 0.0304 },
  { band: "Medium", range: "5-10%", loans: 168911, exposure: 2_268_782_350, expected_loss: 81_881_569,
    observed_default_rate: 0.0741 },
  { band: "High", range: "10-20%", loans: 268899, exposure: 3_203_741_775, expected_loss: 226_915_840,
    observed_default_rate: 0.1493 },
  { band: "Critical", range: "20%+", loans: 123342, exposure: 1_453_261_375, expected_loss: 191_168_231,
    observed_default_rate: 0.286 },
];

/* portfolio_summary.parquet – grade rows (EL now uses grade-segmented LGD) */
export const byGrade: SegmentRow[] = [
  { key: "A", label: "Grade A", loans: 146017, exposure: 2_092_034_900, avg_pd: 0.0671, expected_loss:
    62_327_743, observed_default_rate: 0.0545 },
  { key: "B", label: "Grade B", loans: 221171, exposure: 2_775_406_775, avg_pd: 0.1192, expected_loss:
    152_389_162, observed_default_rate: 0.1127 },
  { key: "C", label: "Grade C", loans: 169208, exposure: 2_045_113_700, avg_pd: 0.1663, expected_loss:
    166_660_574, observed_default_rate: 0.1818 },
  { key: "D", label: "Grade D", loans: 77437, exposure: 928_241_475, avg_pd: 0.1998, expected_loss:
    98_111_177, observed_default_rate: 0.2388 },
  { key: "E", label: "Grade E", loans: 22124, exposure: 265_840_800, avg_pd: 0.2289, expected_loss:
    33_431_105, observed_default_rate: 0.2942 },
  { key: "F", label: "Grade F", loans: 4416, exposure: 43_697_875, avg_pd: 0.2515, expected_loss: 6_053_023,
    observed_default_rate: 0.3384 },
  { key: "G", label: "Grade G", loans: 546, exposure: 6_719_700, avg_pd: 0.2902, expected_loss: 975_542,
    observed_default_rate: 0.4096 },
];

/* portfolio_summary.parquet – purpose rows (top by exposure; EL uses grade-segmented LGD) */
export const byPurpose: SegmentRow[] = [
  { key: "debt_consolidation", label: "Debt consolidation", loans: 365917, exposure: 4_850_745_075, avg_pd:
    0.1412, expected_loss: 326_008_320, observed_default_rate: 0.1477 },
  { key: "credit_card", label: "Credit card", loans: 159351, exposure: 2_163_730_075, avg_pd: 0.1139,
    expected_loss: 113_892_500, observed_default_rate: 0.1179 },
  { key: "home_improvement", label: "Home improvement", loans: 36689, exposure: 440_613_200, avg_pd: 0.118,
    expected_loss: 25_406_572, observed_default_rate: 0.1255 },
  { key: "other", label: "Other", loans: 33517, exposure: 281_027_950, avg_pd: 0.1536, expected_loss:
    21_752_441, observed_default_rate: 0.1645 },
  { key: "major_purchase", label: "Major purchase", loans: 12311, exposure: 121_259_825, avg_pd: 0.1258,
    expected_loss: 7_652_082, observed_default_rate: 0.1317 },
  { key: "small_business", label: "Small business", loans: 6714, exposure: 93_400_000, avg_pd: 0.2064,
    expected_loss: 9_976_482, observed_default_rate: 0.2242 },
  { key: "medical", label: "Medical", loans: 7036, exposure: 52_142_175, avg_pd: 0.1525, expected_loss:
    3_987_156, observed_default_rate: 0.173 },
  { key: "car", label: "Car", loans: 6461, exposure: 51_794_650, avg_pd: 0.1247, expected_loss: 3_262_496,
    observed_default_rate: 0.1195 },
  { key: "house", label: "House", loans: 2516, exposure: 32_039_725, avg_pd: 0.1611, expected_loss:
    2_515_064, observed_default_rate: 0.1883 },
  { key: "moving", label: "Moving", loans: 4491, exposure: 30_776_275, avg_pd: 0.1542, expected_loss:
    2_439_518, observed_default_rate: 0.196 },
  { key: "vacation", label: "Vacation", loans: 4310, exposure: 24_306_175, avg_pd: 0.1473, expected_loss:
    1_896_626, observed_default_rate: 0.1607 },
];

```

```

/* simulator_base FICO bands (min FICO in data = 660) */
export const byFicoBand: SegmentRow[] = [
  { key: "660-690", label: "660-690", loans: 326260, exposure: 3_762_044_000, avg_pd: 0.1695, expected_loss: 0, observed_default_rate: 0.1766 },
  { key: "690-720", label: "690-720", loans: 196461, exposure: 2_678_040_000, avg_pd: 0.118, expected_loss: 0, observed_default_rate: 0.1236 },
  { key: "720-750", label: "720-750", loans: 74725, exposure: 1_098_972_000, avg_pd: 0.075, expected_loss: 0, observed_default_rate: 0.0823 },
  { key: "750+", label: "750+", loans: 43473, exposure: 617_999_000, avg_pd: 0.046, expected_loss: 0, observed_default_rate: 0.0532 },
];

/* fairness.json - income-band approval split (real) */
export const byIncomeBand: SegmentRow[] = [
  { key: "<40k", label: "Under $40k", loans: 69083, exposure: 0, avg_pd: 0, expected_loss: 0, observed_default_rate: 0.1105 },
  { key: "40-70k", label: "$40k-70k", loans: 128838, exposure: 0, avg_pd: 0, expected_loss: 0, observed_default_rate: 0.0974 },
  { key: "70-120k", label: "$70k-120k", loans: 99411, exposure: 0, avg_pd: 0, expected_loss: 0, observed_default_rate: 0.0911 },
  { key: "120k+", label: "$120k+", loans: 37125, exposure: 0, avg_pd: 0, expected_loss: 0, observed_default_rate: 0.0836 },
];

/* portfolio_summary.parquet - vintage rows (EL uses grade-segmented LGD) */
export const byVintage: SegmentRow[] = [
  { key: "2012", label: "2012", loans: 43470, exposure: 507_799_125, avg_pd: 0.1353, expected_loss: 33_272_352, observed_default_rate: 0.1358 },
  { key: "2013", label: "2013", loans: 100422, exposure: 1_272_091_475, avg_pd: 0.1265, expected_loss: 76_549_011, observed_default_rate: 0.1233 },
  { key: "2014", label: "2014", loans: 162570, exposure: 2_046_040_750, avg_pd: 0.1356, expected_loss: 131_149_184, observed_default_rate: 0.1373 },
  { key: "2015", label: "2015", loans: 283173, exposure: 3_626_461_100, avg_pd: 0.1366, expected_loss: 234_475_061, observed_default_rate: 0.1489 },
  { key: "2016", label: "2016", loans: 51284, exposure: 704_662_775, avg_pd: 0.1322, expected_loss: 44_502_719, observed_default_rate: 0.1484 },
];

/* metrics.json */
export const modelTest: ModelSplit = { auc: 0.6907, gini: 0.3813, ks: 0.2782, brier: 0.1194, n: 333721, base_rate: 0.1488 };
export const modelTrain: ModelSplit = { auc: 0.7137, gini: 0.4275, ks: 0.308, brier: 0.1068, n: 306462, base_rate: 0.1325 };

export const benchmark = { hgb: 0.6907, logistic: 0.6804, grade_only: 0.6688 };
export const delong = { z: 19.57, p_value: 3.02e-85, auc_diff: 0.0218 };

export const decile: DecileRow[] = [
  { bucket: 0, n: 33373, mean_pd: 0.031, obs_rate: 0.0319, lift: 0.214 },
  { bucket: 1, n: 33372, mean_pd: 0.0557, obs_rate: 0.0582, lift: 0.391 },
  { bucket: 2, n: 33372, mean_pd: 0.0742, obs_rate: 0.0822, lift: 0.552 },
  { bucket: 3, n: 33372, mean_pd: 0.0924, obs_rate: 0.103, lift: 0.692 },
  { bucket: 4, n: 33372, mean_pd: 0.1119, obs_rate: 0.1232, lift: 0.828 },
  { bucket: 5, n: 33372, mean_pd: 0.1329, obs_rate: 0.1468, lift: 0.986 },
];

```

```

{ bucket: 6, n: 33371, mean_pd: 0.1533, obs_rate: 0.1728, lift: 1.161 },
{ bucket: 7, n: 33373, mean_pd: 0.1833, obs_rate: 0.2025, lift: 1.361 },
{ bucket: 8, n: 33372, mean_pd: 0.2227, obs_rate: 0.2432, lift: 1.635 },
{ bucket: 9, n: 33372, mean_pd: 0.2977, obs_rate: 0.3242, lift: 2.179 },
];

/**
 * Precision / Recall / F1 are NOT in the pipeline artifacts (they need a decision
 * threshold). These are computed at the PD ≥ 0.20 "high-risk flag" from the real
 * risk-band aggregates: positives = Σ loans.observed_default_rate.
 */
const _pos = byRiskBand.reduce((s, b) => s + b.loans * b.observed_default_rate, 0);
const _crit = byRiskBand[3];
const _tp = _crit.loans * _crit.observed_default_rate;
const _fp = _crit.loans - _tp;
const _fn = _pos - _tp;
export const operatingPoint = {
  cutoff: 0.2,
  precision: _tp / _crit.loans,
  recall: _tp / _pos,
  f1: (2 * _tp) / (2 * _tp + _fp + _fn),
  tp: Math.round(_tp),
  fp: Math.round(_fp),
  fn: Math.round(_fn),
};

/* backtest.json – 60-row realized-outcome curve (model_expected_loss / model_profit now
reflect grade-segmented LGD, not a flat multiplier)
tuple: [pd_cut, approval_rate, pred_dr, actual_dr, model_EL, realized_loss, model_profit,
realized_profit, n] */
const _curve: number[][] = [
  [0.03, 0.043, 0.021, 0.0205, 2268500, 2140956, 3525600, 170809, 14338], [0.0363, 0.0643, 0.0251, 0.025, 4075700, 3991518
  , 5685946, 865750, 21448], [0.0425, 0.0879, 0.029, 0.03, 6362680, 6579019, 8310880, 1555362, 29340], [0.0488, 0.122, 0.
  0336, 0.0347, 10038485, 10348727, 12190368, 3299673, 40725], [0.0551, 0.1505, 0.0371, 0.0387, 13532641, 14217627, 156
  05806, 4714453, 50212], [0.0614, 0.1778, 0.0403, 0.0426, 17197981, 18329069, 18823219, 6018301, 59348], [0.0676, 0.21
  01, 0.044, 0.0468, 21832825, 23604986, 22528433, 7547695, 70122], [0.0739, 0.2451, 0.0478, 0.051, 27243294, 29480676,
  26525951, 9603010, 81780], [0.0802, 0.2811, 0.0516, 0.0549, 33247697, 36040683, 30789724, 11904117, 93813], [0.0864,
  0.3138, 0.0549, 0.059, 39034435, 42985636, 34491781, 13384336, 104735], [0.0927, 0.3468, 0.0582, 0.063, 45202220, 502
  79950, 38030033, 15011317, 115743], [0.099, 0.3802, 0.0615, 0.0669, 51780563, 57829916, 41639867, 16743820, 126877],
  [0.1053, 0.4138, 0.0648, 0.0706, 58823639, 65698960, 45051800, 18697571, 138110], [0.1115, 0.4408, 0.0675, 0.0736, 64
  733878, 72411774, 47614636, 20030766, 147112], [0.1178, 0.4729, 0.0706, 0.0767, 72139994, 80453836, 50572283, 220138
  24, 157828], [0.1241, 0.5044, 0.0738, 0.0803, 79787534, 89405392, 53258881, 23208563, 168336], [0.1303, 0.5342, 0.076
  8, 0.0837, 87357600, 98132973, 55563515, 24252785, 178277], [0.1366, 0.5638, 0.0798, 0.0869, 95117175, 106760814, 576
  41730, 25576286, 188139], [0.1429, 0.5953, 0.0829, 0.0908, 103899256, 117186651, 59625590, 26172155, 198670], [0.149
  2, 0.6248, 0.0859, 0.0941, 112272133, 127175430, 61335322, 26695699, 208494], [0.1554, 0.6536, 0.0889, 0.0972, 120922
  350, 136927470, 62764363, 27404493, 218116], [0.1617, 0.6783, 0.0914, 0.0999, 128525100, 145714095, 63892927, 280453
  68, 226351], [0.168, 0.7015, 0.0938, 0.1027, 135987059, 154160814, 64689040, 28435228, 234118], [0.1742, 0.7234, 0.09
  62, 0.1052, 143268129, 162658950, 65162530, 28521794, 241425], [0.1805, 0.7433, 0.0983, 0.1077, 150229111, 170687866
  , 65448059, 28451459, 248068], [0.1868, 0.7615, 0.1004, 0.1097, 156753272, 178155776, 65514500, 28341273, 254144], [0
  .1931, 0.7818, 0.1027, 0.1125, 164420163, 187467655, 65417846, 27675637, 260918], [0.1993, 0.7949, 0.1042, 0.1143, 16
  9545990, 193585688, 65242884, 27293097, 265289], [0.2056, 0.8126, 0.1064, 0.1166, 176568552, 201825097, 64767908, 26
  882651, 271182], [0.2119, 0.8248, 0.1079, 0.1184, 181591477, 208121514, 64314934, 26034680, 275241], [0.2181, 0.8393
  , 0.1097, 0.1204, 187717949, 214997754, 63580208, 25640935, 280097], [0.2244, 0.8511, 0.1113, 0.1221, 192848725, 2210
  57582, 62843062, 24966549, 284024], [0.2307, 0.8657, 0.1132, 0.1241, 199412211, 228813992, 61670325, 23797535, 28888
  8], [0.2369, 0.8786, 0.115, 0.1261, 205388230, 235686254, 60504885, 22906042, 293213], [0.2432, 0.8928, 0.117, 0.1281

```

```
,212026908,243152329,59096668,22167637,297951],[0.2495,0.9047,0.1187,0.1299,217826866,249993168,57808303
,21148882,301912],[0.2558,0.9155,0.1203,0.1318,223322778,256762938,56469991,19674065,305525],[0.262,0.92
49,0.1217,0.1333,228154550,262369548,55200181,18774105,308642],[0.2683,0.9336,0.123,0.1348,232909668,268
134174,53863606,17620042,311553],[0.2746,0.9424,0.1244,0.1363,237794279,274079902,52417502,16301386,3145
13],[0.2808,0.9493,0.1255,0.1376,241758149,279186143,51176794,14921840,316792],[0.2871,0.9546,0.1264,0.1
385,244838525,282762389,50182962,14272428,318562],[0.2934,0.9593,0.1272,0.1393,247782517,286008490,49219
249,13905520,320142],[0.2997,0.9642,0.1281,0.1403,250775890,289539326,48156131,13113756,321776],[0.3059,
0.9686,0.1289,0.1412,253538899,292915993,47128539,12280121,323227],[0.3122,0.9713,0.1294,0.1418,25542182
5,295090717,46410232,11783080,324154],[0.3185,0.9745,0.13,0.1424,257578849,297559515,45558815,11263630,3
25209],[0.3247,0.9786,0.1308,0.1433,260428529,300856562,44394908,10519133,326574],[0.331,0.9804,0.1311,0
.1437,261798333,302601027,43813850,9929043,327184],[0.3373,0.9827,0.1316,0.1443,263571835,304691624,4303
4019,9339558,327961],[0.3436,0.9861,0.1323,0.145,265921866,307405534,41935117,8646030,329070],[0.3498,0.
9878,0.1327,0.1454,267259036,308958587,41288709,8257840,329640],[0.3561,0.9895,0.1331,0.1458,268542966,3
10511644,40667245,7764383,330217],[0.3624,0.9907,0.1334,0.1462,269507955,311706104,40190775,7349883,3306
24],[0.3686,0.9919,0.1336,0.1465,270437169,312874407,39715684,6985474,331010],[0.3749,0.993,0.1339,0.146
7,271348348,313825714,39222557,6721101,331391],[0.3812,0.9936,0.1341,0.1468,271888874,314451287,38940705
,6528929,331599],[0.3875,0.9945,0.1343,0.1471,272655751,315491418,38526282,6136823,331893],[0.3937,0.994
9,0.1344,0.1472,272977871,315813476,38344381,6045411,332025],[0.4,0.9956,0.1346,0.1474,273612203,3166474
63,37983670,5686030,332262],
];

export const backtest: BacktestPoint[] = _curve.map((r) => ({
  pd_cut: r[0],
  approval_rate: r[1],
  pred_default_rate: r[2],
  actual_default_rate: r[3],
  model_expected_loss: r[4],
  realized_credit_loss: r[5],
  model_profit: r[6],
  realized_profit: r[7],
  n: r[8],
}));

export const backtestMeta = {
  model_optimum: { pd_cut: 0.187, approval_rate: 0.762, actual_default_rate: 0.11, model_profit: 65_514_500,
  realized_profit: 28_341_273 },
  realized_optimum: { pd_cut: 0.174, approval_rate: 0.723, actual_default_rate: 0.105, realized_credit_loss:
  162_658_950, realized_profit: 28_521_794 },
  regret: 180_521,
  loss_ratio: 1.148,
  min_cut: 0.03,
  max_cut: 0.4,
};

/* fairness.json – AIR now carries a real bootstrap 98.33% CI (Bonferroni-corrected for
testing 3 dimensions at once, n=2000 resamples) instead of a bare point estimate. */
export const fairness: Record<"income" | "region" | "homeOwnership", FairnessDim> = {
  income: {
    groups: [
      { group: "Under $40k", approval_rate: 0.3581, approved_book_default_rate: 0.1105 },
      { group: "$40k-70k", approval_rate: 0.595, approved_book_default_rate: 0.0974 },
      { group: "$70k-120k", approval_rate: 0.7719, approved_book_default_rate: 0.0911 },
      { group: "$120k+", approval_rate: 0.8612, approved_book_default_rate: 0.0836 },
    ],
    air: 0.416,
  },
};
```

```

    airCi: [0.41, 0.421],
    passes: false,
    flaggedSignificant: true,
  },
  region: {
    groups: [
      { group: "Northeast", approval_rate: 0.647, approved_book_default_rate: 0.0994 },
      { group: "Midwest", approval_rate: 0.6195, approved_book_default_rate: 0.0886 },
      { group: "South", approval_rate: 0.6196, approved_book_default_rate: 0.0976 },
      { group: "West", approval_rate: 0.6313, approved_book_default_rate: 0.0906 },
    ],
    air: 0.958,
    airCi: [0.947, 0.964],
    passes: true,
    flaggedSignificant: false,
  },
  homeOwnership: {
    groups: [
      { group: "Rent", approval_rate: 0.5107, approved_book_default_rate: 0.1056 },
      { group: "Own", approval_rate: 0.5955, approved_book_default_rate: 0.1033 },
      { group: "Mortgage", approval_rate: 0.7464, approved_book_default_rate: 0.0855 },
    ],
    air: 0.684,
    airCi: [0.679, 0.689],
    passes: false,
    flaggedSignificant: true,
  },
  };

export const overallApprovalRate = 0.6282;

/**
 * insight.json – global permutation feature importance of the shipped model
 * (model.joblib), computed on a 15,000-loan sample of the out-of-time test split,
 * 5 repeats, scoring = ROC AUC. This is a real attribution of the actual estimator –
 * not per-loan SHAP. See `explainedLoans` below for a real per-loan sample instead.
 */
export const featureImportance: FeatureImportanceRow[] = [
  { feature: "fico_mid", label: "Credit score (FICO)", importance: 0.02 },
  { feature: "acc_open_past_24mths", label: "Accounts opened, last 24m", importance: 0.0181 },
  { feature: "dti", label: "Debt-to-income ratio", importance: 0.0119 },
  { feature: "loan_to_income", label: "Loan-to-income ratio", importance: 0.0119 },
  { feature: "mths_since_recent_bc", label: "Months since newest bankcard", importance: 0.0052 },
  { feature: "purpose", label: "Loan purpose", importance: 0.0052 },
  { feature: "num_tl_op_past_12m", label: "Trades opened, last 12m", importance: 0.0051 },
  { feature: "inq_last_6mths", label: "Inquiries, last 6 months", importance: 0.0039 },
  { feature: "percent_bc_gt_75", label: "% bankcards over 75% utilised", importance: 0.0033 },
  { feature: "loan_amnt", label: "Loan amount", importance: 0.0033 },
  { feature: "num_actv_bc_tl", label: "Active bankcard trades", importance: 0.0031 },
  { feature: "mths_since_recent_inq", label: "Months since last inquiry", importance: 0.0028 },
];
export const featureImportanceMeta = {
  method: "sklearn permutation_importance, scoring=roc_auc",
  sampleN: 15000,
  nRepeats: 5,

```

```

baselineAuc: 0.68,
};

/**
 * insight.json / lgd.json – LGD segmented by grade, Buhlmann credibility-weighted toward
 * the flat portfolio LGD for thin grades. This is what actually feeds Expected Loss
 * (sql/06_marts.sql joins this by grade) – not a supplementary side stat.
 */
export const lgdByGrade: LgdByGradeRow[] = [
  { grade: "A", lgd: 0.4641, n_charged_off_train: 3458 },
  { grade: "B", lgd: 0.4766, n_charged_off_train: 11790 },
  { grade: "C", lgd: 0.5002, n_charged_off_train: 13090 },
  { grade: "D", lgd: 0.5269, n_charged_off_train: 8582 },
  { grade: "E", lgd: 0.5351, n_charged_off_train: 2837 },
  { grade: "F", lgd: 0.5258, n_charged_off_train: 766 },
  { grade: "G", lgd: 0.503, n_charged_off_train: 73 },
];

/**
 * monitor.json – a REAL, but retrospective, population-stability check between the
 * pipeline's own train (2012-14) and test (2015-16) cohorts. Not a live production
 * drift monitor – there is no scored production traffic in this project – and it says
 * so plainly wherever it's shown.
 */
export const psi: PsiResult = {
  scope: "Retrospective train-vs-test population stability, not a live production monitor.",
  scorePsi: 0.0065,
  scoreBand: "stable",
  features: [
    { feature: "fico_mid", label: "Credit score (FICO)", psi: 0.0038, band: "stable" },
    { feature: "acc_open_past_24mths", label: "Accounts opened, last 24m", psi: 0.0247, band: "stable" },
    { feature: "loan_to_income", label: "Loan-to-income ratio", psi: 0.004, band: "stable" },
    { feature: "dti", label: "Debt-to-income ratio", psi: 0.03, band: "stable" },
    { feature: "annual_inc", label: "Annual income", psi: 0.0067, band: "stable" },
  ],
  nTrain: 306462,
  nTest: 333721,
};

/**
 * explanations.json – a real per-loan attribution sample: 12 real out-of-time test loans
 * (3 per risk band), each explained by resetting one feature at a time to the test
 * population's typical value and measuring the real PD change on the actual trained
 * model. This is a genuine sensitivity on the real model – explicitly NOT an exact
 * Shapley-value (SHAP) decomposition, and not a full per-decision reason-code system.
 */
export const explainedLoans: ExplainedLoan[] = [
  { loanId: 65097230, riskBand: "Low", pd: 0.0328, ficoMid: 737, dti: 30.22, purpose: "credit_card",
    loanAmnt: 8000, topDrivers: [{ feature: "fico_mid", label: "Credit score (FICO)", contributionPp: -3.33
    }, { feature: "bc_open_to_buy", label: "Unused bankcard credit", contributionPp: -0.77 }, { feature:
    "purpose", label: "Loan purpose", contributionPp: -0.5 } ] },
  { loanId: 59120926, riskBand: "Low", pd: 0.014, ficoMid: 797, dti: 14.19, purpose: "debt_consolidation",
    loanAmnt: 12000, topDrivers: [{ feature: "fico_mid", label: "Credit score (FICO)", contributionPp: -2.46
    }, { feature: "revol_bal", label: "Revolving balance", contributionPp: 0.19 }, { feature:
    "mths_since_recent_inq", label: "Months since last inquiry", contributionPp: 0.19 } ] },
];

```

```

{ loanId: 60800555, riskBand: "Low", pd: 0.0342, ficoMid: 697, dti: 10.79, purpose: "debt_consolidation",
  loanAmnt: 12000, topDrivers: [{ feature: "bc_open_to_buy", label: "Unused bankcard credit",
    contributionPp: 0.42 }, { feature: "total_bal_ex_mort", label: "Total balance ex-mortgage",
    contributionPp: 0.42 }, { feature: "avg_cur_bal", label: "Average balance per account", contributionPp:
    -0.41 } ] },
{ loanId: 59312381, riskBand: "Medium", pd: 0.0769, ficoMid: 662, dti: 5.0, purpose: "home_improvement",
  loanAmnt: 5000, topDrivers: [{ feature: "loan_to_income", label: "Loan-to-income ratio", contributionPp:
    -2.6 }, { feature: "dti", label: "Debt-to-income ratio", contributionPp: -2.44 }, { feature:
    "percent_bc_gt_75", label: "% bankcards over 75% utilised", contributionPp: -1.46 } ] },
{ loanId: 60415164, riskBand: "Medium", pd: 0.0834, ficoMid: 697, dti: 22.25, purpose:
  "debt_consolidation", loanAmnt: 14500, topDrivers: [{ feature: "num_tl_op_past_12m", label: "Trades
  opened, last 12m", contributionPp: -1.94 }, { feature: "fico_mid", label: "Credit score (FICO)",
  contributionPp: -1.5 }, { feature: "delinq_2yrs", label: "Delinquencies, last 2 years", contributionPp:
  1.38 } ] },
{ loanId: 50353355, riskBand: "Medium", pd: 0.0807, ficoMid: 697, dti: 29.92, purpose: "credit_card",
  loanAmnt: 6000, topDrivers: [{ feature: "dti", label: "Debt-to-income ratio", contributionPp: 1.7 }, {
  feature: "purpose", label: "Loan purpose", contributionPp: -1.53 }, { feature: "loan_to_income", label:
  "Loan-to-income ratio", contributionPp: -1.02 } ] },
{ loanId: 41183324, riskBand: "High", pd: 0.1368, ficoMid: 677, dti: 32.12, purpose: "credit_card",
  loanAmnt: 10000, topDrivers: [{ feature: "dti", label: "Debt-to-income ratio", contributionPp: 3.98 }, {
  feature: "acc_open_past_24mths", label: "Accounts opened, last 24m", contributionPp: 3.33 }, { feature:
  "purpose", label: "Loan purpose", contributionPp: -2.84 } ] },
{ loanId: 58632562, riskBand: "High", pd: 0.1118, ficoMid: 662, dti: 12.92, purpose: "credit_card",
  loanAmnt: 7700, topDrivers: [{ feature: "purpose", label: "Loan purpose", contributionPp: -2.59 }, {
  feature: "total_acc", label: "Total accounts", contributionPp: 1.75 }, { feature: "num_tl_op_past_12m",
  label: "Trades opened, last 12m", contributionPp: 1.54 } ] },
{ loanId: 52446317, riskBand: "High", pd: 0.1039, ficoMid: 667, dti: 8.34, purpose: "credit_card",
  loanAmnt: 18000, topDrivers: [{ feature: "fico_mid", label: "Credit score (FICO)", contributionPp: 2.18
  }, { feature: "acc_open_past_24mths", label: "Accounts opened, last 24m", contributionPp: 1.86 }, {
  feature: "mort_acc", label: "Mortgage accounts", contributionPp: 1.54 } ] },
{ loanId: 45414433, riskBand: "Critical", pd: 0.2019, ficoMid: 677, dti: 18.24, purpose:
  "debt_consolidation", loanAmnt: 6000, topDrivers: [{ feature: "acc_open_past_24mths", label: "Accounts
  opened, last 24m", contributionPp: 3.91 }, { feature: "num_tl_op_past_12m", label: "Trades opened, last
  12m", contributionPp: 2.25 }, { feature: "mths_since_recent_inq", label: "Months since last inquiry",
  contributionPp: 1.58 } ] },
{ loanId: 61349835, riskBand: "Critical", pd: 0.2151, ficoMid: 697, dti: 27.14, purpose:
  "debt_consolidation", loanAmnt: 10975, topDrivers: [{ feature: "loan_to_income", label: "Loan-to-income
  ratio", contributionPp: 4.43 }, { feature: "emp_length_years", label: "Employment length",
  contributionPp: -4.12 }, { feature: "mths_since_recent_inq", label: "Months since last inquiry",
  contributionPp: 4.11 } ] },
{ loanId: 55960490, riskBand: "Critical", pd: 0.2039, ficoMid: 682, dti: 19.98, purpose: "credit_card",
  loanAmnt: 7200, topDrivers: [{ feature: "tot_hi_cred_lim", label: "Total high credit limit",
  contributionPp: 3.25 }, { feature: "purpose", label: "Loan purpose", contributionPp: -3.04 }, { feature:
  "dti", label: "Debt-to-income ratio", contributionPp: 2.98 } ] },
];

/* averages derived from the real book for the Portfolio metric line */
export const derived = {
  avgLoanSize: totals.exposure / totals.loans,
  avgInterestRate: 0.1199, // simulator_base mean int_rate
  totalLoans: totals.loans,
};

```

credence/src/data/demo.ts

Every DEMO number, generated by a seeded, deterministic PRNG (mulberry32) so the output is identical on every rebuild. `makeBorrowers()` uses a fully transparent, documented scoring function (not a black box) so the borrower table and its 'why is this risky' panel stay internally consistent. `makeMonthly()` generates a 36-month synthetic trend anchored near the real headline totals at the most recent month, so the picture stays plausible without claiming to be measured.

```

/**
 * DEMO data – synthetic, seeded, deterministic. Used only where the pipeline has no
 * row-level or longitudinal data: individual borrower applications, per-decision risk
 * attributions, and monthly time series. Every surface that renders this is badged
 * "DEMO DATA". Endpoints of the time series are anchored near the real headline values
 * so the picture stays coherent; nothing here is a measured result.
 */
import { bandFor } from "../lib/risk";
import type { Alert, Borrower, EmploymentType, MonthPoint, RiskDriver } from "./types";

/* ---- seeded RNG (mulberry32) ---- */
function rng(seed: number) {
  let a = seed >>> 0;
  return () => {
    a |= 0;
    a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

const pick = <T>(r: () => number, arr: T[]): T => arr[Math.floor(r() * arr.length)];
const between = (r: () => number, lo: number, hi: number) => lo + r() * (hi - lo);
const gauss = (r: () => number) => (r() + r() + r() + r() - 2) / 2; // ~N(0,1)-ish, bounded

const PURPOSES = [
  "Debt consolidation",
  "Credit card",
  "Home improvement",
  "Major purchase",
  "Medical",
  "Small business",
  "Car",
  "Moving",
];
const EMPLOYMENT: EmploymentType[] = [
  "Full-time",
  "Full-time",
  "Full-time",
  "Part-time",
  "Self-employed",
  "Contract",
  "Retired",
];
const TITLES = [
  "Operations Manager",
  "Registered Nurse",
  "Software Engineer",
  "Retail Supervisor",
  "Logistics Coordinator",
  "Account Executive",
  "Electrician",
  "Teacher",
  "Project Manager",
  "Customer Success Lead",
];

```

```

];

/**
 * Transparent scoring function. risk_score = base 22 plus driver contributions;
 * PD is a squashed function of the score. Drivers are the SAME terms, so the
 * "why is this borrower risky" panel is internally consistent with the score.
 */
function score(b: {
  dti: number;
  credit_score: number;
  delinq: number;
  emp_years: number;
  loan_to_income: number;
  util: number;
}): { risk_score: number; pd: number; drivers: RiskDriver[] } {
  const dtiC = Math.round((b.dti - 0.18) * 62); // ±
  const scoreC = Math.round((690 - b.credit_score) * 0.13);
  const delinqC = Math.round(b.delinq * 7);
  const empC = Math.round((3 - b.emp_years) * 1.6);
  const ltiC = Math.round((b.loan_to_income - 0.28) * 28);
  const utilC = Math.round((b.util - 0.4) * 18);
  const drivers: RiskDriver[] = [
    { factor: "Debt-to-income", contribution: dtiC, note: `DTI of ${b.dti * 100}.toFixed(0)}% against a book median near 18%.`, },
    { factor: "Credit score", contribution: scoreC, note: `FICO ${b.credit_score}; each 100 points below 690 adds ~13 points of risk.`, },
    { factor: "Recent delinquencies", contribution: delinqC, note: `${b.delinq} delinquency event(s) reported in the last 24 months.`, },
    { factor: "Employment tenure", contribution: empC, note: `${b.emp_years.toFixed(1)} years in current employment; short tenure raises risk.`, },
    { factor: "Loan-to-income", contribution: ltiC, note: `Requested amount is ${b.loan_to_income * 100}.toFixed(0)}% of annual income.`, },
    { factor: "Revolving utilisation", contribution: utilC, note: `${b.util * 100}.toFixed(0)}% of revolving credit in use.`, },
  ].sort((a, z) => Math.abs(z.contribution) - Math.abs(a.contribution));
  const raw = 24 + dtiC + scoreC + delinqC + empC + ltiC + utilC;
  const risk_score = Math.max(2, Math.min(99, Math.round(raw)));
  const pd = Math.max(0.01, Math.min(0.92, 1 / (1 + Math.exp(-(risk_score - 50) / 11))));
  return { risk_score, pd: Math.round(pd * 1000) / 1000, drivers };
}

export function makeBorrowers(seed = 20240611, n = 220): Borrower[] {
  const r = rng(seed);
  const out: Borrower[] = [];
  for (let i = 0; i < n; i++) {
    const credit_score = Math.round(Math.max(620, Math.min(820, 690 + gauss(r) * 46)));
    const annual_income = Math.round(Math.max(24000, Math.min(260000, Math.exp(between(r, 10.7, 11.9)) * (0.8 + r() * 0.5))), );
    const loan_amount = Math.round(Math.max(2000, Math.min(40000, annual_income * between(r, 0.08, 0.55))) / 500, ) * 500;
    const debt_to_income = Math.round(Math.max(2, Math.min(46, 16 + gauss(r) * 12)) * 10) / 10 / 100;
    const emp_years = Math.round(Math.max(0, Math.min(12, 5 + gauss(r) * 4)) * 10) / 10;
    const employment_type = pick(r, EMPLOYMENT);
  }
}

```

```

const delinquencies_2y = r() < 0.7 ? 0 : r() < 0.85 ? 1 : r() < 0.95 ? 2 : 3;
const util = Math.max(0.02, Math.min(1.1, 0.42 + gauss(r) * 0.24));
const credit_history_length = Math.round(Math.max(2, Math.min(32, 12 + gauss(r) * 7)));
const home_ownership = pick(r, ["Rent", "Rent", "Mortgage", "Mortgage", "Own"] as const);
const loan_purpose = pick(r, PURPOSES);
const term = (r() < 0.72 ? 36 : 60) as 36 | 60;
const loan_to_income = loan_amount / annual_income;
const { risk_score, pd, drivers } = score({
  dti: debt_to_income,
  credit_score,
  delinq: delinquencies_2y,
  emp_years,
  loan_to_income,
  util,
});
const interest_rate =
  Math.round((0.062 + pd * 0.42 + (term === 60 ? 0.012 : 0)) * 1000) / 1000;
const risk_category = bandFor(pd);
const expected_loss = Math.round(pd * loan_amount * 0.5);
const decision =
  pd < 0.12 ? "Approved" : pd < 0.2 ? "Manual review" : ("Declined" as const);
const month = 1 + Math.floor(r() * 12);
const application_date = `2018-${String(month).padStart(2, "0")}-${String(
  1 + Math.floor(r() * 27),
).padStart(2, "0")}`;
const loan_status =
  decision === "Declined"
    ? "Current"
    : r() < 0.06 + pd * 0.4
    ? r() < 0.5
    ? "Delinquent"
    : "Default"
    : r() < 0.25
    ? "Paid"
    : ("Current" as const);
out.push({
  loan_id: `LN-${(4820100 + i * 37).toString(36).toUpperCase()}`,
  borrower_id: `BR-${(9310 + i).toString().padStart(5, "0")}`,
  application_date,
  loan_amount,
  annual_income,
  employment_length: emp_years,
  employment_type,
  credit_score,
  debt_to_income,
  credit_history_length,
  home_ownership,
  loan_purpose,
  interest_rate,
  term,
  delinquencies_2y,
  risk_score,
  probability_of_default: pd,
  risk_category,
  expected_loss,

```

```

decision: decision as Borrower["decision"],
loan_status: loan_status as Borrower["loan_status"],
drivers,
});
}
return out;
}

export const borrowers = makeBorrowers();

/* pull one memorable title per borrower for the detail header, deterministic */
export function titleFor(b: Borrower): string {
const idx = parseInt(b.borrower_id.slice(3), 10) % TITLES.length;
return `${TITLES[idx]} · ${b.employment_type}`;
}

/* ---- monthly time series (36 months to 2018-12), anchored near real headlines ---- */
export function makeMonthly(seed = 71119): MonthPoint[] {
const r = rng(seed);
const out: MonthPoint[] = [];
let dr = 0.126;
let delq = 0.03;
let el = 452_000_000;
let cs = 703;
let dti = 0.169;
let psi = 0.03;
for (let i = 0; i < 36; i++) {
const y = 2016 + Math.floor(i / 12);
const m = (i % 12) + 1;
// a mild, broad deterioration through the last ~9 months
const late = i > 26 ? (i - 26) / 18 : 0;
dr = Math.max(0.1, dr + 0.00018 + late * 0.0009 + gauss(r) * 0.0012);
delq = Math.max(0.02, delq + 0.00012 + late * 0.0005 + gauss(r) * 0.0008);
el = Math.max(3e8, el + 1.1e6 + late * 2.6e6 + gauss(r) * 3e6);
cs = cs - 0.04 - late * 0.4 + gauss(r) * 0.55;
dti = dti + 0.00028 + late * 0.001 + gauss(r) * 0.0007;
psi = Math.max(0.01, psi + late * 0.011 + Math.abs(gauss(r)) * 0.0035);
out.push({
month: `${y}-${String(m).padStart(2, "0")}`,
default_rate: Math.round(dr * 10000) / 10000,
delinquency_rate: Math.round(delq * 10000) / 10000,
expected_loss: Math.round(el),
avg_credit_score: Math.round(cs * 10) / 10,
avg_dti: Math.round(dti * 10000) / 10000,
psi: Math.round(psi * 1000) / 1000,
});
}
// ease the final three months onto the real book-level headline (14.1% / $524.2M)
const targets = { dr: 0.1409, el: 524_155_631 };
for (let k = 3; k >= 1; k--) {
const row = out[out.length - k];
const w = (4 - k) / 4;
row.default_rate = Math.round((row.default_rate * (1 - w) + targets.dr * w) * 10000) / 10000;
row.expected_loss = Math.round(row.expected_loss * (1 - w) + targets.el * w);
}
}

```

```

return out;
}

export const monthly = makeMonthly();

/* ---- alerts (DEMO) ---- */
export const alerts: Alert[] = [
{
  id: "a1",
  severity: "High",
  title: "High-risk borrower share rising",
  detail:
    "Applications scoring PD  $\geq$  20% grew for a fourth straight month, concentrated in 36-month debt-consolidation loans.",
  magnitude: "+12.0% MoM",
  window: "last 30 days",
  segment: "Debt consolidation - 36-month",
  date: "2018-12-04",
},
{
  id: "a2",
  severity: "Medium",
  title: "Auto-loan defaults above range",
  detail:
    "Realised default rate on car loans ran 8.4% above its trailing-12-month average this month.",
  magnitude: "+8.4% vs T12M avg",
  window: "December",
  segment: "Car",
  date: "2018-12-02",
},
{
  id: "a3",
  severity: "Medium",
  title: "DTI drifting upward",
  detail:
    "Average applicant debt-to-income has climbed 1.6 points over the quarter; the shift is broad-based across income bands.",
  magnitude: "+1.6 pts QoQ",
  window: "Q4",
  segment: "All segments",
  date: "2018-11-27",
},
{
  id: "a4",
  severity: "Low",
  title: "Portfolio exposure within expected range",
  detail:
    "Total funded exposure held near plan; new originations are pacing to the quarterly target.",
  magnitude: "-0.3% vs plan",
  window: "December",
  segment: "Portfolio",
  date: "2018-12-05",
},
];

```

```

/* emerging-risk timeline entries (DEMO) */
export const emerging: Alert[] = [
{
  id: "e1",
  severity: "High",
  title: "Credit-score distribution deteriorating",
  detail: "The 25th percentile of applicant FICO fell 11 points since September; population stability index crossed 0.10.",
  magnitude: "PSI 0.12",
  window: "Sep - Dec",
  segment: "New applications",
  date: "2018-12-03",
},
{
  id: "e2",
  severity: "Medium",
  title: "Debt-to-income increasing",
  detail: "Mean DTI up 1.6 points over the quarter, steepening in November.",
  magnitude: "+1.6 pts",
  window: "Q4",
  segment: "All",
  date: "2018-11-20",
},
{
  id: "e3",
  severity: "Medium",
  title: "Predicted default probability rising",
  detail: "Mean model PD on the incoming pipeline moved from 13.1% to 14.4%.",
  magnitude: "+1.3 pts",
  window: "Oct - Dec",
  segment: "Pipeline",
  date: "2018-11-14",
},
{
  id: "e4",
  severity: "Low",
  title: "Small-business segment: abnormal delinquency",
  detail: "Early-stage delinquency on small-business loans ran two standard deviations above its mean for one month, then normalised.",
  magnitude: "2.1σ, one month",
  window: "October",
  segment: "Small business",
  date: "2018-10-30",
},
];

```

credence/src/lib/risk.ts

Risk-band definitions and the dark-theme colour tokens for each band — the single place both concept and colour are defined, referenced by every chart and pill in the app.

```

export type RiskBand = "Low" | "Medium" | "High" | "Critical";

export const BANDS: RiskBand[] = ["Low", "Medium", "High", "Critical"];

/** Default PD cut-offs; overridable in Settings. */
export const DEFAULT_CUTS = { medium: 0.05, high: 0.1, critical: 0.2 };

export function bandFor(
  pd: number,
  cuts: { medium: number; high: number; critical: number } = DEFAULT_CUTS,
): RiskBand {
  if (pd < cuts.medium) return "Low";
  if (pd < cuts.high) return "Medium";
  if (pd < cuts.critical) return "High";
  return "Critical";
}

export const RISK_VAR: Record<RiskBand, string> = {
  Low: "var(--risk-low)",
  Medium: "var(--risk-med)",
  High: "var(--risk-high)",
  Critical: "var(--risk-crit)",
};

export const RISK_WASH: Record<RiskBand, string> = {
  Low: "var(--risk-low-wash)",
  Medium: "var(--risk-med-wash)",
  High: "var(--risk-high-wash)",
  Critical: "var(--risk-crit-wash)",
};

export const RISK_HEX: Record<RiskBand, string> = {
  Low: "#3fa277",
  Medium: "#c79238",
  High: "#d2673b",
  Critical: "#c6453a",
};

export const BAND_RANGE_LABEL: Record<RiskBand, string> = {
  Low: "PD < 5%",
  Medium: "5-10%",
  High: "10-20%",
  Critical: "20%+",
};

```

credence/src/lib/format.ts

Small, consistent formatting helpers (currency compaction, percentages, signed deltas) used everywhere so numbers read the same way on every page.

```

/** Formatting helpers. Currency is USD (the pipeline's unit). */

export function usdCompact(n: number, opts: { decimals?: number } = {}): string {
  const d = opts.decimals ?? 1;
  const abs = Math.abs(n);
  if (abs >= 1e9) return `$$${(n / 1e9).toFixed(d)}B`;
  if (abs >= 1e6) return `$$${(n / 1e6).toFixed(d)}M`;
  if (abs >= 1e3) return `$$${(n / 1e3).toFixed(abs >= 1e5 ? 0 : d)}K`;
  return `$$${Math.round(n).toLocaleString("en-US")}`;
}

export function usd(n: number): string {
  return `$$${Math.round(n).toLocaleString("en-US")}`;
}

export function count(n: number): string {
  return Math.round(n).toLocaleString("en-US");
}

export function pct(n: number, decimals = 1): string {
  return ` ${(n * 100).toFixed(decimals)}%`;
}

export function pp(n: number, decimals = 1): string {
  // percentage points, signed
  const v = (n * 100).toFixed(decimals);
  return `${n > 0 ? "+" : ""}${v} pp`;
}

export function signedPct(n: number, decimals = 1): string {
  return `${n > 0 ? "+" : ""}${(n * 100).toFixed(decimals)}%`;
}

export function fixed(n: number, decimals = 2): string {
  return n.toFixed(decimals);
}

export function ordinalDecile(i: number): string {
  return ["1st", "2nd", "3rd", "4th", "5th", "6th", "7th", "8th", "9th", "10th"][i] ?? `${i + 1}th`;
}

```

credence/src/lib/curve.ts

`interpCurve()` linearly interpolates along the real 60-point backtest curve by PD cut-off, and — critically — clamps to the measured range rather than extrapolating past it. This is what makes the threshold simulator honest: it can only show you what was actually backtested.

```

import type { BacktestPoint } from "../data/types";

/**
 * Linear interpolation along the real out-of-time backtest curve by pd_cut.
 * Clamps to the observed range – never extrapolates past what the pipeline measured.
 */
export function interpCurve(points: BacktestPoint[], pdCut: number): BacktestPoint {
  const sorted = [...points].sort((a, b) => a.pd_cut - b.pd_cut);
  const lo = sorted[0];
  const hi = sorted[sorted.length - 1];
  if (pdCut <= lo.pd_cut) return lo;
  if (pdCut >= hi.pd_cut) return hi;
  let a = lo;
  let b = hi;
  for (let i = 0; i < sorted.length - 1; i++) {
    if (sorted[i].pd_cut <= pdCut && sorted[i + 1].pd_cut >= pdCut) {
      a = sorted[i];
      b = sorted[i + 1];
      break;
    }
  }
  const t = (pdCut - a.pd_cut) / (b.pd_cut - a.pd_cut || 1);
  const lerp = (k: keyof BacktestPoint) =>
    (a[k] as number) + t * ((b[k] as number) - (a[k] as number));
  return {
    pd_cut: pdCut,
    approval_rate: lerp("approval_rate"),
    pred_default_rate: lerp("pred_default_rate"),
    actual_default_rate: lerp("actual_default_rate"),
    model_expected_loss: lerp("model_expected_loss"),
    realized_credit_loss: lerp("realized_credit_loss"),
    model_profit: lerp("model_profit"),
    realized_profit: lerp("realized_profit"),
    n: Math.round(lerp("n")),
  };
}

export function curveExtent(points: BacktestPoint[], key: keyof BacktestPoint): [number, number] {
  const vals = points.map((p) => p[key] as number);
  return [Math.min(...vals), Math.max(...vals)];
}

```

credence/src/lib/scales.ts

Minimal linear-scale, tick-generation, and Catmull-Rom-to-Bezier smoothing helpers for the hand-built SVG charts — no charting library dependency.

```

/** Minimal linear scale + tick helpers for the hand-built SVG charts. */

export interface Scale {
  (v: number): number;
  domain: [number, number];
  range: [number, number];
  invert: (px: number) => number;
}

export function linear(domain: [number, number], range: [number, number]): Scale {
  const [d0, d1] = domain;
  const [r0, r1] = range;
  const span = d1 - d0 || 1;
  const fn = ((v: number) => r0 + ((v - d0) / span) * (r1 - r0)) as Scale;
  fn.domain = domain;
  fn.range = range;
  fn.invert = (px: number) => d0 + ((px - r0) / (r1 - r0)) * span;
  return fn;
}

/** "Nice" ticks – a small, readable set covering the domain. */
export function ticks(min: number, max: number, target = 5): number[] {
  if (min === max) return [min];
  const span = max - min;
  const step0 = Math.pow(10, Math.floor(Math.log10(span / target)));
  const err = (span / target) / step0;
  const step =
    err >= 7.5 ? step0 * 10 : err >= 3.5 ? step0 * 5 : err >= 1.5 ? step0 * 2 : step0;
  const start = Math.ceil(min / step) * step;
  const out: number[] = [];
  for (let v = start; v <= max + step * 1e-6; v += step) {
    out.push(Number(v.toFixed(10)));
  }
  return out;
}

export function niceMax(v: number): number {
  if (v <= 0) return 1;
  const mag = Math.pow(10, Math.floor(Math.log10(v)));
  const n = v / mag;
  const step = n <= 1 ? 1 : n <= 2 ? 2 : n <= 5 ? 5 : 10;
  return step * mag;
}

/** Catmull-Rom → cubic Bézier path for a smooth line through points. */
export function smoothPath(pts: [number, number][]): string {
  if (pts.length < 2) return "";
  const d = [`M ${pts[0][0]} ${pts[0][1]}`];
  for (let i = 0; i < pts.length - 1; i++) {
    const p0 = pts[i - 1] ?? pts[i];
    const p1 = pts[i];
    const p2 = pts[i + 1];
    const p3 = pts[i + 2] ?? p2;
    const c1x = p1[0] + (p2[0] - p0[0]) / 6;
    const c1y = p1[1] + (p2[1] - p0[1]) / 6;
    const c2x = p2[0] - (p3[0] - p1[0]) / 6;
    const c2y = p2[1] - (p3[1] - p1[1]) / 6;
    d.push(`C ${c1x} ${c1y} ${c2x} ${c2y} ${p2[0]} ${p2[1]}`);
  }
  return d.join(" ");
}

export function linePath(pts: [number, number][]): string {
  return pts.map((p, i) => `${i ? "L" : "M"} ${p[0]} ${p[1]}`).join(" ");
}

```

credence/src/components/AppShell.tsx

The persistent left navigation rail and the shared PageHead (title + lede) used at the top of every page.

```

import type { ReactNode } from "react";
import { NavLink } from "react-router-dom";
import { Icon, type IconName } from "../Icon";

const PRIMARY: { to: string; label: string; icon: IconName }[] = [
  { to: "/", label: "Overview", icon: "overview" },
  { to: "/portfolio", label: "Portfolio", icon: "portfolio" },
  { to: "/borrowers", label: "Borrowers", icon: "borrowers" },
  { to: "/decisions", label: "Decisions", icon: "decisions" },
  { to: "/monitoring", label: "Monitoring", icon: "monitoring" },
];

const FOOTER: { to: string; label: string; icon: IconName }[] = [
  { to: "/settings", label: "Settings", icon: "settings" },
  { to: "/data-model", label: "Data / Model", icon: "info" },
];

function Item({ to, label, icon }: { to: string; label: string; icon: IconName }) {
  return (
    <NavLink to={to} end={to === "/"} className="navitem">
      <Icon name={icon} size={19} />
      <span>{label}</span>
    </NavLink>
  );
}

export function AppShell({ children }: { children: ReactNode }) {
  return (
    <div className="shell">
      <nav className="rail" aria-label="Primary">
        <div className="rail__mark">
          <b>CRENDENCE</b>
          <span>Risk Intelligence</span>
        </div>
        <div className="rail__group">
          {PRIMARY.map((i) => (
            <Item key={i.to} {...i} />
          ))}
        </div>
        <div className="rail__spacer" />
        <div className="rail__group">
          {FOOTER.map((i) => (
            <Item key={i.to} {...i} />
          ))}
        </div>
      </nav>
      <main className="main">{children}</main>
    </div>
  );
}

export function PageHead({
  title,
  lede,
  aside,

```

```

}): {
  title: string;
  lede?: ReactNode;
  aside?: ReactNode;
}) {
  return (
    <div className="pagehead">
      <div>
        <h1>{title}</h1>
        {lede && <p>{lede}</p>}
      </div>
      {aside && <div className="pagehead__aside">{aside}</div>}
    </div>
  );
}

```

credence/src/components/primitives.tsx

The base design-system components every page is built from: Stat (a KPI figure with label/sub/sparkline), Card, Section, Seg (segmented control), DemoBadge, RiskPill, Delta (a signed, colour-coded change indicator).

```
import type { ReactNode } from "react";
import { Icon } from "../Icon";

/* ----- Delta ----- */
export function Delta({
  value,
  format = (v) => `${v > 0 ? "+" : ""}${(v * 100).toFixed(1)}%`,
  /** does an increase read as bad? (risk metrics: yes) */
  invert = false,
  neutral = false,
}): {
  value: number;
  format?: (v: number) => string;
  invert?: boolean;
  neutral?: boolean;
}) {
  const tone = neutral
    ? "flat"
    : value === 0
    ? "flat"
    : (value > 0) !== invert
    ? "pos"
    : "neg";
  const up = value > 0;
  return (
    <span className={`delta delta-${tone}`}>
      {value !== 0 && <Icon name={up ? "arrowUp" : "arrowDown"} size={12} strokeWidth={2} />}
      {format(value)}
    </span>
  );
}

/* ----- Stat ----- */
export function Stat({
  label,
  figure,
  sub,
  spark,
}): {
  label: string;
  figure: ReactNode;
  sub?: ReactNode;
  spark?: ReactNode;
}) {
  return (
    <div>
      <div className="stat__label">{label}</div>
      <div className="stat__figure">{figure}</div>
      {sub && <div className="stat__sub">{sub}</div>}
      {spark && <div className="stat__spark">{spark}</div>}
    </div>
  );
}

/* ----- Section ----- */
```

```

export function Section({
  title,
  desc,
  aside,
  children,
}): {
  title: string;
  desc?: ReactNode;
  aside?: ReactNode;
  children: ReactNode;
} {
  return (
    <section className="section">
      <div className="section__head">
        <div>
          <h2>{title}</h2>
          {desc} && <p>{desc}</p>
        </div>
        {aside} && <div className="section__aside">{aside}</div>
      </div>
      {children}
    </section>
  );
}

/* ----- Card ----- */
export function Card({
  title,
  note,
  aside,
  variant,
  children,
}): {
  title?: string;
  note?: ReactNode;
  aside?: ReactNode;
  variant?: "inset" | "flush";
  children: ReactNode;
} {
  return (
    <div className={`card${variant ? ` card--${variant}` : ""}`>
      {(title || aside) && (
        <div style={{ display: "flex", justifyContent: "space-between", alignItems: "baseline", gap: 12 }}>
          <div>
            {title} && <div className="card__title">{title}</div>
            {note} && <div className="card__note">{note}</div>
          </div>
          {aside}
        </div>
      )}
      <div className={title ? "card__body" : undefined}>{children}</div>
    </div>
  );
}

```

```

/* ----- Segmented control ----- */
export function Seg<T extends string>({
  options,
  value,
  onChange,
  labels,
}: {
  options: readonly T[];
  value: T;
  onChange: (v: T) => void;
  labels?: Partial<Record<T, string>>;
}) {
  return (
    <div className="seg" role="group">
      {options.map((o) => (
        <button key={o} aria-pressed={o === value} onClick={() => onChange(o)}>
          {labels?.[o] ?? o}
        </button>
      ))}
    </div>
  );
}

/* ----- DemoBadge ----- */
export function DemoBadge({ what = "synthetic data" }: { what?: string }) {
  return (
    <span
      className="demobadge"
      title={`Demo data – ${what}. Not a measured result from the pipeline.`}
    >
      <i /> Demo data
    </span>
  );
}

/* ----- Switch ----- */
export function Switch({
  on,
  onToggle,
  label,
}: {
  on: boolean;
  onToggle: () => void;
  label: string;
}) {
  return (
    <button
      className="switch"
      role="switch"
      aria-checked={on}
      aria-pressed={on}
      aria-label={label}
      onClick={onToggle}
    />
  );
}

```

```

}

/* ----- RiskPill ----- */
import { RISK_HEX } from "../lib/risk";
import type { RiskBand } from "../lib/risk";
export function RiskPill({ band }: { band: RiskBand }) {
  return (
    <span className="pill">
      <i style={{ background: RISK_HEX[band] }} />
      {band}
    </span>
  );
}

export function Crumb({ children }: { children: ReactNode }) {
  return (
    <div className="crumb">
      <Icon name="chevronRight" size={12} /> {children}
    </div>
  );
}

```

credence/src/components/blocks.tsx

Composite, page-level blocks: RiskSpectrum (the Overview hero visualization — funded exposure by risk band as one proportional bar), FairnessSignal (the real 4/5ths-rule flag, added post-audit), AlertList/AlertTimeline (the DEMO monitoring feed).

```
import type { Alert, FairnessDim } from "../data/types";
import { RISK_HEX } from "../lib/risk";
import type { RiskBand } from "../lib/risk";
import { count, pct } from "../lib/format";

/* ----- RiskSpectrum — the hero ----- */
export function RiskSpectrum({
  bands,
}: {
  bands: { band: RiskBand; range: string; exposure: number; share: number; count: number }[];
}) {
  const elevated = bands.filter((b) => b.band === "High" || b.band === "Critical");
  const elevatedShare = elevated.reduce((s, b) => s + b.share, 0);
  const critical = bands.find((b) => b.band === "Critical");
  return (
    <div>
      <div className="spectrum" role="img" aria-label="Portfolio risk distribution by exposure">
        {bands.map((b) => (
          <div
            key={b.band}
            className="spectrum__seg"
            style={{ flex: `${b.share} 0 0`, background: RISK_HEX[b.band] }}
            title={`${b.band} — ${pct(b.share)} of exposure`}
          >
            <b>{b.band}</b>
            <span>{pct(b.share, 0)}</span>
          </div>
        ))}
      </div>
      <div className="spectrum__legend">
        {bands.map((b) => (
          <span className="spectrum__key" key={b.band}>
            <i style={{ background: RISK_HEX[b.band] }} />
            {b.band} · {b.range} · {count(b.count)} loans
          </span>
        ))}
      </div>
      <p className="spectrum__readout">
        <b>{pct(elevatedShare, 0)}</b> of funded exposure sits in <b>Elevated</b> risk or
        higher, and <b>{pct(critical?.share ?? 0, 0)}</b> ({count(critical?.count ?? 0)}{" "}
        loans) is <b>Critical</b> — PD at or above 20%, where the observed default rate is{" "}
        {pct(0.286, 0)}.
      </p>
    </div>
  );
}

/* ----- FairnessSignal — real 4/5ths-rule adverse-impact flag ----- */
const AIR_THRESHOLD = 0.8;

export function FairnessSignal({
  dims,
}: {
  dims: { key: string; label: string; dim: FairnessDim }[];
}) {
```

```

const failing = dims
  .filter((d) => d.dim.air < AIR_THRESHOLD)
  .map((d) => {
    const lo = d.dim.groups.reduce((a, b) => (a.approval_rate < b.approval_rate ? a : b));
    const hi = d.dim.groups.reduce((a, b) => (a.approval_rate > b.approval_rate ? a : b));
    return { ...d, lo, hi };
  });
if (!failing.length) return null;

return (
  <div>
    {failing.map((f) => {
      const critical = f.dim.air < 0.5;
      const color = critical ? RISK_HEX.Critical : RISK_HEX.High;
      return (
        <div key={f.key} className="grade-conc" style={{ gridTemplateColumns: "128px 1fr 1fr" }}>
          <span style={{ fontWeight: 600 }}>{f.label}</span>
          <div className="dualbar">
            <i style={{ width: `${Math.min(100, (f.dim.air / AIR_THRESHOLD) * 100)}%`, background: color }} />
          </div>
          <span>
            AIR <b style={{ color }}>{f.dim.air.toFixed(2)}</b> – {f.lo.group} approved at{" "}
            {pct(f.lo.approval_rate / f.hi.approval_rate, 0)} the rate of {f.hi.group}, below the
            0.80 threshold
          </span>
        </div>
      );
    })}
    <p className="note">
      {failing.length} of {dims.length} disparate-impact checks fail the 4/5ths rule (adverse-impact
      ratio < 0.80). Approved-book default rates stay within a few points across every group in
      each check – the gap sits in who gets approved, not in how those loans perform. This is a
      proxy check on income, region and home ownership, not a protected-class fair-lending test –
      Lending Club data carries no race, sex, age, or national-origin fields.
    </p>
  </div>
);
}

/* ----- AlertList – "needs attention" ----- */
function sevHex(s: Alert["severity"]) {
  return s === "High" ? RISK_HEX.High : s === "Medium" ? RISK_HEX.Medium : RISK_HEX.Low;
}
export function AlertList({ items }: { items: Alert[] }) {
  return (
    <div className="alerts">
      {items.map((a) => (
        <div className="alert" key={a.id}>
          <span className="alert__dot" style={{ background: sevHex(a.severity) }} />
          <div>
            <div className="alert__title">{a.title}</div>
            <div className="alert__detail">{a.detail}</div>
            <div className="alert__meta">
              <span style={{ color: sevHex(a.severity) }} className="alert__sev">

```

```

    {a.severity}
  </span>
  <span>{a.segment}</span>
  <span>{a.window}</span>
</div>
</div>
<div className="alert__mag">{a.magnitude}</div>
</div>
  )}
</div>
);
}

/* ----- AlertTimeline - emerging risk ----- */
export function AlertTimeline({ items }: { items: Alert[] }) {
  return (
    <div className="timeline">
      {items.map((a) => (
        <div className="timeline__item" key={a.id} style={{ color: sevHex(a.severity) }}>
          <div className="timeline__date">
            {new Date(a.date).toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric"
            })}
          </div>
          <div className="alert__title" style={{ marginTop: 2, color: "var(--ink)" }}>
            {a.title}
          </div>
          <div className="alert__detail">{a.detail}</div>
          <div className="alert__meta">
            <span className="alert__sev" style={{ color: sevHex(a.severity) }}>
              {a.severity}
            </span>
            <span>{a.segment}</span>
            <span>{a.window}</span>
            <span style={{ color: "var(--ink)", fontWeight: 600 }}>{a.magnitude}</span>
          </div>
        </div>
      ))}
    </div>
  );
}

```

credence/src/components/charts.tsx

Every chart in the app, hand-built as SVG (no charting library): Sparkline, LineChart (with hover crosshair and tooltip), BarStrip and BarPair (horizontal and grouped comparisons, with a `highlight` prop added post-audit for Portfolio's cross-filtering), CalibrationChart (predicted-vs-observed scatter with a 45-degree reference line), ContributionBars (diverging +/- bars for the borrower drawer), Distribution (a tinted histogram).

```

import { useRef, useState } from "react";
import { linear, smoothPath, ticks } from "../lib/scales";

function niceMax(v: number): number {
  if (v <= 0) return 1;
  const mag = Math.pow(10, Math.floor(Math.log10(v)));
  const n = v / mag;
  const step = n <= 1 ? 1 : n <= 1.5 ? 1.5 : n <= 2 ? 2 : n <= 3 ? 3 : n <= 5 ? 5 : n <= 7.5 ? 7.5 : 10;
  return step * mag;
}

/* ----- */
/* Sparkline – tiny real-data line, no axis */
/* ----- */
export function Sparkline({
  values,
  width = 96,
  height = 26,
  color = "var(--ink-3)",
}: {
  values: number[];
  width?: number;
  height?: number;
  color?: string;
}) {
  if (values.length < 2) return null;
  const min = Math.min(...values);
  const max = Math.max(...values);
  const x = linear([0, values.length - 1], [1, width - 1]);
  const y = linear([min, max], [height - 2, 2]);
  const pts = values.map((v, i) => [x(i), y(v)] as [number, number]);
  return (
    <svg width={width} height={height} aria-hidden="true">
      <path d={smoothPath(pts)} fill="none" stroke={color} strokeWidth={1.5} strokeLinecap="round" />
      <circle cx={pts[pts.length - 1][0]} cy={pts[pts.length - 1][1]} r={2} fill={color} />
    </svg>
  );
}

/* ----- */
/* LineChart – 1..3 series, minimal axes, hover crosshair */
/* ----- */
export interface Series {
  label: string;
  color: string;
  values: number[];
}
export function LineChart({
  x: xLabels,
  series,
  height = 240,
  yFormat = (v) => `${v}`,
  yFrom0 = true,
  markers,
  smooth = true,

```

```

    }: {
      x: string[];
      series: Series[];
      height?: number;
      yFormat?: (v: number) => string;
      yFrom0?: boolean;
      markers?: { at: number; label: string }[];
      smooth?: boolean;
    }) {
      const W = 800;
      const H = height;
      const m = { t: 12, r: 16, b: 26, l: 52 };
      const [hi, setHi] = useState<number | null>(null);
      const wrapRef = useRef<HTMLDivElement>(null);

      const all = series.flatMap((s) => s.values);
      const rawMin = Math.min(...all);
      const rawMax = Math.max(...all);
      let yMin: number;
      let yMax: number;
      if (yFrom0) {
        yMin = 0;
        yMax = niceMax(rawMax * 1.06);
      } else {
        const pad = (rawMax - rawMin) * 0.28 || rawMax * 0.08 || 1;
        yMin = Math.max(0, rawMin - pad);
        yMax = rawMax + pad;
      }
      const x = linear([0, xLabels.length - 1], [m.l, W - m.r]);
      const y = linear([yMin, yMax], [H - m.b, m.t]);
      const yTicks = ticks(yMin, yMax, 4);

      function onMove(e: React.PointerEvent<SVGSVGElement>) {
        const rect = e.currentTarget.getBoundingClientRect();
        const px = ((e.clientX - rect.left) / rect.width) * W;
        const idx = Math.round(x.invert(px));
        setHi(Math.max(0, Math.min(xLabels.length - 1, idx)));
      }

      const step = Math.ceil(xLabels.length / 7);

      return (
        <div className="chart" ref={wrapRef} style={{ position: "relative" }}>
          <svg viewBox={`0 0 ${W} ${H}`} onPointerMove={onMove} onPointerLeave={() => setHi(null)}>
            {yTicks.map((t) => (
              <g key={t}>
                <line x1={m.l} x2={W - m.r} y1={y(t)} y2={y(t)} className="chart__hairline" />
                <text x={m.l - 8} y={y(t) + 3} textAnchor="end" className="chart__axis">
                  {yFormat(t)}
                </text>
              </g>
            ))}
            <line x1={m.l} x2={W - m.r} y1={y(yMin)} y2={y(yMin)} className="chart__baseline" />
            {xLabels.map((lab, i) =>
              i % step === 0 || i === xLabels.length - 1 ? (

```

```

<text key={i} x={x(i)} y={H - 8} textAnchor="middle" className="chart__axis">
{lab}
</text>
) : null,
)}
{markers?.map((mk) => (
<g key={mk.label}>
<line x1={x(mk.at)} x2={x(mk.at)} y1={m.t} y2={H - m.b} stroke="var(--line-strong)"
strokeDasharray="3 3" />
<text x={x(mk.at)} y={m.t + 2} textAnchor="middle" className="chart__label" dy={-2}>
{mk.label}
</text>
</g>
)})
{series.map((s) => {
const pts = s.values.map((v, i) => [x(i), y(v)] as [number, number]);
return (
<path
key={s.label}
d={smooth ? smoothPath(pts) : pts.map((p, i) => `${i ? "L" : "M"} ${p[0]} ${p[1]}`).join(" ")
className="chart__series"
stroke={s.color}
/>
);
)}}
{hi !== null && (
<>
<line x1={x(hi)} x2={x(hi)} y1={m.t} y2={H - m.b} className="chart__crosshair" />
{series.map((s) => (
<circle key={s.label} cx={x(hi)} cy={y(s.values[hi])} r={3.5} fill={s.color}
className="chart__dot" />
))}
</>
)}
</svg>
{hi !== null && (
<div className="tooltip" style={{ left: `${(x(hi) / W) * 100}%`, top:
`${(y(Math.max(...series.map((s) => s.values[hi]))) / H) * 100}%` }}>
<b>{xLabels[hi]}</b>
{series.map((s) => (
<div className="tooltip__row" key={s.label}>
<i style={{ background: s.color }} />
{s.label} {yFormat(s.values[hi])}
</div>
))}
</div>
)}
</div>
);
}

/* ----- */
/* BarStrip – compact horizontal bars (where risk concentrates) */
/* ----- */
export function BarStrip({

```

```

rows,
valueFormat,
color = "var(--ink-2)",
onSelect,
highlight,
}: {
  rows: { name: string; value: number; sub?: string }[];
  valueFormat: (v: number) => string;
  color?: string;
  onSelect?: (name: string) => void;
  /** when set, dims every row that doesn't match (case-insensitive substring) */
  highlight?: string | null;
}) {
  const max = Math.max(...rows.map((r) => r.value));
  const hl = highlight?.toLowerCase();
  return (
    <div>
      {rows.map((r) => {
        const matched = !hl || r.name.toLowerCase().includes(hl);
        return (
          <div
            key={r.name}
            className={`strip__row${onSelect ? " strip__row--link" : ""}`}
            onClick={onSelect ? () => onSelect(r.name) : undefined}
            role={onSelect ? "button" : undefined}
            tabIndex={onSelect ? 0 : undefined}
            onKeyDown={onSelect ? (e) => e.key === "Enter" && onSelect(r.name) : undefined}
            style={{ opacity: matched ? 1 : 0.35, transition: "opacity var(--t-fade) var(--ease)" }}
          >
            <span className="strip__name" style={{ fontWeight: hl && matched ? 600 : undefined }}>
              {r.name}
            </span>
            <span className="strip__track">
              <span className="strip__fill" style={{ width: `${(r.value / max) * 100}%`, background: color }} />
            </span>
            <span className="strip__val">
              {valueFormat(r.value)}
              {r.sub ? ` · ${r.sub}` : ""}
            </span>
          </div>
        );
      })}
    </div>
  );
}

/* ----- */
/* BarPair – grouped vertical bars (approval vs rejection etc.) */
/* ----- */
export function BarPair({
  groups,
  a,
  b,
  yFormat = (v) => `${(v * 100).toFixed(0)}%`,

```

```

height = 220,
}: {
  groups: string[];
  a: { label: string; color: string; values: number[] };
  b: { label: string; color: string; values: number[] };
  yFormat?: (v: number) => string;
  height?: number;
}) {
  const W = 760;
  const H = height;
  const m = { t: 12, r: 12, b: 30, l: 46 };
  const [hi, setHi] = useState<number | null>(null);
  const yMax = niceMax(Math.max(...a.values, ...b.values) * 1.05);
  const y = linear([0, yMax], [H - m.b, m.t]);
  const bandW = (W - m.l - m.r) / groups.length;
  const barW = Math.min(26, bandW * 0.3);
  const yTicks = ticks(0, yMax, 4);
  return (
    <div className="chart" style={{ position: "relative" }}>
      <svg viewBox={`0 0 ${W} ${H}`}>
        {yTicks.map((t) => (
          <g key={t}>
            <line x1={m.l} x2={W - m.r} y1={y(t)} y2={y(t)} className="chart__hairline" />
            <text x={m.l - 8} y={y(t) + 3} textAnchor="end" className="chart__axis">
              {yFormat(t)}
            </text>
          </g>
        ))}
        <line x1={m.l} x2={W - m.r} y1={y(0)} y2={y(0)} className="chart__baseline" />
        {groups.map((g, i) => {
          const cx = m.l + bandW * i + bandW / 2;
          return (
            <g key={g} onPointerEnter={() => setHi(i)} onPointerLeave={() => setHi(null)}>
              <rect
                x={cx - barW - 2}
                y={y(a.values[i])}
                width={barW}
                height={y(0) - y(a.values[i])}
                fill={a.color}
                rx={2}
                opacity={hi === null || hi === i ? 1 : 0.4}
              />
              <rect
                x={cx + 2}
                y={y(b.values[i])}
                width={barW}
                height={y(0) - y(b.values[i])}
                fill={b.color}
                rx={2}
                opacity={hi === null || hi === i ? 1 : 0.4}
              />
              <text x={cx} y={H - 10} textAnchor="middle" className="chart__axis">
                {g}
              </text>
            </g>
          )
        })}
      </svg>
    </div>
  )
}

```

```

    });
  })
</svg>
<div style={{ display: "flex", gap: 18, marginTop: 8 }}>
  {[a, b].map((s) => (
    <span key={s.label} className="spectrum__key">
      <i style={{ background: s.color }} />
      {s.label}
    </span>
  ))}
</div>
{hi !== null && (
  <div
    className="tooltip"
    style={{ left: `${(m.l + bandw * hi + bandw / 2) / W} * 100}%`, top: "6%" }}
  >
    <b>{groups[hi]}</b>
    <div className="tooltip__row">
      <i style={{ background: a.color }} />
      {a.label} {yFormat(a.values[hi])}
    </div>
    <div className="tooltip__row">
      <i style={{ background: b.color }} />
      {b.label} {yFormat(b.values[hi])}
    </div>
  </div>
)}
</div>
);
}

/* ----- */
/* CalibrationChart – predicted vs observed dots + 45° reference */
/* ----- */
export function CalibrationChart({
  points,
  height = 300,
}: {
  points: { pred: number; obs: number; n: number }[];
  height?: number;
}) {
  const W = 480;
  const H = height;
  const m = { t: 14, r: 14, b: 40, l: 44 };
  const max = Math.max(...points.flatMap((p) => [p.pred, p.obs])) * 1.08;
  const x = linear([0, max], [m.l, W - m.r]);
  const y = linear([0, max], [H - m.b, m.t]);
  const t = ticks(0, max, 4);
  const [hi, setHi] = useState<number | null>(null);
  return (
    <div className="chart" style={{ position: "relative", maxWidth: W }}>
      <svg viewBox={`0 0 ${W} ${H}`}>
        {t.map((v) => (
          <g key={v}>
            <line x1={m.l} x2={W - m.r} y1={y(v)} y2={y(v)} className="chart__hairline" />

```

```

<text x={m.l - 6} y={y(v) + 3} textAnchor="end" className="chart__axis">
  {(v * 100).toFixed(0)}%
</text>
<text x={x(v)} y={H - 22} textAnchor="middle" className="chart__axis">
  {(v * 100).toFixed(0)}%
</text>
</g>
))}
<line x1={x(0)} y1={y(0)} x2={x(max)} y2={y(max)} stroke="var(--line-strong)" strokeDasharray="4 4"
/>
<path
d={points.map((p, i) => `${i ? "L" : "M"} ${x(p.pred)} ${y(p.obs)}`)}
fill="none"
stroke="var(--accent)"
strokeWidth={1.5}
opacity={0.5}
/>
{points.map((p, i) => (
<circle
key={i}
cx={x(p.pred)}
cy={y(p.obs)}
r={hi === i ? 5 : 4}
fill="var(--accent)"
className="chart__dot"
onPointerEnter={() => setHi(i)}
onPointerLeave={() => setHi(null)}
/>
))}
<text x={(W) / 2} y={H - 4} textAnchor="middle" className="chart__label">
Predicted default probability
</text>
<text
x={12}
y={H / 2}
textAnchor="middle"
className="chart__label"
transform={`rotate(-90 12 ${H / 2})`}
>
Observed default rate
</text>
</svg>
{hi !== null && (
<div className="tooltip" style={{ left: `${(x(points[hi].pred) / W) * 100}%`, top:
`${(y(points[hi].obs) / H) * 100}%` }}>
<b>Decile {hi + 1}</b>
<div>predicted {(points[hi].pred * 100).toFixed(1)}%</div>
<div>observed {(points[hi].obs * 100).toFixed(1)}%</div>
</div>
)}
</div>
);
}
/* ----- */

```

```

/* ContributionBars – diverging ± from a zero baseline */
/* ----- */
export function ContributionBars({
  rows,
  expanded,
  onToggle,
}: {
  rows: { factor: string; contribution: number; note: string }[];
  expanded: string | null;
  onToggle: (f: string) => void;
}) {
  const maxAbs = Math.max(...rows.map((r) => Math.abs(r.contribution)), 1);
  return (
    <div>
      {rows.map((r) => {
        const pos = r.contribution >= 0;
        const w = (Math.abs(r.contribution) / maxAbs) * 50;
        const open = expanded === r.factor;
        return (
          <div className="contrib__row" key={r.factor}>
            <div
              className="contrib__top"
              role="button"
              tabIndex={0}
              aria-expanded={open}
              onClick={() => onToggle(r.factor)}
              onKeyDown={(e) => e.key === "Enter" && onToggle(r.factor)}
            >
              <span className="contrib__name">{r.factor}</span>
              <span className={`contrib__val contrib__val--${pos ? "pos" : "neg"}`}>
                {pos ? "+" : "-"}
                {Math.abs(r.contribution)}
              </span>
            </div>
            <div className="contrib__bar">
              <span className="contrib__zero" style={{ left: "50%" }} />
              <i
                style={{
                  left: pos ? "50%" : `${50 - w}%`,
                  width: `${w}%`,
                  background: pos ? "var(--risk-high)" : "var(--risk-low)",
                }}
              />
            </div>
            {open && <div className="contrib__note">{r.note}</div>}
          </div>
        );
      })}
    </div>
  );
}

/* ----- */
/* Distribution – histogram (loan count by band) */
/* ----- */

```

```

export function Distribution({
  bins,
  color = "var(--ink-2)",
  height = 200,
  valueFormat = (v) => v.toLocaleString("en-US"),
  highlight,
}: {
  bins: { label: string; value: number; tint?: string }[];
  color?: string;
  height?: number;
  valueFormat?: (v: number) => string;
  /** when set, dims every bin whose label doesn't match */
  highlight?: string | null;
}) {
  const W = 760;
  const H = height;
  const m = { t: 10, r: 8, b: 28, l: 8 };
  const max = niceMax(Math.max(...bins.map((b) => b.value)));
  const bw = (W - m.l - m.r) / bins.length;
  const y = linear([0, max], [H - m.b, m.t]);
  const [hi, setHi] = useState<number | null>(null);
  return (
    <div className="chart" style={{ position: "relative" }}>
      <svg viewBox={`0 0 ${W} ${H}`}>
        <line x1={m.l} x2={W - m.r} y1={y(0)} y2={y(0)} className="chart__baseline" />
        {bins.map((b, i) => (
          <g key={b.label} onPointerEnter={() => setHi(i)} onPointerLeave={() => setHi(null)}>
            <rect
              x={m.l + bw * i + bw * 0.14}
              y={y(b.value)}
              width={bw * 0.72}
              height={y(0) - y(b.value)}
              fill={b.tint ?? color}
              rx={2}
              opacity={
                (hi === null || hi === i) && (!highlight || b.label === highlight)
                ? 1
                : hi === i
                ? 1
                : 0.3
              }
            />
            <text x={m.l + bw * i + bw / 2} y={H - 10} textAnchor="middle" className="chart__axis">
              {b.label}
            </text>
          </g>
        ))}
      </svg>
      {hi !== null && (
        <div className="tooltip" style={{ left: `${((m.l + bw * hi + bw / 2) / W) * 100}%`, top:
          `${(y(bins[hi].value) / H) * 100}%` }}>
          <b>{bins[hi].label}</b> {valueFormat(bins[hi].value)}
        </div>
      )}
    </div>
  );
}

```

[credence/src/pages/Overview.tsx](#)

The flagship page and a representative example of how every page is built: pull real aggregates from `data/real.ts`, demo series from `data/demo.ts`, compose them with the shared primitives/blocks/charts. Shows the KPI row, the risk-distribution hero, the real fair-lending signal (conditionally rendered only when a dimension actually fails), the risk-over-time trend, the concentration breakdown with click-through into Portfolio via URL search params, and the DEMO alert feed.

```

import { useMemo, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { PageHead } from "../components/AppShell";
import { AlertList, FairnessSignal, RiskSpectrum } from "../components/blocks";
import { LineChart, Sparkline } from "../components/charts";
import { Card, Delta, Seg, Section, Stat } from "../components/primitives";
import { DemoBadge } from "../components/primitives";
import { alerts, monthly } from "../data/demo";
import * as R from "../data/real";
import { count, pct, usdCompact } from "../lib/format";
import { RISK_HEX } from "../lib/risk";

const METRICS = ["Default rate", "Delinquency", "Expected loss"] as const;
const RANGES = ["3M", "6M", "1Y", "All"] as const;

export function Overview() {
  const nav = useNavigate();
  const [metric, setMetric] = useState<(typeof METRICS)[number]>("Default rate");
  const [range, setRange] = useState<(typeof RANGES)[number]>("1Y");

  const criticalLoans = R.byRiskBand[3].loans;
  const totalExposure = R.totals.exposure;

  const bands = R.byRiskBand.map((b) => ({
    band: b.band,
    range: b.range,
    exposure: b.exposure,
    count: b.loans,
    share: b.exposure / totalExposure,
  })));

  const series = useMemo(() => {
    const n = range === "3M" ? 3 : range === "6M" ? 6 : range === "1Y" ? 12 : monthly.length;
    const slice = monthly.slice(-n);
    const key =
      metric === "Default rate" ? "default_rate" : metric === "Delinquency" ? "delinquency_rate" :
      "expected_loss";
    return {
      x: slice.map((m) => m.month.slice(2).replace("-", "/")),
      values: slice.map((m) => m[key as "default_rate"]),
    };
  }, [metric, range]);

  const drTrend = monthly.slice(-12).map((m) => m.default_rate);
  const drDelta = drTrend[drTrend.length - 1] - drTrend[0];

  const fairnessDims = [
    { key: "income", label: "Income band", dim: R.fairness.income },
    { key: "region", label: "Region", dim: R.fairness.region },
    { key: "homeOwnership", label: "Home ownership", dim: R.fairness.homeOwnership },
  ];
  const fairnessFailing = fairnessDims.filter((d) => d.dim.air < 0.8).length;

  return (
    <div className="page">

```

```

<PageHead title="Credit Risk" lede="Portfolio health and risk intelligence – the whole book at a
glance." />

<div className="statrow">
  <Stat
    label="Portfolio exposure"
    figure={USDCompact(totalExposure)}
    sub={<span>{count(R.totals.loans)} loans · 2012-2016 vintages</span>}
  />
  <Stat
    label="Default rate"
    figure={pct(R.totals.observed_default_rate, 1)}
    sub={<Delta value={drDelta} invert format={v => `${v > 0 ? "+" : ""}${(v * 100).toFixed(1)} pts
/ 12m` } />}
  />
  <Stat
    label="High-risk accounts"
    figure={count(criticalLoans)}
    sub={<span>PD ≥ 20% · USDCompact(R.byRiskBand[3].exposure)} exposure</span>}
  />
  <Stat
    label="Expected loss"
    figure={USDCompact(R.totals.expected_loss)}
    sub={<span>{pct(R.totals.expected_loss / totalExposure, 1)} of exposure · LGD {pct(R.totals.lgd,
0)}</span>}
  />
  <Stat
    label="Risk trend"
    figure={
      <span style={{ color: drDelta > 0 ? RISK_HEX.High : RISK_HEX.Low, fontSize: 22 }}>
        {drDelta > 0 ? "↑" : "↓"} {Math.abs(drDelta * 100).toFixed(1)} pts
      </span>
    }
    sub={<span>default rate, trailing 12 months</span>}
    spark={<Sparkline values={drTrend} color={drDelta > 0 ? RISK_HEX.High : RISK_HEX.Low} />}
  />
</div>

<Section
  title="Portfolio risk distribution"
  desc="Funded exposure by model risk band. The single question this answers: how risky is the book
right now?"
  >
  <RiskSpectrum bands={bands} />
</Section>

{fairnessFailing > 0 && (
  <Section
    title="Fair-lending signal"
    desc="Adverse-impact check on the approval policy, against the pipeline's real fairness analysis."
    aside={
      <>
        <span className="tag tag--real">REAL</span>
        <Link to="/decisions" className="pill" style={{ marginLeft: 8 }}>
          Full breakdown →
      </>
    }
  >
  </Section>
)

```

```

</Link>
</>
}
>
<FairnessSignal dims={fairnessDims} />
</Section>
))

<Section
  title="Risk over time"
  desc="Direction of travel for the metrics a committee asks about first."
  aside={
    <>
      <DemoBadge what="monthly trend series" />
      <Seg options={METRICS} value={metric} onChange={setMetric} />
      <Seg options={RANGES} value={range} onChange={setRange} />
    </>
  }
  >
  <LineChart
    x={series.x}
    series={[
      {
        label: metric,
        color: metric === "Expected loss" ? "var(--accent)" : RISK_HEX.High,
        values: series.values,
      },
    ]}
    yFormat={(v) =>
      metric === "Expected loss" ? usdCompact(v, { decimals: 0 }) : `${(v * 100).toFixed(1)}%`
    }
    yFrom0={metric === "Expected loss"}
    height={230}
  />
</Section>

<Section
  title="Where risk is concentrated"
  desc="Expected-loss share against each dimension. Click a grade or purpose to open it in Portfolio."
  >
  <div className="stripset stripset--split">
    <div>
      <div className="strip__head">By grade – click to drill in</div>
      <ConcentrationStrips
        rows={R.byGrade.map((g) => ({ name: g.label, el: g.expected_loss, expo: g.exposure })))}
        onSelect={(name) => nav(`/portfolio?grade=${name.replace("Grade ", "")}`)}
      />
    </div>
    <div>
      <div className="strip__head">By loan purpose</div>
      <ConcentrationStrips
        rows={R.byPurpose.slice(0, 7).map((p) => ({ name: p.label, el: p.expected_loss, expo:
          p.exposure })))}
        onSelect={(name) => nav(`/portfolio?purpose=${encodeURIComponent(name)}`)}
      />
    </div>
  </div>

```

```

</div>
<div>
<div className="strip_head">By credit-score band</div>
<ConcentrationStrips
rows={R.byFicoBand.map((f) => ({ name: f.label, el: f.exposure * f.avg_pd * 0.5, expo:
f.exposure })))}
/>
</div>
<div>
<div className="strip_head">By origination vintage</div>
<ConcentrationStrips
rows={R.byVintage.map((v) => ({ name: v.label, el: v.expected_loss, expo: v.exposure })))}
/>
</div>
</div>
<p className="note">
Grades <b>C and D</b> carry {pct((R.byGrade[2].expected_loss + R.byGrade[3].expected_loss) /
R.totals.expected_loss, 0)}% of expected loss on {pct((R.byGrade[2].exposure + R.byGrade[3].exposure) / totalExposure, 0)} of
exposure – the book's
risk sits one notch below the middle.
</p>
</Section>

<Section title="Needs attention" desc="Movements outside the expected range, ordered by urgency."
aside={<DemoBadge what="monitoring alerts" />}>
<Card variant="flush">
<div style={{ padding: "0 var(--s-6)" }}>
<AlertList items={alerts} />
</div>
</Card>
</Section>
</div>
);
}

function ConcentrationStrips({
rows,
onSelect,
}): {
rows: { name: string; el: number; expo: number }[];
onSelect?: (name: string) => void;
} {
const totalEl = rows.reduce((s, r) => s + r.el, 0);
const totalExpo = rows.reduce((s, r) => s + r.expo, 0);
const max = Math.max(...rows.map((r) => r.el / totalEl));
return (
<div>
{rows.map((r) => {
const elShare = r.el / totalEl;
const expoShare = r.expo / totalExpo;
const ratio = elShare / expoShare;
return (
<div
key={r.name}

```

```

className={`strip__row${onSelect ? " strip__row--link" : ""}`}
onClick={onSelect ? () => onSelect(r.name) : undefined}
role={onSelect ? "button" : undefined}
tabIndex={onSelect ? 0 : undefined}
onKeyDown={onSelect ? (e) => e.key === "Enter" && onSelect(r.name) : undefined}
>
<span className="strip__name">{r.name}</span>
<span className="strip__track">
<span
  className="strip__fill"
  style={{
    width: `${(elShare / max) * 100}%`,
    background: ratio > 1.15 ? "var(--risk-high)" : ratio > 0.9 ? "var(--risk-med)" :
    "var(--ink-3)",
  }}
/>
</span>
<span className="strip__val">
{pct(elShare, 0)} of loss · {ratio.toFixed(2)}x
</span>
</div>
);
}}
</div>
);
}

```

The rest of CREDENCE (same patterns, not repeated in full)

File	What it does
credence/src/pages/Portfolio.tsx	Exposure/risk sliced by grade, purpose, FICO band, income band, vintage, region — six real, cross-filtering dimensions (added post-audit) plus the segmented LGD-by-grade panel.
credence/src/pages/Borrowers.tsx	Sortable/searchable table over the 220 synthetic borrowers; opens a detail drawer via a `?id=` URL param.
credence/src/components/DataTable.tsx	The generic sortable/searchable table component Borrowers.tsx is built on.
credence/src/components/BorrowerDrawer.tsx	The borrower detail slide-over: financial/credit/loan profile, the DEMO contribution-bar explanation, and a decision narrative now clearly labelled as the synthetic-set's own policy, distinct from the pipeline's real 15% backtested cut-off.
credence/src/pages/Decisions.tsx	The threshold simulator page: real backtest-driven KPIs (with the post-audit fix — 'high-risk approval' is now a correctly-derived 0%, not a broken formula), approval-vs-rejection by income band, predicted-vs-realised default/loss by decision.
credence/src/components/ThresholdSimulator.tsx	The interactive slider bound to `interpCurve()` — the centrepiece of Decisions, real backtest data throughout.
credence/src/pages/Monitoring.tsx	Trend tiles (DEMO), the real train-vs-test model-performance comparison, calibration, the real global feature-importance ranking, and the two post-audit additions: real retrospective PSI and the real per-loan attribution sample.

<code>credence/src/pages/DataModel.tsx</code>	The accountability page: dataset/model provenance, a full REAL/DEMO panel ledger, and a 'Known limitations' section naming exactly what this build does not fake and why.
<code>credence/src/pages/Settings.tsx</code>	Display density, reduced-motion, and the risk-band display cut-offs (explicitly separate from the real underwriting cut-off used in Decisions).
<code>credence/src/components/Icon.tsx</code>	The hand-drawn SVG icon set used throughout the nav rail and inline UI.
<code>credence/src/app/settings.tsx</code>	The React context backing Settings — density, reduced-motion, and risk-band cut-off state.

Interview Q&A; — Easiest to Hardest

Grouped in three tiers: foundational concepts, intermediate methodology questions, and advanced questions grounded in this exact codebase's specific choices, limitations, and history.

Foundational

Q1. What is "probability of default" (PD)?

The model's estimated chance that a given loan ends in default, expressed as a number between 0 and 1 (or 0-100%). It's a prediction made at or near origination, before the outcome is known — the central quantity almost everything else in credit risk is built from.

Q2. What are EAD and LGD, and how do they combine into Expected Loss?

EAD (exposure at default) is the dollar amount actually at risk if a loan defaults — here, the amount funded at origination. LGD (loss given default) is the share of that exposure the lender doesn't recover. Expected Loss = PD x EAD x LGD: the probability of the bad outcome, times how much money is exposed, times how much of it is actually lost when it happens.

Q3. What's the difference between a default and a delinquency?

Delinquency is being behind on payments — an early-warning state a loan can recover from. Default is a defined, typically terminal, negative outcome (in this project: 'Charged Off' or 'Default' status) after which the lender treats the loan as a loss. Delinquency often precedes default but doesn't guarantee it.

Q4. What is AUC / the ROC curve, in plain terms?

AUC (area under the ROC curve) measures how well a model ranks positive cases above negative ones, from 0.5 (no better than random) to 1.0 (perfect separation). Concretely: if you picked one defaulter and one non-defaulter at random, AUC is the probability the model scores the defaulter as riskier.

Q5. Why do we split data into train and test sets?

To measure how a model performs on data it hasn't seen, which is the only honest proxy for how it will perform in the real world. A model evaluated only on the data it was trained on will always look better than it actually is — it can memorize quirks of that specific data rather than learning a generalizable pattern (overfitting).

Q6. What's a confusion matrix, and what are precision and recall?

A 2x2 table of predicted-vs-actual outcomes at a chosen decision threshold: true positives, false positives, true negatives, false negatives. Precision = $TP / (TP + FP)$ — of everything you flagged, how much was actually positive. Recall = $TP / (TP + FN)$ — of everything that was actually positive, how much did you catch. They trade off against each other as you move the threshold.

Q7. What is class imbalance, and why would it matter for credit risk?

When one outcome is much rarer than the other — here, defaults are roughly 14-15% of loans, non-defaults the rest. A naive model can get high 'accuracy' by just predicting the majority class, so accuracy is a poor metric; you need metrics that account for the imbalance (AUC, precision/recall, KS) and a deliberate decision about whether to rebalance the training data at all.

Q8. Why is predicting default a classification problem, not regression?

The outcome being predicted (defaulted / didn't default) is categorical, not a continuous number — so the model estimates a probability of class membership (classification) rather than a numeric value (regression). Regression would apply if you were predicting something continuous, like the eventual loss AMOUNT rather than whether a loss happens.

Q9. What is a JOIN in SQL, and can you name a couple of types?

A JOIN combines rows from two tables based on a matching key. INNER JOIN keeps only rows with a match in both tables; LEFT JOIN keeps every row from the left table and fills unmatched right-side columns with NULL. This project uses LEFT JOINS, for example, to attach a grade-specific LGD to a loan while still keeping loans whose grade has no LGD estimate — falling back to a default value via COALESCE.

Q10. What is feature engineering?

Deriving new input variables from the raw data that make patterns easier for a model to learn — for example, this project computes loan-to-income (loan amount divided by income) rather than leaving the model to infer that ratio from the two raw numbers separately.

Intermediate

Q11. Why use an out-of-time split instead of random train/test split here?

Credit risk is inherently time-dependent — a model is always used to score FUTURE applicants based on PAST data. An out-of-time split (train on loans issued before a cutoff date, test on loans issued after) mirrors that real deployment condition. A random split would let information from the same time period leak between train and test, overstating how well the model will generalize to genuinely new loans.

Q12. What is data leakage? Give an example relevant to credit scoring.

When information that wouldn't actually be available at prediction time ends up in the training data, making the model look better than it really is. Classic example: including a loan's interest rate as a feature when that rate was SET by the lender's own risk grade — the model would just be learning to reverse-engineer an existing decision, not predicting risk independently. This project explicitly excludes grade/sub_grade/int_rate from the model for exactly this reason.

Q13. What is model calibration, and why does it matter for a probability model?

Calibration means a stated probability is actually true on average — if a model says a group of loans has 15% PD, about 15% of them should actually default. A model can rank risk well (good AUC) while being badly calibrated (systematically over- or under-stating probabilities) — and if you're using PD to compute dollar figures like Expected Loss, calibration matters as much as ranking.

Q14. Why choose gradient boosting over logistic regression here (or vice versa)?

Gradient boosting can capture non-linear relationships and interactions between features automatically, which often gives it an edge over logistic regression's linear decision boundary on tabular data with many correlated bureau attributes. But it's less directly interpretable. The right answer here isn't 'boosting is always better' — it's 'test both and report the honest comparison,' which is exactly what this project does (HGB 0.691 vs logistic 0.680 vs grade-alone 0.669).

Q15. What is the DeLong test, and why would you use it here?

A statistical significance test for whether one model's AUC is genuinely higher than another's on the same sample, or just higher by chance. It matters because 'my AUC is 0.02 higher' means nothing on its own — a DeLong test turns that into a testable claim (this project: $z=19.6$, $p=3e-85$ vs the grade-alone baseline, so the improvement is real, not noise).

Q16. What is the Kolmogorov-Smirnov (KS) statistic in credit scoring?

The largest gap between the cumulative distribution of good loans and the cumulative distribution of bad loans, as you sort by model score. It's a classic, second view of discrimination alongside AUC — instead of comparing all pairs, it asks how well-separated the two groups are at their single most-different point.

Q17. Explain decile / lift analysis for a scored population.

Sort the population by predicted PD and split into ten equal-sized groups (deciles, worst to best). For each decile, compare the model's mean predicted PD to the actually observed default rate — this checks calibration group by group, not just on one blended average, and reveals things a single AUC number hides (this project's own finding: the top decile is systematically under-predicted, a documented limitation).

Q18. What is the 4/5ths rule in fair lending?

A standard regulatory screening guideline: take the approval rate of the lowest-approved group and divide by the approval rate of the highest-approved group (the adverse-impact ratio). A ratio below 0.80 is treated as worth investigating for disparate impact — a policy that's neutral on its face but produces a disproportionate outcome across groups.

Q19. What is Population Stability Index (PSI), and why do risk teams track it?

A measure of how much a population has drifted from some reference distribution — used to catch a model going stale because the world it's scoring has changed since it was trained. PSI under 0.10 is generally read as stable, 0.10-0.25 as a moderate shift worth watching, above 0.25 as a material shift. Without it, a model can silently degrade in production with no signal until losses show up.

Q20. Why wouldn't you just use accuracy to judge a PD model here?

With a ~14-15% base default rate, a model that predicts 'never defaults' for every loan would be about 85% accurate while being completely useless. Accuracy doesn't account for class imbalance or for the fact that missing a real default is far more costly than a false alarm — metrics like AUC, KS, precision/recall at a real decision threshold, and calibration all say something accuracy can't.

Advanced / project-specific

Q21. Walk me through this project's data pipeline end to end.

Raw CSV -> DuckDB (schema-agnostic load) -> staging (typing, dedup, quality/maturity filters) -> a three-way split into features/benchmark/outcome marts (the leakage firewall) -> model training on the features mart only, out-of-time split -> batch scoring -> Expected Loss computed once in the serving marts using grade-segmented LGD -> backtest, fairness, monitoring, and explainability all run as separate downstream stages against the same scored, split data -> everything exported as JSON/parquet artifacts the dashboard and BI layer read from.

Q22. How does this project prevent leakage, concretely?

Two independent layers. Architecturally, the SQL layer never puts a post-origination or benchmark-only field in the same table the model reads from — `mart_loan_features`, `mart_loan_benchmark`, and `mart_loan_outcomes` are physically separate tables. Programmatically, `src/features.py` asserts at training time that a FORBIDDEN column list is absent from the modelling frame, as a second, independent check on top of the schema separation — not instead of it.

Q23. Why is grade / int_rate excluded from the model but used in the benchmark?

Because they're Lending Club's OWN risk model's output — training a new model to predict default using another model's risk score as an input would be circular (the new model would partly just be learning to reproduce the old one, not independently assessing risk). They're still useful as a BENCHMARK — 'does this new model beat what the lender was already doing' — which is a legitimate, different question than 'is this a valid model input.'

Q24. This project's AUC is 0.69. Is that good? How do you know?

On its own, no single AUC number is 'good' or 'bad' — it has to be compared to a baseline. Here, the honest comparison is against Lending Club's own risk grade used as a naive PD estimate (AUC 0.669): the model beats it by 0.022 AUC, and a DeLong test confirms that gap is statistically real ($p=3e-85$), not noise. So: a real, modest, tested improvement over the existing baseline — not a dramatic one, and that's stated plainly rather than oversold.

Q25. Explain how LGD is estimated here, and why it's credibility-weighted.

LGD = 1 minus the recovery rate (principal repaid plus post-charge-off recoveries, over funded amount), computed on charged-off TRAIN loans, segmented by Lending Club grade. A raw per-grade average is unreliable for a grade with very few charged-off loans (grade G: 73) — a small sample can produce a noisy, untrustworthy estimate. Buhlmann credibility weighting blends the grade's own raw estimate with the flat portfolio LGD, in proportion to how much data actually supports that grade ($\text{credibility} = n / (n + K)$) — grades with abundant data mostly trust themselves, thin grades shrink toward the population estimate instead of reporting noise as signal.

Q26. Walk me through Expected Loss — where exactly is it computed and what feeds it?

Computed exactly once, in `sql/06_marts.sql`: $EL = PD$ (from `mart_scores`, the trained model's output) $\times$ EAD (funded amount) $\times$ LGD (grade-segmented, credibility-weighted, joined in from the `lgd_by_grade` table `src/lgd.py` writes). Every downstream consumer — the portfolio summary, the simulator, the backtest — reads this one EL column rather than recomputing it independently, so there's no risk of two different EL figures silently disagreeing.

Q27. What does the backtest actually validate, and what's its biggest limitation?

It validates that a given approval threshold, applied to real out-of-time test loans, would have produced the profit and default outcomes the model claims — using actual cash flows, not the model's self-reported predictions. The biggest limitation: every loan in the data was historically APPROVED by Lending Club, so there's no outcome data for a hypothetically-rejected population. Tightening the cutoff is well-validated by this backtest; loosening it extrapolates into applicants this data has never actually observed the outcome of.

Q28. Is the threshold simulator "optimizing" the lending policy? Why or why not?

No — and this is a deliberate, precise word choice, not a technicality. It's a brute-force sweep over 60 discrete cut-off points on a single lever (the PD threshold), reading off wherever realised profit happens to peak. A true optimizer would search a continuous space, potentially over multiple levers jointly, subject to constraints. What's here is correctly called a threshold sweep or backtest, not an optimization — using the stronger word would overclaim what the code actually does.

Q29. What did the fairness check find, and how confident are you in that finding?

Two of three proxy dimensions fail the 4/5ths rule: income band (adverse-impact ratio 0.42) and home ownership (0.68); region passes (0.96). Confidence is real, not just a point estimate — a 2000-sample bootstrap confidence interval, Bonferroni-corrected for testing three dimensions simultaneously, keeps the upper bound of both failing dimensions below the 0.80 threshold, so this isn't sampling noise. Also notable: approved-book default rates stay close across every group, so the disparity is in who gets approved, not in loan performance once approved.

Q30. Why is PSI here described as "retrospective," and why can't it be a live monitor?

Because it compares two already-existing historical cohorts (loans issued 2012-14 vs 2015-16) that the pipeline already has — it is NOT computed against a stream of live, continuously-scored applications, because no such stream exists in this project (there's no production system generating new applications to score). Calling it 'live monitoring' would be a real overclaim; calling it what it is — a genuine, real population-stability comparison between two real cohorts — is accurate and still useful, just bounded.

Q31. What is the per-loan attribution method here, and why isn't it called SHAP?

For a real test loan, each input feature is reset, one at a time, to the test population's typical value (median for numeric fields, mode for categorical), and the resulting change in the model's real predicted PD is recorded. That's a genuine, correct single-feature sensitivity on the real model. It is NOT called SHAP because true Shapley values require averaging a feature's marginal contribution over every possible ORDER features could be added in (a combinatorial calculation) — this method perturbs one feature at a time from the full baseline, which is simpler, real, and useful, but a different (and less complete) calculation. Precision in naming it avoids claiming a stronger guarantee than the method actually provides.

Q32. If you were handed a different lender's dataset tomorrow, what would break, and why?

Almost the entire SQL layer. `sql/02_staging.sql` references roughly 40 literal Lending-Club-specific column names directly (`fico_range_low`, `emp_length`, `loan_status` values like 'Charged Off', etc.) — a dataset with different column names or a different status vocabulary would fail immediately at the first staging query with a 'column does not exist' error, before any Python even runs. There's no schema-mapping or column-alias layer today; the project is honestly a single-dataset pipeline with configurable THRESHOLDS, not a general, schema-agnostic one.

Q33. What's the single biggest limitation of this project's Expected Loss number, and how would you fix it?

EAD is a static funded-amount snapshot from origination, not a time-varying outstanding balance — so EL systematically overstates exposure for any loan that's already made payments and paid down principal. The fix is a real EAD model (or at minimum, an amortization-schedule-based average outstanding balance) instead of a constant — a known, stated gap, not a silent one.

Q34. Two real bugs were found in the dashboard during an audit. What were they, and how would you have caught them earlier?

A 'high-risk approval' statistic computed as $(\text{band.loans} * \text{rate}) / \text{band.loans}$ — which algebraically cancels to just the overall approval rate regardless of which band you're looking at, a tautology that was never actually representing the Critical-band figure it claimed to. And an 'average risk score' hardcoded as a literal string that didn't match what the underlying synthetic-data generator actually produced when run. Both would have been caught by a small test asserting displayed derived statistics against the real output of their own data source — exactly the kind of test `tests/test_methodology.py` exists to add going forward.

Q35. What would you deliberately NOT try to build with this dataset, and why?

A risk-migration matrix and a macro/stress-testing engine. A migration matrix needs a loan's risk classification tracked over time (a month-by-month delinquency panel); this dataset only has an origination snapshot plus one terminal outcome — there's nothing to track migration FROM. A stress scenario needs a downturn in the training data to calibrate against; the 2012-16 window contains no recession, so any stress scenario would be pure, unvalidated extrapolation. Building either anyway would produce a confident-looking number with nothing real behind it — worse than stating the limitation plainly, which is what this project does instead.